O'REILLY®

Second Edition

Generative Deep Learning

Teaching Machines to Paint, Write, Compose, and Play



David Foster

Foreword by Karl Friston

O'REILLY®

Generative Deep Learning

Generative AI is the hottest topic in tech. This practical book teaches machine learning engineers and data scientists how to use TensorFlow and Keras to create impressive generative deep learning models from scratch, including variational autoencoders (VAEs), generative adversarial networks (GANs), Transformers, normalizing flows, energy-based models, and denoising diffusion models.

The book starts with the basics of deep learning and progresses to cutting-edge architectures. Through tips and tricks, you'll understand how to make your models learn more efficiently and become more creative.

- **Discover how VAEs can change facial expressions in photos**
- **Train GANs to generate images based on your own dataset**
- **Build diffusion models to produce new varieties of flowers**
- **Train your own GPT for text generation**
- **Learn how large language models like ChatGPT are trained**
- **Explore state-of-the-art architectures such as StyleGAN2 and ViT-VQGAN**
- **Compose polyphonic music using Transformers and MuseGAN**
- **Understand how generative world models can solve reinforcement learning tasks**
- **Dive into multimodal models such as DALL.E 2, Imagen, and Stable Diffusion**

This book also explores the future of generative AI and how individuals and companies can proactively begin to leverage this remarkable new technology to create competitive advantage.

"*Generative Deep Learning* is an accessible introduction to the deep learning toolkit for generative modeling. If you are a creative practitioner who loves to tinker with code and want to apply deep learning to your work, then this is the book for you."

—David Ha
Head of Strategy, Stability AI

"An excellent book that dives right into all of the major techniques behind state-of-the-art generative deep learning. An exciting exploration of one of the most fascinating domains in AI!"

—François Chollet
Creator of Keras

David Foster is the cofounder of Applied Data Science Partners.

AI / DEEP LEARNING

US $79.99 CAN $99.99
ISBN: 978-1-098-13418-1



Twitter: @oreillymedia
linkedin.com/company/oreilly-media
youtube.com/oreillymedia

Praise for *Generative Deep Learning*

Generative Deep Learning is an accessible introduction to the deep learning toolkit for generative modeling. If you are a creative practitioner who loves to tinker with code and want to apply deep learning to your work, then this is the book for you.

—David Ha, Head of Strategy, Stability AI

An excellent book that dives right into all of the major techniques behind state-of-the-art generative deep learning. You'll find intuitive explanations and clever analogies—backed by didactic, highly readable code examples. An exciting exploration of one of the most fascinating domains in AI!

—François Chollet, Creator of Keras

David Foster's explanations of complex concepts are clear and concise, enriched with intuitive visuals, code examples, and exercises. An excellent resource for students and practitioners!

—Suzana Ilić, Principal Program Manager Responsible AI, Microsoft Azure OpenAI

Generative AI is the next revolutionary step in AI technology that will have a massive impact on the world. This book provides a great introduction to this field and its incredible potential and potential risks.

—Connor Leahy, CEO at Conjecture and Cofounder of EleutherAI

Predicting the world means understanding the world—in all modalities. In that sense, generative AI is solving the very core of intelligence.

—Jonas Andrulis, Founder & CEO Aleph Alpha

Generative AI is reshaping countless industries and powering a new generation of creative tools. This book is the perfect way to get going with generative modeling and start building with this revolutionary technology yourself.

—Ed Newton-Rex, VP Audio at Stability AI and composer

David taught me everything I know about machine learning and has a knack for explaining the underlying concepts. *Generative Deep Learning* is my go-to resource for generative AI and lives on a shelf next to my work desk amongst my small collection of favorite technical books.

—Zack Thoutt, CPO at AutoSalesVelocity

Generative AI is likely to have a profound impact on society. This book gives an introduction to the field that is accessible without skimping on technical detail.

—Raza Habib, Cofounder of Humanloop

When people ask me how to get started with generative AI, I always recommend David's book. The second edition is awesome because it covers the strongest models, such as diffusion models and Transformers. Definitely a must-have for anyone interested in computational creativity!

—Dr. Tristan Behrens, AI Expert and AI Music Artist in Residence at KI Salon Heilbronn

Dense in tech knowledge, this is my number one go-to literature when I have ideas around generative AI. It should be on every data scientist's bookshelf.

—Martin Musiol, Founder of generativeAI.net

The book covers the full taxonomy of generative models in excellent detail. One of the best things I found about the book is that it covers the important theory behind the models as well as solidifying the reader's understanding with practical examples. I must point out that the chapter on GANs is one of the best explanations I have read and provides intuitive means to fine-tune your models. The book covers a wide range of generative AI modalities including text, image, and music. A great resource for anyone getting started with GenAI.

—Aishwarya Srinivasan, Data Scientist, Google Cloud

SECOND EDITION

Generative Deep Learning

Teaching Machines to Paint, Write, Compose, and Play

David Foster

Foreword by Karl Friston

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Generative Deep Learning
by David Foster

Copyright © 2023 Applied Data Science Partners Ltd. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Nicole Butterfield
Development Editor: Michele Cronin
Production Editor: Christopher Faucher
Copyeditor: Charles Roumeliotis
Proofreader: Rachel Head

Indexer: Judith McConville
Interior Designer: David Futato
Cover Designer: Karen Montgomery
Illustrator: Kate Dullea

July 2019: First Edition
May 2023: Second Edition

Revision History for the Second Edition
2023-04-28: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781098134181* for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Generative Deep Learning*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-13418-1

[LSI]

For Alina, the loveliest noise vector of them all.

Table of Contents

Foreword xv

Preface xvii

Part I. Introduction to Generative Deep Learning

1. Generative Modeling 3
What Is Generative Modeling? 4
Generative Versus Discriminative Modeling 5
The Rise of Generative Modeling 6
Generative Modeling and AI 8
Our First Generative Model 9
Hello World! 9
The Generative Modeling Framework 10
Representation Learning 12
Core Probability Theory 15
Generative Model Taxonomy 18
The Generative Deep Learning Codebase 20
Cloning the Repository 20
Using Docker 21
Running on a GPU 21
Summary 21

2. Deep Learning 23
Data for Deep Learning 24
Deep Neural Networks 25

What Is a Neural Network? 25
Learning High-Level Features 26
TensorFlow and Keras 27
Multilayer Perceptron (MLP) 28
Preparing the Data 28
Building the Model 30
Compiling the Model 35
Training the Model 37
Evaluating the Model 38
Convolutional Neural Network (CNN) 40
Convolutional Layers 41
Batch Normalization 46
Dropout 49
Building the CNN 51
Training and Evaluating the CNN 53
Summary 54

Part II. Methods

3. Variational Autoencoders. 59
Introduction 60
Autoencoders 61
The Fashion-MNIST Dataset 62
The Autoencoder Architecture 63
The Encoder 64
The Decoder 65
Joining the Encoder to the Decoder 67
Reconstructing Images 69
Visualizing the Latent Space 70
Generating New Images 71
Variational Autoencoders 74
The Encoder 75
The Loss Function 80
Training the Variational Autoencoder 82
Analysis of the Variational Autoencoder 84
Exploring the Latent Space 85
The CelebA Dataset 85
Training the Variational Autoencoder 87
Analysis of the Variational Autoencoder 89
Generating New Faces 90

Latent Space Arithmetic 91
Morphing Between Faces 92
Summary 93

4. Generative Adversarial Networks. 95
Introduction 96
Deep Convolutional GAN (DCGAN) 97
The Bricks Dataset 98
The Discriminator 99
The Generator 101
Training the DCGAN 104
Analysis of the DCGAN 109
GAN Training: Tips and Tricks 110
Wasserstein GAN with Gradient Penalty (WGAN-GP) 113
Wasserstein Loss 114
The Lipschitz Constraint 115
Enforcing the Lipschitz Constraint 116
The Gradient Penalty Loss 117
Training the WGAN-GP 119
Analysis of the WGAN-GP 121
Conditional GAN (CGAN) 122
CGAN Architecture 123
Training the CGAN 124
Analysis of the CGAN 126
Summary 127

5. Autoregressive Models. 129
Introduction 130
Long Short-Term Memory Network (LSTM) 131
The Recipes Dataset 132
Working with Text Data 133
Tokenization 134
Creating the Training Set 137
The LSTM Architecture 138
The Embedding Layer 138
The LSTM Layer 140
The LSTM Cell 142
Training the LSTM 144
Analysis of the LSTM 146
Recurrent Neural Network (RNN) Extensions 149
Stacked Recurrent Networks 149

Gated Recurrent Units 151
Bidirectional Cells 153
PixelCNN 153
Masked Convolutional Layers 154
Residual Blocks 156
Training the PixelCNN 158
Analysis of the PixelCNN 159
Mixture Distributions 162
Summary 164

6. Normalizing Flow Models. 167
Introduction 168
Normalizing Flows 169
Change of Variables 170
The Jacobian Determinant 172
The Change of Variables Equation 173
RealNVP 174
The Two Moons Dataset 174
Coupling Layers 175
Training the RealNVP Model 181
Analysis of the RealNVP Model 184
Other Normalizing Flow Models 186
GLOW 186
FFJORD 187
Summary 188

7. Energy-Based Models. 189
Introduction 189
Energy-Based Models 191
The MNIST Dataset 192
The Energy Function 193
Sampling Using Langevin Dynamics 194
Training with Contrastive Divergence 197
Analysis of the Energy-Based Model 201
Other Energy-Based Models 202
Summary 203

8. Diffusion Models. 205
Introduction 206
Denoising Diffusion Models (DDM) 208
The Flowers Dataset 208

The Forward Diffusion Process 209
The Reparameterization Trick 210
Diffusion Schedules 211
The Reverse Diffusion Process 214
The U-Net Denoising Model 217
Training the Diffusion Model 224
Sampling from the Denoising Diffusion Model 225
Analysis of the Diffusion Model 228
Summary 231

Part III. Applications

9. Transformers. 235
Introduction 236
GPT 236
The Wine Reviews Dataset 237
Attention 238
Queries, Keys, and Values 239
Multihead Attention 241
Causal Masking 242
The Transformer Block 245
Positional Encoding 248
Training GPT 250
Analysis of GPT 252
Other Transformers 255
T5 256
GPT-3 and GPT-4 259
ChatGPT 260
Summary 264

10. Advanced GANs. 267
Introduction 268
ProGAN 269
Progressive Training 269
Outputs 276
StyleGAN 277
The Mapping Network 278
The Synthesis Network 279
Outputs from StyleGAN 280
StyleGAN2 281

Weight Modulation and Demodulation 282
Path Length Regularization 283
No Progressive Growing 284
Outputs from StyleGAN2 286
Other Important GANs 286
Self-Attention GAN (SAGAN) 286
BigGAN 288
VQ-GAN 289
ViT VQ-GAN 292
Summary 294

11. Music Generation. 297
Introduction 298
Transformers for Music Generation 299
The Bach Cello Suite Dataset 300
Parsing MIDI Files 300
Tokenization 303
Creating the Training Set 304
Sine Position Encoding 305
Multiple Inputs and Outputs 307
Analysis of the Music-Generating Transformer 309
Tokenization of Polyphonic Music 313
MuseGAN 317
The Bach Chorale Dataset 317
The MuseGAN Generator 320
The MuseGAN Critic 326
Analysis of the MuseGAN 327
Summary 329

12. World Models. 331
Introduction 331
Reinforcement Learning 332
The CarRacing Environment 334
World Model Overview 336
Architecture 336
Training 338
Collecting Random Rollout Data 339
Training the VAE 340
The VAE Architecture 341
Exploring the VAE 343
Collecting Data to Train the MDN-RNN 346

Training the MDN-RNN 346
The MDN-RNN Architecture 347
Sampling from the MDN-RNN 348
Training the Controller 348
The Controller Architecture 349
CMA-ES 349
Parallelizing CMA-ES 351
In-Dream Training 353
Summary 356

13. Multimodal Models. 359
Introduction 360
DALL.E 2 361
Architecture 362
The Text Encoder 362
CLIP 362
The Prior 367
The Decoder 369
Examples from DALL.E 2 373
Imagen 377
Architecture 377
DrawBench 378
Examples from Imagen 379
Stable Diffusion 380
Architecture 380
Examples from Stable Diffusion 381
Flamingo 381
Architecture 382
The Vision Encoder 382
The Perceiver Resampler 383
The Language Model 385
Examples from Flamingo 388
Summary 389

14. Conclusion. 391
Timeline of Generative AI 392
2014–2017: The VAE and GAN Era 394
2018–2019: The Transformer Era 394
2020–2022: The Big Model Era 395
The Current State of Generative AI 396
Large Language Models 396

Text-to-Code Models 400
Text-to-Image Models 402
Other Applications 405
The Future of Generative AI 407
Generative AI in Everyday Life 407
Generative AI in the Workplace 409
Generative AI in Education 410
Generative AI Ethics and Challenges 411
Final Thoughts 413

Index **417**

Foreword

This book is becoming part of my life. On finding a copy in my living room I asked my son, "When did you get this?" He replied, "When you gave it to me," bemused by my senior moment. Going through various sections together, I came to regard *Generative Deep Learning* as the *Gray's Anatomy* of Generative AI.

The author dissects the anatomy of generative AI with an incredible clarity and reassuring authority. He offers a truly remarkable account of a fast-moving field, underwritten with pragmatic examples, engaging narratives, and references that are so current, it reads like a living history.

Throughout his deconstructions, the author maintains a sense of wonder and excitement about the potential of generative AI—especially evident in the book's compelling dénouement. Having laid bare the technology, he reminds us that we are at the dawn of a new age of intelligence, an age in which generative AI holds a mirror up to our language, our art, our creativity; reflecting not just what we have created, but what we could create—what we can create—limited only by "your own imagination."

The central theme of generative models in artificial intelligence resonates deeply with me, because I see exactly the same themes emerging in the natural sciences; namely, a view of ourselves as generative models of our lived world. I suspect in the next edition of this book we will read about the confluence of artificial and natural intelligence. Until that time, I will keep this edition next to my copy of *Gray's Anatomy*, and other treasures on my bookshelf.

— Karl Friston, FRS
Professor of Neuroscience
University College London

Preface

> What I cannot create, I do not understand.
>
> —Richard Feynman

Generative AI is one of the most revolutionary technologies of our time, transforming the way we interact with machines. Its potential to revolutionize the way we live, work, and play has been the subject of countless conversations, debates, and predictions. But what if there was an even greater potential to this powerful technology? What if the possibilities of generative AI extend beyond our current imagination? The future of generative AI may be more exciting than we ever thought possible…

Since our earliest days, we have sought opportunities to generate original and beautiful creations. For early humans, this took the form of cave paintings depicting wild animals and abstract patterns, created with pigments placed carefully and methodically onto rock. The Romantic Era gave us the mastery of Tchaikovsky symphonies, with their ability to inspire feelings of triumph and tragedy through sound waves, woven together to form beautiful melodies and harmonies. And in recent times, we have found ourselves rushing to bookshops at midnight to buy stories about a fictional wizard, because the combination of letters creates a narrative that wills us to turn the page and find out what happens to our hero.

It is therefore not surprising that humanity has started to ask the ultimate question of creativity: can we create something that is in itself creative?

This is the question that generative AI aims to answer. With recent advances in methodology and technology, we are now able to build machines that can paint original artwork in a given style, write coherent blocks of text with long-term structure, compose music that is pleasant to listen to, and develop winning strategies for complex games by generating imaginary future scenarios. This is just the start of a generative revolution that will leave us with no choice but to find answers to some of the biggest questions about the mechanics of creativity, and ultimately, what it means to be human.

In short, there has never been a better time to learn about generative AI—so let's get started!

Objective and Approach

This book assumes no prior knowledge of generative AI. We will build up all of the key concepts from scratch in a way that is intuitive and easy to follow, so don't worry if you have no experience with generative AI. You have come to the right place!

Rather than only covering the techniques that are currently in vogue, this book serves as a complete guide to generative modeling that covers a broad range of model families. There is no one technique that is objectively *better* or *worse* than any other—in fact, many state-of-the-art models now mix together ideas from across the broad spectrum of approaches to generative modeling. For this reason, it is important to keep abreast of developments across all areas of generative AI, rather than focusing on one particular kind of technique. One thing is certain: the field of generative AI is moving fast, and you never know where the next groundbreaking idea will come from!

With this in mind, the approach I will take is to show you how to train your own generative models on your own data, rather than relying on pre-trained off-the-shelf models. While there are now many impressive open source generative models that can be downloaded and run in a few lines of code, the aim of this book is to dig deeper into their architecture and design from first principles, so that you gain a complete understanding of how they work and can code up examples of each technique from scratch using Python and Keras.

In summary, this book can be thought of as a map of the current generative AI landscape that covers both theory and practical applications, including full working examples of key models from the literature. We will walk through the code for each step by step, with clear signposts that show how the code implements the theory underpinning each technique. This book can be read cover to cover or used as a reference book that you can dip into. Above all, I hope you find it a useful and enjoyable read!



Throughout the book, you will find short, allegorical stories that help explain the mechanics of some of the models we will be building. I believe that one of the best ways to teach a new abstract theory is to first convert it into something that isn't quite so abstract, such as a story, before diving into the technical explanation. The story and the model explanation are just the same mechanics explained in two different domains—you might therefore find it useful to refer back to the relevant story while learning about the technical details of each model!

Prerequisites

This book assumes that you have experience coding in Python. If you are not familiar with Python, the best place to start is through LearnPython.org. There are many free resources online that will allow you to develop enough Python knowledge to work with the examples in this book.

Also, since some of the models are described using mathematical notation, it will be useful to have a solid understanding of linear algebra (for example, matrix multiplication) and general probability theory. A useful resource is Deisenroth et al.'s book *Mathematics for Machine Learning* (Cambridge University Press), which is freely available.

The book assumes no prior knowledge of generative modeling (we will examine the key concepts in Chapter 1) or TensorFlow and Keras (these libraries will be introduced in Chapter 2).

Roadmap

This book is divided into three parts.

Part I is a general introduction to generative modeling and deep learning, where we explore the core concepts that underpin all of the techniques in later parts of the book:

- In Chapter 1, "Generative Modeling", we define generative modeling and consider a toy example that we can use to understand some of the key concepts that are important to all generative models. We also lay out the taxonomy of generative model families that we will explore in Part II of this book.
- In Chapter 2, "Deep Learning", we begin our exploration of deep learning and neural networks by building our first example of a multilayer perceptron (MLP) using Keras. We then adapt this to include convolutional layers and other improvements, to observe the difference in performance.

Part II walks through the six key techniques that we will be using to build generative models, with practical examples for each:

- In Chapter 3, "Variational Autoencoders", we consider the variational autoencoder (VAE) and see how it can be used to generate images of faces and morph between faces in the model's latent space.
- In Chapter 4, "Generative Adversarial Networks", we explore generative adversarial networks (GANs) for image generation, including deep convolutional GANs, conditional GANs, and improvements such as the Wasserstein GAN that make the training process more stable.

- In Chapter 5, "Autoregressive Models", we turn our attention to autoregressive models, starting with an introduction to recurrent neural networks such as long short-term memory networks (LSTMs) for text generation and PixelCNN for image generation.
- In Chapter 6, "Normalizing Flow Models", we focus on normalizing flows, including an intuitive theoretical exploration of the technique and a practical example of how to build a RealNVP model to generate images.
- In Chapter 7, "Energy-Based Models", we cover energy-based models, including important methods such as how to train using contrastive divergence and sample using Langevin dynamics.
- In Chapter 8, "Diffusion Models", we dive into a practical guide to building diffusion models, which drive many state-of-the-art image generation models such as DALL.E 2 and Stable Diffusion.

Finally, in Part III we build on these foundations to explore the inner workings of state-of-the-art models for image generation, writing, composing music, and model-based reinforcement learning:

- In Chapter 9, "Transformers", we explore the lineage and technical details of the StyleGAN models, as well as other state-of-the-art GANs for image generation such as VQ-GAN.
- In Chapter 10, "Advanced GANs", we consider the Transformer architecture, including a practical walkthrough for building your own version of GPT for text generation.
- In Chapter 11, "Music Generation", we turn our attention to music generation, including a guide to working with music data and application of techniques such as Transformers and MuseGAN.
- In Chapter 12, "World Models", we see how generative models can be used in the context of reinforcement learning, with the application of world models and Transformer-based methods.
- In Chapter 13, "Multimodal Models", we explain the inner workings of four state-of-the-art multimodal models that incorporate more than one type of data, including DALL.E 2, Imagen, and Stable Diffusion for text-to-image generation and Flamingo, a visual language model.
- In Chapter 14, "Conclusion", we recap the key milestones of generative AI to date and discuss the ways in which generative AI will revolutionize our daily lives in years to come.

Changes in the Second Edition

Thank you to everyone who read the first edition of this book—I am really pleased that so many of you have found it a useful resource and provided feedback on things that you would like to see in the second edition. The field of generative deep learning has progressed significantly since the first edition was published in 2019, so as well as refreshing the existing content I have added several new chapters to bring the material in line with the current state of the art.

The following is a summary of the main updates, in terms of the individual chapters and general book improvements:

- Chapter 1 now includes a section on the different families of generative models and a taxonomy of how they are related.
- Chapter 2 contains improved diagrams and more detailed explanations of key concepts.
- Chapter 3 is refreshed with a new worked example and accompanying explanations.
- Chapter 4 now includes an explanation of conditional GAN architectures.
- Chapter 5 now includes a section on autoregressive models for images (e.g., PixelCNN).
- Chapter 6 is an entirely new chapter, describing the RealNVP model.
- Chapter 7 is also a new chapter, focusing on techniques such as Langevin dynamics and contrastive divergence.
- Chapter 8 is a newly written chapter on denoising the diffusion models that power many of today's state-of-the-art applications.
- Chapter 9 is an expansion of the material provided in the conclusion of the first edition, with deeper focus on architectures of the various StyleGAN models and new material on VQ-GAN.
- Chapter 10 is a new chapter that explores the Transformer architecture in detail.
- Chapter 11 includes modern Transformer architectures, replacing the LSTM models from the first edition.
- Chapter 12 includes updated diagrams and descriptions, with a section on how this approach is informing state-of-the-art reinforcement learning today.
- Chapter 13 is a new chapter that explains in detail how impressive models like DALL.E 2, Imagen, Stable Diffusion, and Flamingo work.
- Chapter 14 is updated to reflect the outstanding progress in the field since the first edition and give a more complete and detailed view of where generative AI is heading in the future.

- All comments given as feedback to the first edition and typos identified have been addressed (to the best of my knowledge!).
- Chapter goals have been added at the start of each chapter, so that you can see the key topics covered in the chapter before you start reading.
- Some of the allegorical stories have been rewritten to be more concise and clear—I am pleased that so many readers have said that the stories have helped them to better understand the key concepts!
- The headings and subheadings of each chapter have been aligned so that is it clear which parts of the chapter are focused on explanation and which are focused on building your own models.

Other Resources

I highly recommend the following books as general introductions to machine learning and deep learning:

- *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* by Aurélien Géron (O'Reilly)
- *Deep Learning with Python* by Francois Chollet (Manning)

Most of the papers in this book are sourced through arXiv, a free repository of scientific research papers. It is now common for authors to post papers to arXiv before they are fully peer-reviewed. Reviewing the recent submissions is a great way to keep on top of the most cutting-edge developments in the field.

I also highly recommend the website Papers with Code, where you can find the latest state-of-the-art results in a variety of machine learning tasks, alongside links to the papers and official GitHub repositories. It is an excellent resource for anyone wanting to quickly understand which techniques are currently achieving the highest scores in a range of tasks and has certainly helped me to decide which techniques to include in this book.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`
: Used for commands and program listings, as well as within paragraphs to refer to program elements such as variable or function names.

`Constant width italic`
: Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element signifies a warning or caution.

Codebase

The code examples in this book can be found in a GitHub repository. I have deliberately ensured that none of the models require prohibitively large amounts of computational resources to train, so that you can start training your own models without having to spend lots of time or money on expensive hardware. There is a comprehensive guide in the repository on how to get started with Docker and set up cloud resources with GPUs on Google Cloud if required.

The following changes have been made to the codebase since the first edition:

- All examples are now runnable from within a single notebook, instead of some code being imported from modules across the codebase. This is so that you can run each example cell by cell and delve into exactly how each model is built, piece by piece.
- The sections of each notebook are now broadly aligned between examples.
- Many of the examples in this book now utilize code snippets from the amazing open source Keras repository—this is to avoid creating a completely detached open source repository of Keras generative AI examples, when there already exist excellent implementations available through the Keras website. I have added references and links to the original authors of code that I have utilized from the Keras website throughout this book and in the repository.

- I have added new data sources and improved the data collection process from the first edition—now, there is a script that can be easily run to collect data from the required sources in order to train the examples in the book, using tools such as the Kaggle API.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at *https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition*.

If you have a technical question or a problem using the code examples, please send email to *bookquestions@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Generative Deep Learning*, 2nd edition, by David Foster (O'Reilly). Copyright 2023 Applied Data Science Partners Ltd., 978-1-098-13418-1."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/generative-dl*.

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly-media*

Follow us on Twitter: *https://twitter.com/oreillymedia*

Watch us on YouTube: *https://youtube.com/oreillymedia*

Acknowledgments

There are so many people I would like to thank for helping me write this book.

First, I would like to thank everyone who has taken time to technically review the book—in particular Vishwesh Ravi Shrimali, Lipi Deepaakshi Patnaik, Luba Elliot, and Lorna Barclay. Thanks also to Samir Bico for helping to review and test the codebase that accompanies this book. Your input has been invaluable.

Also, a huge thanks to my colleagues at Applied Data Science Partners, Ross Witeszczak, Amy Bull, Ali Parandeh, Zine Eddine, Joe Rowe, Gerta Salillari, Aleshia Parkes, Evelina Kireilyte, Riccardo Tolli, Mai Do, Khaleel Syed, and Will Holmes. Your patience with me while I have taken time to finish the book is hugely appreciated, and I am greatly looking forward to all the machine learning projects we will complete together in the future! Particular thanks to Ross—had we not decided to start a business together, this book might never have taken shape, so thank you for believing in me as your business partner!

I also want to thank anyone who has ever taught me anything mathematical—I was extremely fortunate to have fantastic math teachers at school, who developed my interest in the subject and encouraged me to pursue it further at university. I would

like to thank you for your commitment and for going out of your way to share your knowledge of the subject with me.

A huge thank you goes to the staff at O'Reilly for guiding me through the process of writing this book. A special thanks goes to Michele Cronin, who has been there at each step, providing useful feedback and sending me friendly reminders to keep completing chapters! Also to Nicole Butterfield, Christopher Faucher, Charles Roumeliotis, and Suzanne Huston for getting the book into production, and Mike Loukides for first reaching out to ask if I'd be interested in writing a book. You have all been so supportive of this project from the start, and I want to thank you for providing me with a platform on which to write about something that I love.

Throughout the writing process, my family has been a constant source of encouragement and support. A huge thank you goes to my mum, Gillian Foster, for checking every single line of text for typos and for teaching me how to add up in the first place! Your attention to detail has been extremely helpful while proofreading this book, and I'm really grateful for all the opportunities that both you and dad have given me. My dad, Clive Foster, originally taught me how to program a computer—this book is full of practical examples, and that's thanks to his early patience while I fumbled around in BASIC trying to make football games as a teenager. My brother, Rob Foster, is the most modest genius you will ever find, particularly within linguistics—chatting with him about AI and the future of text-based machine learning has been amazingly helpful. Last, I would like to thank my Nana, who was always a constant source of inspiration and fun for all of us. Her love of literature was one of the reasons I first decided that writing a book would be an exciting thing to do.

I would also like to thank my wife, Lorna Barclay. As well as providing me with endless support and cups of tea throughout the writing process, you have rigorously checked every word of this book in meticulous detail. I couldn't have done it without you. Thank you for always being there for me, and for making this journey so much more enjoyable. I promise I won't talk about generative AI at the dinner table for at least a few days after the book is published.

Lastly, I would like to thank our beautiful baby daughter Alina for providing endless entertainment during the long nights of book-writing. Your adorable giggles have been the perfect background music to my typing. Thanks for being my inspiration and for always keeping me on my toes. You're the real brains behind this operation.

PART I

Introduction to Generative Deep Learning

Part I is a general introduction to generative modeling and deep learning—the two fields that we need to understand in order to get started with generative deep learning!

In Chapter 1 we will define generative modeling and consider a toy example that we can use to understand some of the key concepts that are important to all generative models. We will also lay out the taxonomy of generative model families that we will explore in Part II of this book.

Chapter 2 provides a guide to the deep learning tools and techniques that we will need to start building more complex generative models. In particular, we will build our first example of a deep neural network—a multilayer perceptron (MLP)—using Keras. We will then adapt this to include convolutional layers and other improvements, to observe the difference in performance.

By the end of Part I you will have a good understanding of the core concepts that underpin all of the techniques in later parts of the book.

CHAPTER 1
Generative Modeling

Chapter Goals

In this chapter you will:

- Learn the key differences between generative and discriminative models.
- Understand the desirable properties of a generative model through a simple example.
- Learn about the core probabilistic concepts that underpin generative models.
- Explore the different families of generative models.
- Clone the codebase that accompanies this book, so that you can get started building generative models!

This chapter is a general introduction to the field of generative modeling.

We will start with a gentle theoretical introduction to generative modeling and see how it is the natural counterpart to the more widely studied discriminative modeling. We will then establish a framework that describes the desirable properties that a good generative model should have. We will also lay out the core probabilistic concepts that are important to know, in order to fully appreciate how different approaches tackle the challenge of generative modeling.

This will lead us naturally to the penultimate section, which lays out the six broad families of generative models that dominate the field today. The final section explains how to get started with the codebase that accompanies this book.

What Is Generative Modeling?

Generative modeling can be broadly defined as follows:

> Generative modeling is a branch of machine learning that involves training a model to produce new data that is similar to a given dataset.

What does this mean in practice? Suppose we have a dataset containing photos of horses. We can *train* a generative model on this dataset to capture the rules that govern the complex relationships between pixels in images of horses. Then we can *sample* from this model to create novel, realistic images of horses that did not exist in the original dataset. This process is illustrated in Figure 1-1.

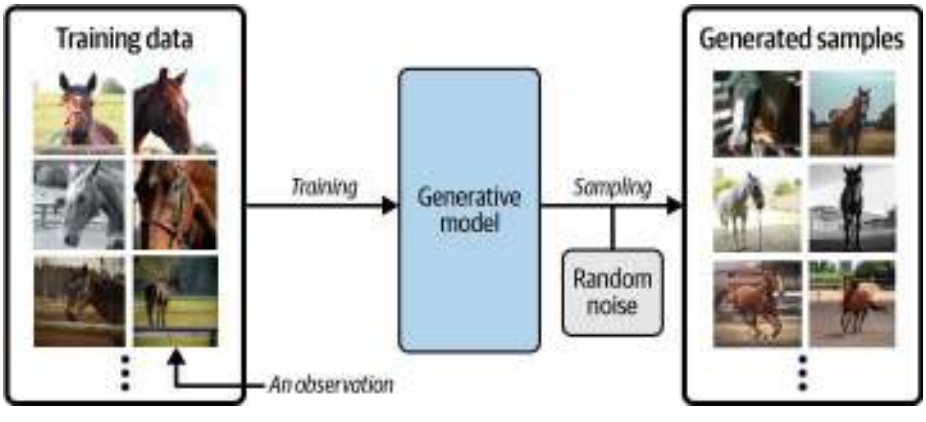


Figure 1-1. A generative model trained to generate realistic photos of horses

In order to build a generative model, we require a dataset consisting of many examples of the entity we are trying to generate. This is known as the *training data*, and one such data point is called an *observation*.

Each observation consists of many *features*. For an image generation problem, the features are usually the individual pixel values; for a text generation problem, the features could be individual words or groups of letters. It is our goal to build a model that can generate new sets of features that look as if they have been created using the same rules as the original data. Conceptually, for image generation this is an incredibly difficult task, considering the vast number of ways that individual pixel values can be assigned and the relatively tiny number of such arrangements that constitute an image of the entity we are trying to generate.

A generative model must also be *probabilistic* rather than *deterministic*, because we want to be able to sample many different variations of the output, rather than get the same output every time. If our model is merely a fixed calculation, such as taking the average value of each pixel in the training dataset, it is not generative. A generative

model must include a random component that influences the individual samples generated by the model.

In other words, we can imagine that there is some unknown probabilistic distribution that explains why some images are likely to be found in the training dataset and other images are not. It is our job to build a model that mimics this distribution as closely as possible and then sample from it to generate new, distinct observations that look as if they could have been included in the original training set.

Generative Versus Discriminative Modeling

In order to truly understand what generative modeling aims to achieve and why this is important, it is useful to compare it to its counterpart, *discriminative modeling*. If you have studied machine learning, most problems you will have faced will have most likely been discriminative in nature. To understand the difference, let's look at an example.

Suppose we have a dataset of paintings, some painted by Van Gogh and some by other artists. With enough data, we could train a discriminative model to predict if a given painting was painted by Van Gogh. Our model would learn that certain colors, shapes, and textures are more likely to indicate that a painting is by the Dutch master, and for paintings with these features, the model would upweight its prediction accordingly. Figure 1-2 shows the discriminative modeling process—note how it differs from the generative modeling process shown in Figure 1-1.

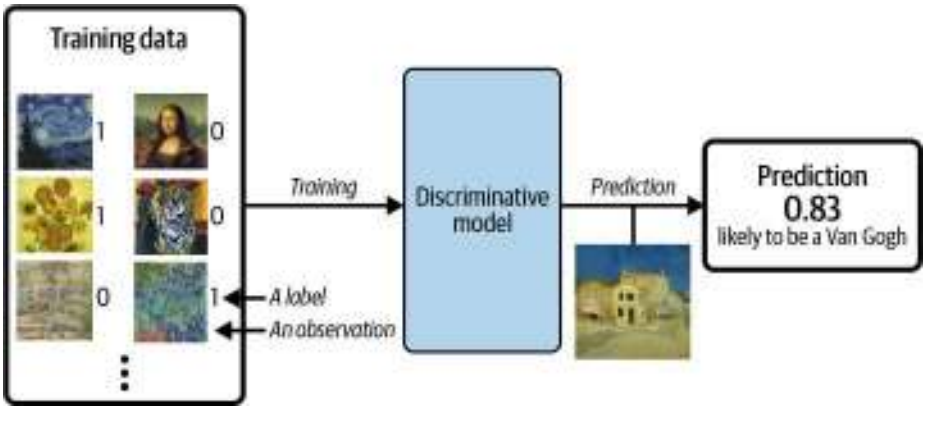


Figure 1-2. A discriminative model trained to predict if a given image is painted by Van Gogh

When performing discriminative modeling, each observation in the training data has a *label*. For a binary classification problem such as our artist discriminator, Van Gogh paintings would be labeled 1 and non–Van Gogh paintings labeled 0. Our model then

learns how to discriminate between these two groups and outputs the probability that a new observation has label 1—i.e., that it was painted by Van Gogh.

In contrast, generative modeling doesn't require the dataset to be labeled because it concerns itself with generating entirely new images, rather than trying to predict a label of a given image.

Let's define these types of modeling formally, using mathematical notation:

Discriminative modeling estimates $p(y|\mathbf{x})$.

That is, discriminative modeling aims to model the probability of a label y given some observation $\mathbf{x}$.

Generative modeling estimates $p(\mathbf{x})$.

That is, generative modeling aims to model the probability of observing an observation $\mathbf{x}$. Sampling from this distribution allows us to generate new observations.



Conditional Generative Models

Note that we can also build a generative model to model the conditional probability $p(\mathbf{x}|y)$—the probability of seeing an observation $\mathbf{x}$ with a specific label y.

For example, if our dataset contains different types of fruit, we could tell our generative model to specifically generate an image of an apple.

An important point to note is that even if we were able to build a perfect discriminative model to identify Van Gogh paintings, it would still have no idea how to create a painting that looks like a Van Gogh. It can only output probabilities against existing images, as this is what it has been trained to do. We would instead need to train a generative model and sample from this model to generate images that have a high chance of belonging to the original training dataset.

The Rise of Generative Modeling

Until recently, discriminative modeling has been the driving force behind most progress in machine learning. This is because for any discriminative problem, the corresponding generative modeling problem is typically much more difficult to tackle. For example, it is much easier to train a model to predict if a painting is by Van Gogh than it is to train a model to generate a Van Gogh–style painting from scratch.

Similarly, it is much easier to train a model to predict if a page of text was written by Charles Dickens than it is to build a model to generate a set of paragraphs in the style of Dickens. Until recently, most generative challenges were simply out of reach and many doubted that they could ever be solved. Creativity was considered a purely human capability that couldn't be rivaled by AI.

However, as machine learning technologies have matured, this assumption has gradually weakened. In the last 10 years many of the most interesting advancements in the field have come through novel applications of machine learning to generative modeling tasks. For example, Figure 1-3 shows the striking progress that has already been made in facial image generation since 2014.



Figure 1-3. Face generation using generative modeling has improved significantly over the last decade (adapted from Brundage et al., 2018)[1]

As well as being easier to tackle, discriminative modeling has historically been more readily applicable to practical problems across industry than generative modeling. For example, a doctor may benefit from a model that predicts if a given retinal image shows signs of glaucoma, but wouldn't necessarily benefit from a model that can generate novel pictures of the back of an eye.

However, this is also starting to change, with the proliferation of companies offering generative services that target specific business problems. For example, it is now possible to access APIs that generate original blog posts given a particular subject matter, produce a variety of images of your product in any setting you desire, or write social media content and ad copy to match your brand and target message. There are also clear positive applications of generative AI for industries such as game design and cinematography, where models trained to output video and music are beginning to add value.

Generative Modeling and AI

As well as the practical uses of generative modeling (many of which are yet to be discovered), there are three deeper reasons why generative modeling can be considered the key to unlocking a far more sophisticated form of artificial intelligence that goes beyond what discriminative modeling alone can achieve.

Firstly, purely from a theoretical point of view, we shouldn't limit our machine training to simply categorizing data. For completeness, we should also be concerned with training models that capture a more complete understanding of the data distribution, beyond any particular label. This is undoubtedly a more difficult problem to solve, due to the high dimensionality of the space of feasible outputs and the relatively small number of creations that we would class as belonging to the dataset. However, as we shall see, many of the same techniques that have driven development in discriminative modeling, such as deep learning, can be utilized by generative models too.

Secondly, as we shall see in Chapter 12, generative modeling is now being used to drive progress in other fields of AI, such as reinforcement learning (the study of teaching agents to optimize a goal in an environment through trial and error). Suppose we want to train a robot to walk across a given terrain. A traditional approach would be to run many experiments where the agent tries out different strategies in the terrain, or a computer simulation of the terrain. Over time the agent would learn which strategies are more successful than others and therefore gradually improve. A challenge with this approach is that it is fairly inflexible because it is trained to optimize the policy for one particular task. An alternative approach that has recently gained traction is to instead train the agent to learn a *world model* of the environment using a generative model, independent of any particular task. The agent can quickly adapt to new tasks by testing strategies in its own world model, rather than in the real environment, which is often computationally more efficient and does not require retraining from scratch for each new task.

Finally, if we are to truly say that we have built a machine that has acquired a form of intelligence that is comparable to a human's, generative modeling must surely be part of the solution. One of the finest examples of a generative model in the natural world is the person reading this book. Take a moment to consider what an incredible generative model you are. You can close your eyes and imagine what an elephant would look like from any possible angle. You can imagine a number of plausible different endings to your favorite TV show, and you can plan your week ahead by working through various futures in your mind's eye and taking action accordingly. Current neuroscientific theory suggests that our perception of reality is not a highly complex discriminative model operating on our sensory input to produce predictions of what we are experiencing, but is instead a generative model that is trained from birth to produce simulations of our surroundings that accurately match the future. Some theories even suggest that the output from this generative model is what we directly

perceive as reality. Clearly, a deep understanding of how we can build machines to acquire this ability will be central to our continued understanding of the workings of the brain and general artificial intelligence.

Our First Generative Model

With this in mind, let's begin our journey into the exciting world of generative modeling. To begin with, we'll look at a toy example of a generative model and introduce some of the ideas that will help us to work through the more complex architectures that we will encounter later in the book.

Hello World!

Let's start by playing a generative modeling game in just two dimensions. I have chosen a rule that has been used to generate the set of points $\mathbf{X}$ in Figure 1-4. Let's call this rule p_{data}. Your challenge is to choose a different point $\mathbf{x} = (x_1, x_2)$ in the space that looks like it has been generated by the same rule.

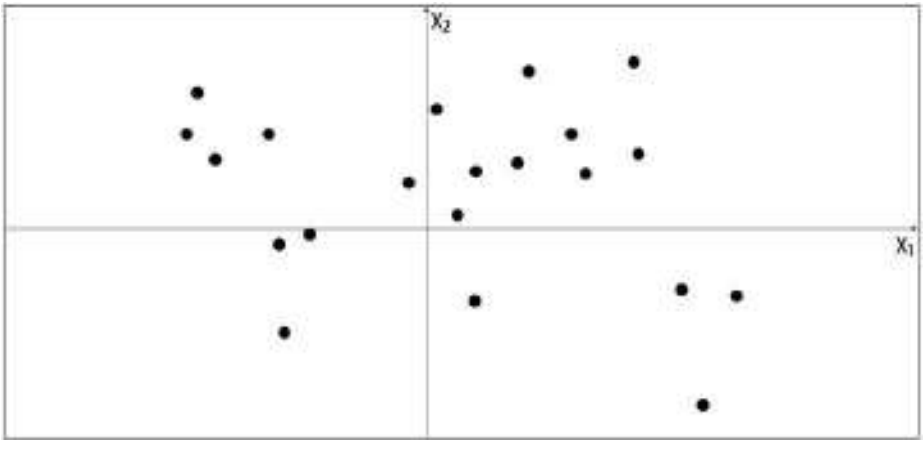


Figure 1-4. A set of points in two dimensions, generated by an unknown rule p_{data}

Where did you choose? You probably used your knowledge of the existing data points to construct a mental model, p_{model}, of whereabouts in the space the point is more likely to be found. In this respect, p_{model} is an *estimate* of p_{data}. Perhaps you decided that p_{model} should look like Figure 1-5—a rectangular box where points may be found, and an area outside of the box where there is no chance of finding any points.

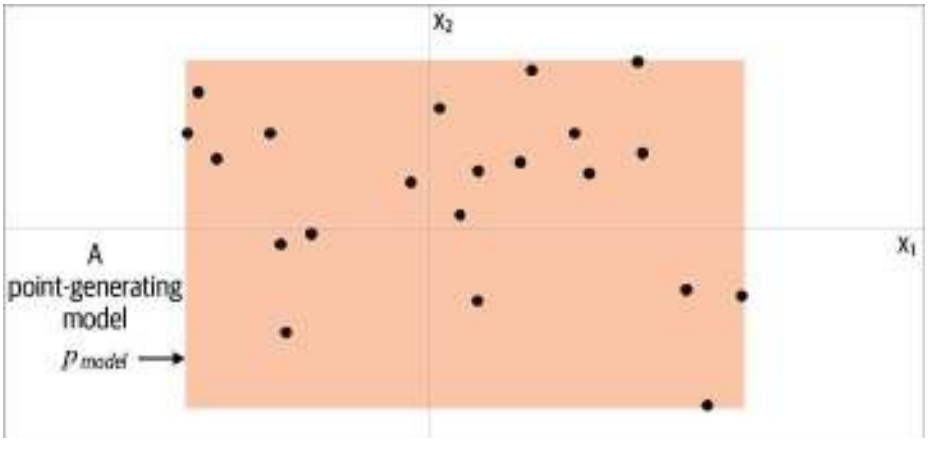


Figure 1-5. The orange box, p_{model}, is an estimate of the true data-generating distribution, p_{data}

To generate a new observation, you can simply choose a point at random within the box, or more formally, *sample* from the distribution p_{model}. Congratulations, you have just built your first generative model! You have used the training data (the black points) to construct a model (the orange region) that you can easily sample from to generate other points that appear to belong to the training set.

Let's now formalize this thinking into a framework that can help us understand what generative modeling is trying to achieve.

The Generative Modeling Framework

We can capture our motivations and goals for building a generative model in the following framework.

The Generative Modeling Framework

- We have a dataset of observations $\mathbf{X}$.
- We assume that the observations have been generated according to some unknown distribution, p_{data}.
- We want to build a generative model p_{model} that mimics p_{data}. If we achieve this goal, we can sample from p_{model} to generate observations that appear to have been drawn from p_{data}.
- Therefore, the desirable properties of p_{model} are:

Accuracy
: If p_{model} is high for a generated observation, it should look like it has been drawn from p_{data}. If \(p_{model}\) is low for a generated observation, it should *not* look like it has been drawn from p_{data}.

Generation
: It should be possible to easily sample a new observation from p_{model}.

Representation
: It should be possible to understand how different high-level features in the data are represented by p_{model}.

Let's now reveal the true data-generating distribution, p_{data}, and see how the framework applies to this example. As we can see from Figure 1-6, the data-generating rule is simply a uniform distribution over the land mass of the world, with no chance of finding a point in the sea.

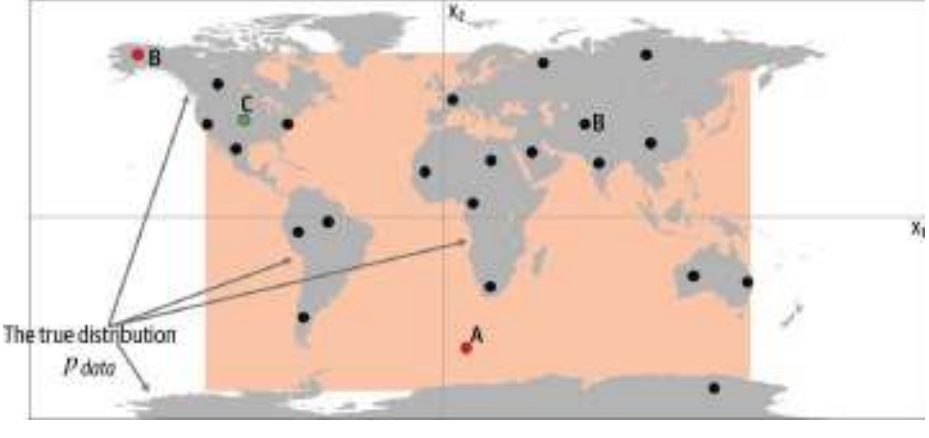


Figure 1-6. The orange box, p_{model}, is an estimate of the true data-generating distribution, p_{data} (the gray area)

Clearly, our model, p_{model}, is an oversimplification of p_{data}. We can inspect points A, B, and C to understand the successes and failures of our model in terms of how accurately it mimics p_{data}:

- Point A is an observation that is generated by our model but does not appear to have been generated by p_{data} as it's in the middle of the sea.

- Point B could never have been generated by p_{model} as it sits outside the orange box. Therefore, our model has some gaps in its ability to produce observations across the entire range of potential possibilities.
- Point C is an observation that could be generated by p_{model} and also by p_{data}.

Despite its shortcomings, the model is easy to sample from, because it is simply a uniform distribution over the orange box. We can easily choose a point at random from inside this box, in order to sample from it.

Also, we can certainly say that our model is a simple representation of the underlying complex distribution that captures some of the underlying high-level features. The true distribution is separated into areas with lots of land mass (continents) and those with no land mass (the sea). This is a high-level feature that is also true of our model, except we have one large continent, rather than many.

This example has demonstrated the fundamental concepts behind generative modeling. The problems we will be tackling in this book will be far more complex and high-dimensional, but the underlying framework through which we approach the problem will be the same.

Representation Learning

It is worth delving a little deeper into what we mean by learning a *representation* of the high-dimensional data, as it is a topic that will recur throughout this book.

Suppose you wanted to describe your appearance to someone who was looking for you in a crowd of people and didn't know what you looked like. You wouldn't start by stating the color of pixel 1 of a photo of you, then pixel 2, then pixel 3, etc. Instead, you would make the reasonable assumption that the other person has a general idea of what an average human looks like, then amend this baseline with features that describe groups of pixels, such as *I have very blond hair* or *I wear glasses*. With no more than 10 or so of these statements, the person would be able to map the description back into pixels to generate an image of you in their head. The image wouldn't be perfect, but it would be a close enough likeness to your actual appearance for them to find you among possibly hundreds of other people, even if they've never seen you before.

This is the core idea behind *representation learning*. Instead of trying to model the high-dimensional sample space directly, we describe each observation in the training set using some lower-dimensional *latent space* and then learn a mapping function that can take a point in the latent space and map it to a point in the original domain. In other words, each point in the latent space is a *representation* of some high-dimensional observation.

What does this mean in practice? Let's suppose we have a training set consisting of grayscale images of biscuit tins (Figure 1-7).

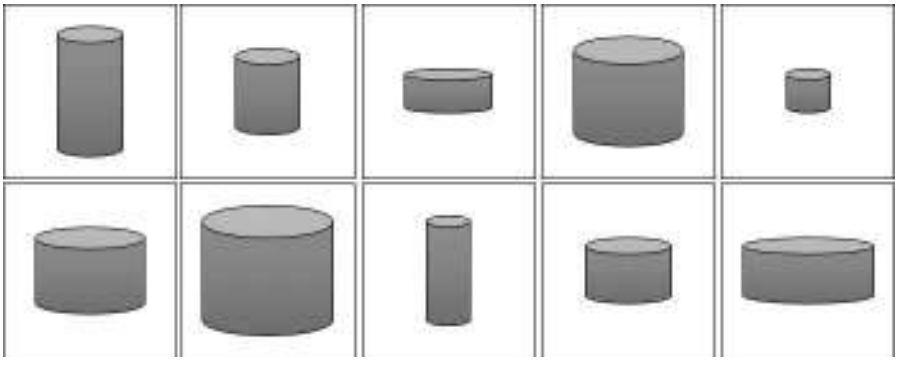


Figure 1-7. The biscuit tin dataset

To us, it is obvious that there are two features that can uniquely represent each of these tins: the height and width of the tin. That is, we can convert each image of a tin to a point in a latent space of just two dimensions, even though the training set of images is provided in high-dimensional pixel space. Notably, this means that we can also produce images of tins that do not exist in the training set, by applying a suitable mapping function f to a new point in the latent space, as shown in Figure 1-8.

Realizing that the original dataset can be described by the simpler latent space is not so easy for a machine—it would first need to establish that height and width are the two latent space dimensions that best describe this dataset, then learn the mapping function f that can take a point in this space and map it to a grayscale biscuit tin image. Machine learning (and specifically, deep learning) gives us the ability to train machines that can find these complex relationships without human guidance.

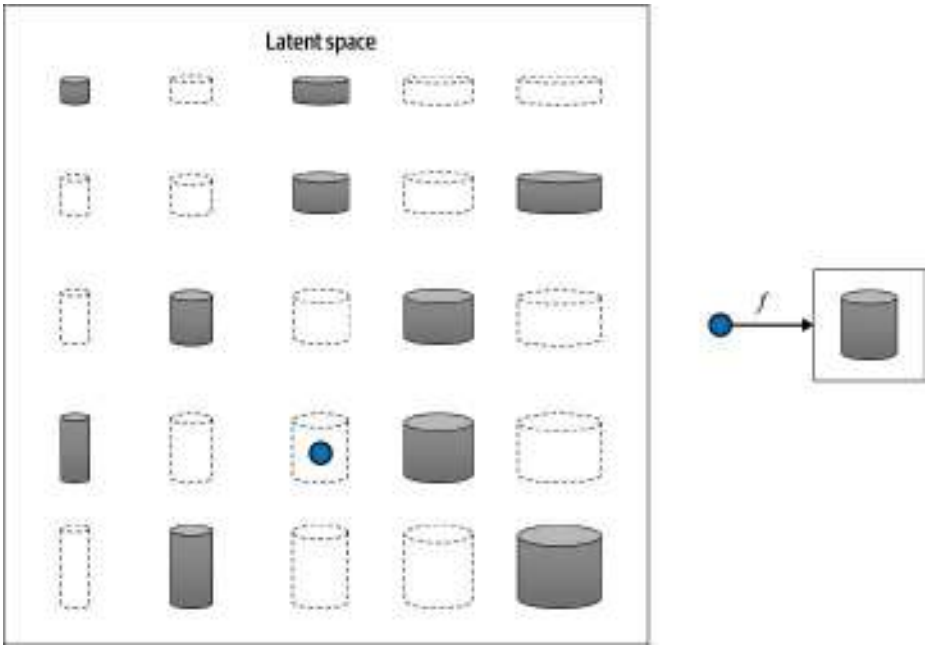


Figure 1-8. The 2D latent space of biscuit tins and the function f that maps a point in the latent space back to the original image domain

One of the benefits of training models that utilize a latent space is that we can perform operations that affect high-level properties of the image by manipulating its representation vector within the more manageable latent space. For example, it is not obvious how to adjust the shading of every single pixel to make an image of a biscuit tin *taller*. However, in the latent space, it's simply a case of increasing the *height* latent dimension, then applying the mapping function to return to the image domain. We shall see an explicit example of this in the next chapter, applied not to biscuit tins but to faces.

The concept of encoding the training dataset into a latent space so that we can sample from it and decode the point back to the original domain is common to many generative modeling techniques, as we shall see in later chapters of this book. Mathematically speaking, *encoder-decoder* techniques try to transform the highly nonlinear *manifold* on which the data lies (e.g., in pixel space) into a simpler latent space that can be sampled from, so that it is likely that any point in the latent space is the representation of a well-formed image, as shown in Figure 1-9.

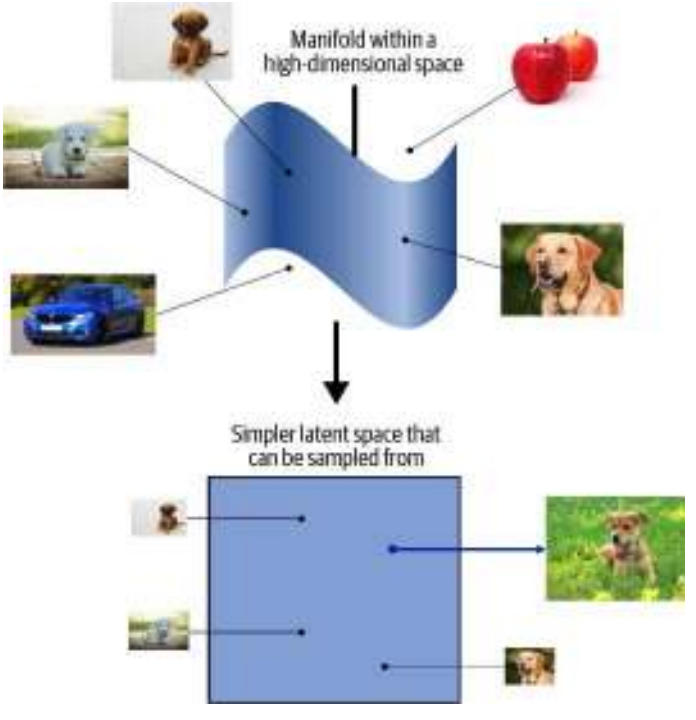


Figure 1-9. The dog manifold in high-dimensional pixel space is mapped to a simpler latent space that can be sampled from

Core Probability Theory

We have already seen that generative modeling is closely connected to statistical modeling of probability distributions. Therefore, it now makes sense to introduce some core probabilistic and statistical concepts that will be used throughout this book to explain the theoretical background of each model.

If you have never studied probability or statistics, don't worry. To build many of the deep learning models that we shall see later in this book, it is not essential to have a deep understanding of statistical theory. However, to gain a full appreciation of the task that we are trying to tackle, it's worth trying to build up a solid understanding of basic probabilistic theory. This way, you will have the foundations in place to understand the different families of generative models that will be introduced later in this chapter.

As a first step, we shall define five key terms, linking each one back to our earlier example of a generative model that models the world map in two dimensions:

Sample space
: The *sample space* is the complete set of all values an observation $\mathbf{x}$ can take.



In our previous example, the sample space consists of all points of latitude and longitude $\mathbf{x} = (x_1, x_2)$ on the world map. For example, $\mathbf{x}$ = (40.7306, –73.9352) is a point in the sample space (New York City) that belongs to the true data-generating distribution. $\mathbf{x}$ = (11.3493, 142.1996) is a point in the sample space that does not belong to the true data-generating distribution (it's in the sea).

Probability density function
: A *probability density function* (or simply *density function*) is a function $p(\mathbf{x})$ that maps a point $\mathbf{x}$ in the sample space to a number between 0 and 1. The integral of the density function over all points in the sample space must equal 1, so that it is a well-defined probability distribution.



In the world map example, the density function of our generative model is 0 outside of the orange box and constant inside of the box, so that the integral of the density function over the entire sample space equals 1.

While there is only one true density function $p_{data}(\mathbf{x})$ that is assumed to have generated the observable dataset, there are infinitely many density functions $p_{model}(\mathbf{x})$ that we can use to estimate $p_{data}(\mathbf{x})$.

Parametric modeling
: *Parametric modeling* is a technique that we can use to structure our approach to finding a suitable $p_{model}(\mathbf{x})$. A *parametric model* is a family of density functions $p_\theta(\mathbf{x})$ that can be described using a finite number of parameters, θ.



If we assume a uniform distribution as our model family, then the set all possible boxes we could draw on Figure 1-5 is an example of a parametric model. In this case, there are four parameters: the coordinates of the bottom-left (θ_1, θ_2) and top-right (θ_3, θ_4) corners of the box.

Thus, each density function $p_\theta(\mathbf{x})$ in this parametric model (i.e., each box) can be uniquely represented by four numbers, $\theta = (\theta_1, \theta_2, \theta_3, \theta_4)$.

Likelihood

The *likelihood* $\mathscr{L}(\theta|\mathbf{x})$ of a parameter set θ is a function that measures the plausibility of θ, given some observed point $\mathbf{x}$. It is defined as follows:

$$\mathscr{L}(\theta|\mathbf{x}) = p_\theta(\mathbf{x})$$

That is, the likelihood of θ given some observed point $\mathbf{x}$ is defined to be the value of the density function parameterized by θ, at the point $\mathbf{x}$. If we have a whole dataset $\mathbf{X}$ of independent observations, then we can write:

$$\mathscr{L}(\theta|\mathbf{X}) = \prod_{\mathbf{x} \in \mathbf{X}} p_\theta(\mathbf{x})$$



In the world map example, an orange box that only covered the left half of the map would have a likelihood of 0—it couldn't possibly have generated the dataset, as we have observed points in the right half of the map. The orange box in Figure 1-5 has a positive likelihood, as the density function is positive for all data points under this model.

Since the product of a large number of terms between 0 and 1 can be quite computationally difficult to work with, we often use the *log-likelihood* ℓ instead:

$$\ell(\theta|\mathbf{X}) = \sum_{\mathbf{x} \in \mathbf{X}} \log p_\theta(\mathbf{x})$$

There are statistical reasons why the likelihood is defined in this way, but we can also see that this definition intuitively makes sense. The likelihood of a set of parameters θ is defined to be the probability of seeing the data if the true data-generating distribution was the model parameterized by θ.



Note that the likelihood is a function of the *parameters*, not the data. It should *not* be interpreted as the probability that a given parameter set is correct—in other words, it is not a probability distribution over the parameter space (i.e., it doesn't sum/integrate to 1, with respect to the parameters).

It makes intuitive sense that the focus of parametric modeling should be to find the optimal value $\hat{\theta}$ of the parameter set that maximizes the likelihood of observing the dataset $\mathbf{X}$.

Maximum likelihood estimation

Maximum likelihood estimation is the technique that allows us to estimate $\hat{\theta}$—the set of parameters θ of a density function $p_\theta(\mathbf{x})$ that is most likely to explain some observed data $\mathbf{X}$. More formally:

$$\hat{\theta} = \arg\max_{\mathbf{x}} \ell(\theta|\mathbf{X})$$

$\hat{\theta}$ is also called the *maximum likelihood estimate* (MLE).



In the world map example, the MLE is the smallest rectangle that still contains all of the points in the training set.

Neural networks typically *minimize* a loss function, so we can equivalently talk about finding the set of parameters that *minimize the negative log-likelihood*:

$$\hat{\theta} = \arg\min_{\theta}(-\ell(\theta|\mathbf{X})) = \arg\min_{\theta}\left(-\log p_\theta(\mathbf{X})\right)$$

Generative modeling can be thought of as a form of maximum likelihood estimation, where the parameters θ are the weights of the neural networks contained in the model. We are trying to find the values of these parameters that maximize the likelihood of observing the given data (or equivalently, minimize the negative log-likelihood).

However, for high-dimensional problems, it is generally not possible to directly calculate $p_\theta(\mathbf{x})$—it is *intractable*. As we shall see in the next section, different families of generative models take different approaches to tackling this problem.

Generative Model Taxonomy

While all types of generative models ultimately aim to solve the same task, they all take slightly different approaches to modeling the density function $p_\theta(\mathbf{x})$. Broadly speaking, there are three possible approaches:

1. Explicitly model the density function, but constrain the model in some way, so that the density function is tractable (i.e., it can be calculated).
2. Explicitly model a tractable approximation of the density function.

3. Implicitly model the density function, through a stochastic process that directly generates data.

These are shown in Figure 1-10 as a taxonomy, alongside the six families of generative models that we will explore in Part II of this book. Note that these families are not mutually exclusive—there are many examples of models that are hybrids between two different kinds of approaches. You should think of the families as different general approaches to generative modeling, rather than explicit model architectures.

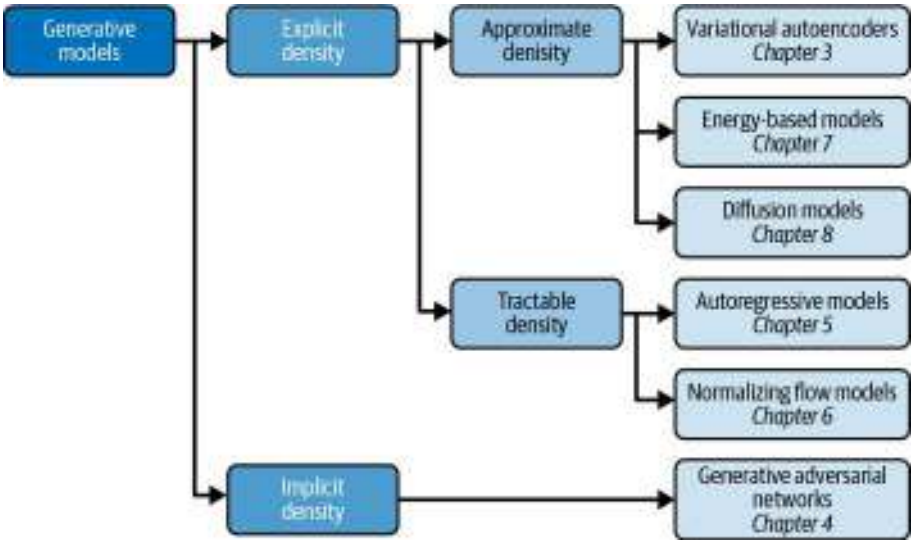


Figure 1-10. A taxonomy of generative modeling approaches

The first split that we can make is between models where the probability density function $p(\mathbf{x})$ is modeled *explicitly* and those where it is modeled *implicitly*.

Implicit density models do not aim to estimate the probability density at all, but instead focus solely on producing a stochastic process that directly generates data. The best-known example of an implicit generative model is a *generative adversarial network*. We can further split *explicit density models* into those that directly optimize the density function (tractable models) and those that only optimize an approximation of it.

Tractable models place constraints on the model architecture, so that the density function has a form that makes it easy to calculate. For example, *autoregressive models* impose an ordering on the input features, so that the output can be generated sequentially—e.g., word by word, or pixel by pixel. *Normalizing flow models* apply a series of tractable, invertible functions to a simple distribution, in order to generate more complex distributions.

Approximate density models include *variational autoencoders*, which introduce a latent variable and optimize an approximation of the joint density function. *Energy-based models* also utilize approximate methods, but do so via Markov chain sampling, rather than variational methods. *Diffusion models* approximate the density function by training a model to gradually denoise a given image that has been previously corrupted.

A common thread that runs through all of the generative model family types is *deep learning*. Almost all sophisticated generative models have a deep neural network at their core, because they can be trained from scratch to learn the complex relationships that govern the structure of the data, rather than having to be hardcoded with information a priori. We'll explore deep learning in Chapter 2, with practical examples of how to get started building your own deep neural networks.

The Generative Deep Learning Codebase

The final section of this chapter will get you set up to start building generative deep learning models by introducing the codebase that accompanies this book.



Many of the examples in this book are adapted from the excellent open source implementations that are available through the Keras website. I highly recommend you check out this resource, as new models and examples are constantly being added.

Cloning the Repository

To get started, you'll first need to clone the Git repository. *Git* is an open source version control system and will allow you to copy the code locally so that you can run the notebooks on your own machine, or in a cloud-based environment. You may already have this installed, but if not, follow the instructions relevant to your operating system.

To clone the repository for this book, navigate to the folder where you would like to store the files and type the following into your terminal:

```
git clone https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition.git
```

You should now be able to see the files in a folder on your machine.

Using Docker

The codebase for this book is intended to be used with *Docker*, a free containerization technology that makes getting started with a new codebase extremely easy, regardless of your architecture or operating system. If you have never used Docker, don't worry—there is a description of how to get started in the *README* file in the book repository.

Running on a GPU

If you don't have access to your own GPU, that's also no problem! All of the examples in this book will train on a CPU, though this will take longer than if you use a GPU-enabled machine. There is also a section in the *README* about setting up a Google Cloud environment that gives you access to a GPU on a pay-as-you-go basis.

Summary

This chapter introduced the field of generative modeling, an important branch of machine learning that complements the more widely studied discriminative modeling. We discussed how generative modeling is currently one of the most active and exciting areas of AI research, with many recent advances in both theory and applications.

We started with a simple toy example and saw how generative modeling ultimately focuses on modeling the underlying distribution of the data. This presents many complex and interesting challenges, which we summarized into a framework for understanding the desirable properties of any generative model.

We then walked through the key probabilistic concepts that will help to fully understand the theoretical foundations of each approach to generative modeling and laid out the six different families of generative models that we will explore in Part II of this book. We also saw how to get started with the *Generative Deep Learning* codebase, by cloning the repository.

In Chapter 2, we will begin our exploration of deep learning and see how to use Keras to build models that can perform discriminative modeling tasks. This will give us the necessary foundation to tackle generative deep learning problems in later chapters.

References

1. Miles Brundage et al., "The Malicious Use of Artificial Intelligence: Forecasting, Prevention, and Mitigation," February 20, 2018, *https://www.eff.org/files/2018/02/20/malicious_ai_report_final.pdf*.

CHAPTER 2
Deep Learning

Chapter Goals

In this chapter you will:

- Learn about the different types of unstructured data that can be modeled using deep learning.
- Define a deep neural network and understand how it can be used to model complex datasets.
- Build a multilayer perceptron to predict the content of an image.
- Improve the performance of the model by using convolutional layers, dropout, and batch normalization layers.

Let's start with a basic definition of deep learning:

> Deep learning is a class of machine learning algorithms that uses *multiple stacked layers of processing units* to learn high-level representations from *unstructured* data.

To understand deep learning fully, we need to delve into this definition a bit further. First, we'll take a look at the different types of unstructured data that deep learning can be used to model, then we'll dive into the mechanics of building multiple stacked layers of processing units to solve classification tasks. This will provide the foundation for future chapters where we focus on deep learning for generative tasks.

Data for Deep Learning

Many types of machine learning algorithms require *structured*, tabular data as input, arranged into columns of features that describe each observation. For example, a person's age, income, and number of website visits in the last month are all features that could help to predict if the person will subscribe to a particular online service in the coming month. We could use a structured table of these features to train a logistic regression, random forest, or XGBoost model to predict the binary response variable—did the person subscribe (1) or not (0)? Here, each individual feature contains a nugget of information about the observation, and the model would learn how these features interact to influence the response.

Unstructured data refers to any data that is not naturally arranged into columns of features, such as images, audio, and text. There is of course spatial structure to an image, temporal structure to a recording or passage of text, and both spatial and temporal structure to video data, but since the data does not arrive in columns of features, it is considered unstructured, as shown in Figure 2-1.

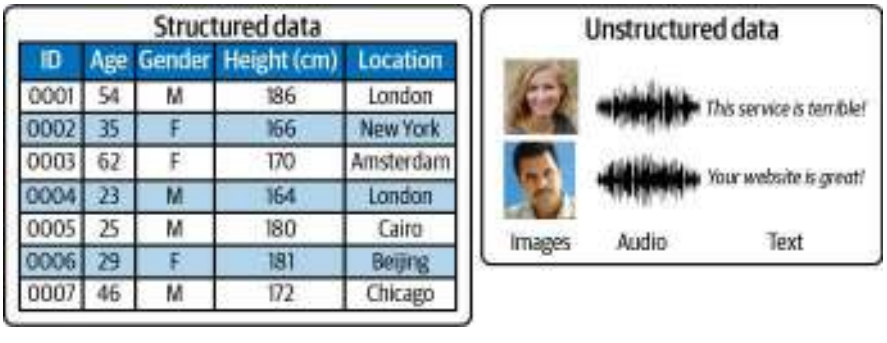


Figure 2-1. The difference between structured and unstructured data

When our data is unstructured, individual pixels, frequencies, or characters are almost entirely uninformative. For example, knowing that pixel 234 of an image is a muddy shade of brown doesn't really help identify if the image is of a house or a dog, and knowing that character 24 of a sentence is an *e* doesn't help predict if the text is about football or politics.

Pixels or characters are really just the dimples of the canvas into which higher-level informative features, such as an image of a chimney or the word *striker*, are embedded. If the chimney in the image were placed on the other side of the house, the image would still contain a chimney, but this information would now be carried by completely different pixels. If the word *striker* appeared slightly earlier or later in the text, the text would still be about football, but different character positions would provide this information. The granularity of the data combined with the high degree

of spatial dependence destroys the concept of the pixel or character as an informative feature in its own right.

For this reason, if we train logistic regression, random forest, or XGBoost models on raw pixel values, the trained model will often perform poorly for all but the simplest of classification tasks. These models rely on the input features to be informative and not spatially dependent. A deep learning model, on the other hand, can learn how to build high-level informative features by itself, directly from the unstructured data.

Deep learning can be applied to structured data, but its real power, especially with regard to generative modeling, comes from its ability to work with unstructured data. Most often, we want to generate unstructured data such as new images or original strings of text, which is why deep learning has had such a profound impact on the field of generative modeling.

Deep Neural Networks

The majority of deep learning systems are *artificial neural networks* (ANNs, or just *neural networks* for short) with multiple stacked hidden layers. For this reason, *deep learning* has now almost become synonymous with *deep neural networks*. However, any system that employs many layers to learn high-level representations of the input data is also a form of deep learning (e.g., deep belief networks).

Let's start by breaking down exactly what we mean by a neural network and then see how they can be used to learn high-level features from unstructured data.

What Is a Neural Network?

A neural network consists of a series of stacked *layers*. Each layer contains *units* that are connected to the previous layer's units through a set of *weights*. As we shall see, there are many different types of layers, but one of the most common is the *fully connected* (or *dense*) layer that connects all units in the layer directly to every unit in the previous layer.

Neural networks where all adjacent layers are fully connected are called *multilayer perceptrons* (MLPs). This is the first type of neural network that we will study. An example of an MLP is shown in Figure 2-2.

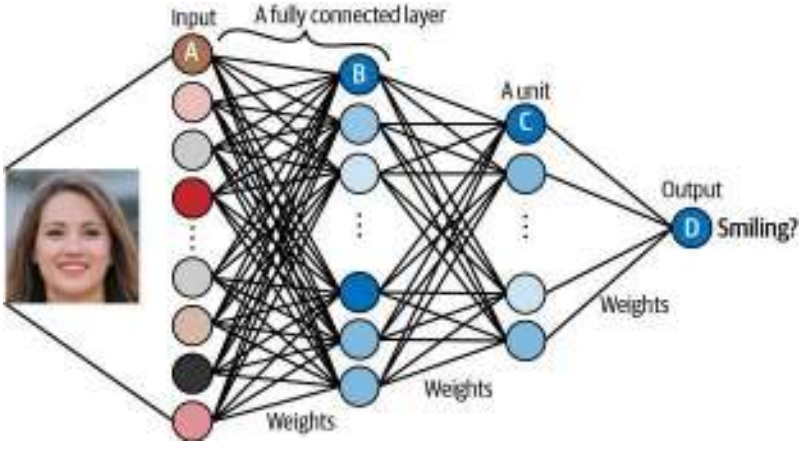


Figure 2-2. An example of a multilayer perceptron that predicts if a face is smiling

The input (e.g., an image) is transformed by each layer in turn, in what is known as a *forward pass* through the network, until it reaches the output layer. Specifically, each unit applies a nonlinear transformation to a weighted sum of its inputs and passes the output through to the subsequent layer. The final output layer is the culmination of this process, where the single unit outputs a probability that the original input belongs to a particular category (e.g., *smiling*).

The magic of deep neural networks lies in finding the set of weights for each layer that results in the most accurate predictions. The process of finding these weights is what we mean by *training* the network.

During the training process, batches of images are passed through the network and the predicted outputs are compared to the ground truth. For example, the network might output a probability of 80% for an image of someone who really is smiling and a probability of 23% for an image of someone who really isn't smiling. A perfect prediction would output 100% and 0% for these examples, so there is a small amount of error. The error in the prediction is then propagated backward through the network, adjusting each set of weights a small amount in the direction that improves the prediction most significantly. This process is appropriately called *backpropagation*. Gradually, each unit becomes skilled at identifying a particular feature that ultimately helps the network to make better predictions.

Learning High-Level Features

The critical property that makes neural networks so powerful is their ability to learn features from the input data, without human guidance. In other words, we do not need to do any feature engineering, which is why neural networks are so useful! We

can let the model decide how it wants to arrange its weights, guided only by its desire to minimize the error in its predictions.

For example, let's walk through the network shown in Figure 2-2, assuming it has already been trained to accurately predict if a given input face is smiling:

1. Unit A receives the value for an individual channel of an input pixel.
2. Unit B combines its input values so that it fires strongest when a particular low-level feature such as an edge is present.
3. Unit C combines the low-level features so that it fires strongest when a higher-level feature such as *teeth* are seen in the image.
4. Unit D combines the high-level features so that it fires strongest when the person in the original image is smiling.

Units in each subsequent layer are able to represent increasingly sophisticated aspects of the original input, by combining lower-level features from the previous layer. Amazingly, this arises naturally out of the training process—we do not need to *tell* each unit what to look for, or whether it should look for high-level features or low-level features.

The layers between the input and output layers are called *hidden* layers. While our example only has two hidden layers, deep neural networks can have many more. Stacking large numbers of layers allows the neural network to learn progressively higher-level features by gradually building up information from the lower-level features in previous layers. For example, ResNet,[1] designed for image recognition, contains 152 layers.

Next, we'll dive straight into the practical side of deep learning and get set up with TensorFlow and Keras so that you can start building your own deep neural networks.

TensorFlow and Keras

TensorFlow is an open source Python library for machine learning, developed by Google. TensorFlow is one of the most utilized frameworks for building machine learning solutions, with particular emphasis on the manipulation of tensors (hence the name). It provides the low-level functionality required to train neural networks, such as computing the gradient of arbitrary differentiable expressions and efficiently executing tensor operations.

Keras is a high-level API for building neural networks, built on top of TensorFlow (Figure 2-3). It is extremely flexible and very user-friendly, making it an ideal choice for getting started with deep learning. Moreover, Keras provides numerous useful building blocks that can be plugged together to create highly complex deep learning architectures through its functional API.



Figure 2-3. TensorFlow and Keras are excellent tools for building deep learning solutions

If you are just getting started with deep learning, I can highly recommend using TensorFlow and Keras. This setup will allow you to build any network that you can think of in a production environment, while also giving you an easy-to-learn API that enables rapid development of new ideas and concepts. Let's start by seeing how easy it is to build a multilayer perceptron using Keras.

Multilayer Perceptron (MLP)

In this section, we will train an MLP to classify a given image using *supervised learning*. Supervised learning is a type of machine learning algorithm in which the computer is trained on a labeled dataset. In other words, the dataset used for training includes input data with corresponding output labels. The goal of the algorithm is to learn a mapping between the input data and the output labels, so that it can make predictions on new, unseen data.

The MLP is a discriminative (rather than generative) model, but supervised learning will still play a role in many types of generative models that we will explore in later chapters of this book, so it is a good place to start our journey.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/02_deeplearning/01_mlp/mlp.ipynb* in the book repository.

Preparing the Data

For this example we will be using the CIFAR-10 dataset, a collection of 60,000 32 × 32–pixel color images that comes bundled with Keras out of the box. Each image is classified into exactly one of 10 classes, as shown in Figure 2-4.



Figure 2-4. Example images from the CIFAR-10 dataset (source: Krizhevsky, 2009)[2]

By default, the image data consists of integers between 0 and 255 for each pixel channel. We first need to preprocess the images by scaling these values to lie between 0 and 1, as neural networks work best when the absolute value of each input is less than 1.

We also need to change the integer labeling of the images to one-hot encoded vectors, because the neural network output will be a probability that the image belongs to each class. If the class integer label of an image is *i*, then its one-hot encoding is a vector of length 10 (the number of classes) that has 0s in all but the *i*th element, which is 1. These steps are shown in Example 2-1.

Example 2-1. Preprocessing the CIFAR-10 dataset

```
import numpy as np
from tensorflow.keras import datasets, utils

(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data() ❶
```

```
NUM_CLASSES = 10

x_train = x_train.astype('float32') / 255.0 ❷
x_test = x_test.astype('float32') / 255.0

y_train = utils.to_categorical(y_train, NUM_CLASSES) ❸
y_test = utils.to_categorical(y_test, NUM_CLASSES)
```

❶ Load the CIFAR-10 dataset. `x_train` and `x_test` are `numpy` arrays of shape `[50000, 32, 32, 3]` and `[10000, 32, 32, 3]`, respectively. `y_train` and `y_test` are `numpy` arrays of shape `[50000, 1]` and `[10000, 1]`, respectively, containing the integer labels in the range 0 to 9 for the class of each image.

❷ Scale each image so that the pixel channel values lie between 0 and 1.

❸ One-hot encode the labels—the new shapes of `y_train` and `y_test` are `[50000, 10]` and `[10000, 10]`, respectively.

We can see that the training image data (`x_train`) is stored in a *tensor* of shape `[50000, 32, 32, 3]`. There are no *columns* or *rows* in this dataset; instead, this is a tensor with four dimensions. A tensor is just a multidimensional array—it is the natural extension of a matrix to more than two dimensions. The first dimension of this tensor references the index of the image in the dataset, the second and third relate to the size of the image, and the last is the channel (i.e., red, green, or blue, since these are RGB images).

For example, Example 2-2 shows how we can find the channel value of a specific pixel in an image.

Example 2-2. The green channel (1) value of the pixel in the (12,13) position of image 54

```
x_train[54, 12, 13, 1]
# 0.36862746
```

Building the Model

In Keras you can either define the structure of a neural network as a `Sequential` model or using the functional API.

A `Sequential` model is useful for quickly defining a linear stack of layers (i.e., where one layer follows on directly from the previous layer without any branching). We can define our MLP model using the `Sequential` class as shown in Example 2-3.

Example 2-3. Building our MLP using a `Sequential` model

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Flatten(input_shape=(32, 32, 3)),
    layers.Dense(200, activation = 'relu'),
    layers.Dense(150, activation = 'relu'),
    layers.Dense(10, activation = 'softmax'),
])
```

Many of the models in this book require that the output from a layer is passed to multiple subsequent layers, or conversely, that a layer receives input from multiple preceding layers. For these models, the `Sequential` class is not suitable and we would need to use the functional API instead, which is a lot more flexible.



I recommend that even if you are just starting out building linear models with Keras, you still use the functional API rather than `Sequential` models, since it will serve you better in the long run as your neural networks become more architecturally complex. The functional API will give you complete freedom over the design of your deep neural network.

Example 2-4 shows the same MLP coded using the functional API. When using the functional API, we use the `Model` class to define the overall input and output layers of the model.

Example 2-4. Building our MLP using the functional API

```
from tensorflow.keras import layers, models

input_layer = layers.Input(shape=(32, 32, 3))
x = layers.Flatten()(input_layer)
x = layers.Dense(units=200, activation = 'relu')(x)
x = layers.Dense(units=150, activation = 'relu')(x)
output_layer = layers.Dense(units=10, activation = 'softmax')(x)
model = models.Model(input_layer, output_layer)
```

Both methods give identical models—a diagram of the architecture is shown in Figure 2-5.

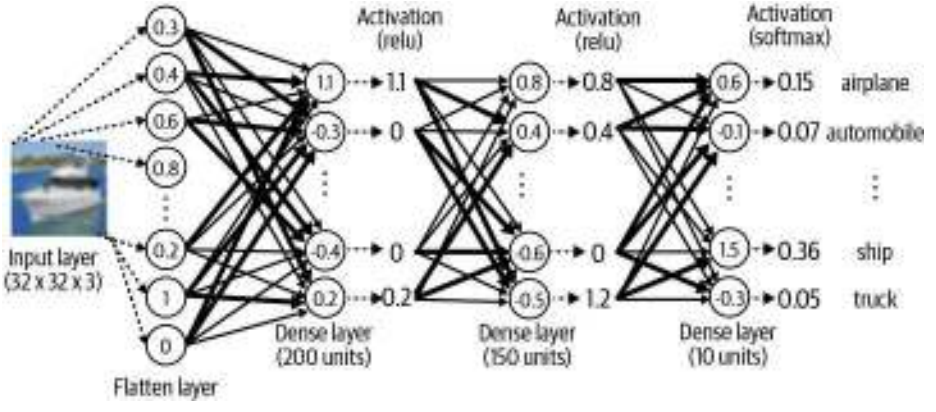


Figure 2-5. A diagram of the MLP architecture

Let's now look in more detail at the different layers and activation functions used within the MLP.

Layers

To build our MLP, we used three different types of layers: `Input`, `Flatten`, and `Dense`.

The `Input` layer is an entry point into the network. We tell the network the shape of each data element to expect as a tuple. Notice that we do not specify the batch size; this isn't necessary as we can pass any number of images into the `Input` layer simultaneously. We do not need to explicitly state the batch size in the `Input` layer definition.

Next we flatten this input into a vector, using a `Flatten` layer. This results in a vector of length 3,072 (= 32 × 32 × 3). The reason we do this is because the subsequent `Dense` layer requires that its input is flat, rather than a multidimensional array. As we shall see later, other layer types require multidimensional arrays as input, so you need to be aware of the required input and output shape of each layer type to understand when it is necessary to use `Flatten`.

The `Dense` layer is one of the most fundamental building blocks of a neural network. It contains a given number of units that are densely connected to the previous layer—that is, every unit in the layer is connected to every unit in the previous layer, through a single connection that carries a weight (which can be positive or negative). The output from a given unit is the weighted sum of the inputs it receives from the previous layer, which is then passed through a nonlinear *activation function* before being sent to the following layer. The activation function is critical to ensure the neural network is able to learn complex functions and doesn't just output a linear combination of its inputs.

Activation functions

There are many kinds of activation function, but three of the most important are ReLU, sigmoid, and softmax.

The *ReLU* (rectified linear unit) activation function is defined to be 0 if the input is negative and is otherwise equal to the input. The *LeakyReLU* activation function is very similar to ReLU, with one key difference: whereas the ReLU activation function returns 0 for input values less than 0, the LeakyReLU function returns a small negative number proportional to the input. ReLU units can sometimes die if they always output 0, because of a large bias toward negative values pre-activation. In this case, the gradient is 0 and therefore no error is propagated back through this unit. LeakyReLU activations fix this issue by always ensuring the gradient is nonzero. ReLU-based functions are among the most reliable activations to use between the layers of a deep network to encourage stable training.

The *sigmoid* activation is useful if you wish the output from the layer to be scaled between 0 and 1—for example, for binary classification problems with one output unit or multilabel classification problems, where each observation can belong to more than one class. Figure 2-6 shows ReLU, LeakyReLU, and sigmoid activation functions side by side for comparison.

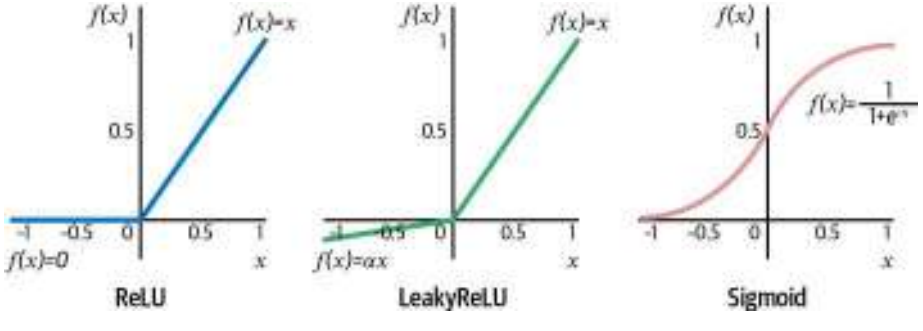


Figure 2-6. The ReLU, LeakyReLU, and sigmoid activation functions

The *softmax* activation function is useful if you want the total sum of the output from the layer to equal 1; for example, for multiclass classification problems where each observation only belongs to exactly one class. It is defined as:

$$y_i = \frac{e^{x_i}}{\Sigma_{j=1}^{J} e^{x_j}}$$

Here, J is the total number of units in the layer. In our neural network, we use a softmax activation in the final layer to ensure that the output is a set of 10 probabilities

that sum to 1, which can be interpreted as the likelihood that the image belongs to each class.

In Keras, activation functions can be defined within a layer (Example 2-5) or as a separate layer (Example 2-6).

Example 2-5. A ReLU activation function defined as part of a `Dense` layer

```
x = layers.Dense(units=200, activation = 'relu')(x)
```

Example 2-6. A ReLU activation function defined as its own layer

```
x = layers.Dense(units=200)(x)
x = layers.Activation('relu')(x)
```

In our example, we pass the input through two `Dense` layers, the first with 200 units and the second with 150, both with ReLU activation functions.

Inspecting the model

We can use the `model.summary()` method to inspect the shape of the network at each layer, as shown in Table 2-1.

Table 2-1. Output from the `model.summary()` method

Layer (type)	Output shape	Param #
InputLayer	(None, 32, 32, 3)	0
Flatten	(None, 3072)	0
Dense	(None, 200)	614,600
Dense	(None, 150)	30,150
Dense	(None, 10)	1,510

Total params	646,260
Trainable params	646,260
Non-trainable params	0

Notice how the shape of our `Input` layer matches the shape of `x_train` and the shape of our `Dense` output layer matches the shape of `y_train`. Keras uses `None` as a marker for the first dimension to show that it doesn't yet know the number of observations that will be passed into the network. In fact, it doesn't need to; we could just as easily pass 1 observation through the network at a time as 1,000. That's because tensor operations are conducted across all observations simultaneously using linear algebra—this is the part handled by TensorFlow. It is also the reason why you get a performance increase when training deep neural networks on GPUs instead of CPUs: GPUs are

optimized for large tensor operations since these calculations are also necessary for complex graphics manipulation.

The `summary` method also gives the number of parameters (weights) that will be trained at each layer. If ever you find that your model is training too slowly, check the summary to see if there are any layers that contain a huge number of weights. If so, you should consider whether the number of units in the layer could be reduced to speed up training.



Make sure you understand how the number of parameters is calculated in each layer! It's important to remember that by default, each unit within a given layer is also connected to one additional *bias* unit that always outputs 1. This ensures that the output from the unit can still be nonzero even when all inputs from the previous layer are 0.

Therefore, the number of parameters in the 200-unit `Dense` layer is 200 * (3,072 + 1) = 614,600.

Compiling the Model

In this step, we compile the model with an optimizer and a loss function, as shown in Example 2-7.

Example 2-7. Defining the optimizer and the loss function

```
from tensorflow.keras import optimizers

opt = optimizers.Adam(learning_rate=0.0005)
model.compile(loss='categorical_crossentropy', optimizer=opt,
              metrics=['accuracy'])
```

Let's now look in more detail at what we mean by loss functions and optimizers.

Loss functions

The *loss function* is used by the neural network to compare its predicted output to the ground truth. It returns a single number for each observation; the greater this number, the worse the network has performed for this observation.

Keras provides many built-in loss functions to choose from, or you can create your own. Three of the most commonly used are mean squared error, categorical cross-entropy, and binary cross-entropy. It is important to understand when it is appropriate to use each.

If your neural network is designed to solve a regression problem (i.e., the output is continuous), then you might use the *mean squared error* loss. This is the mean of the

squared difference between the ground truth y_i and predicted value p_i of each output unit, where the mean is taken over all n output units:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - p_i)^2$$

If you are working on a classification problem where each observation only belongs to one class, then *categorical cross-entropy* is the correct loss function. This is defined as follows:

$$-\sum_{i=1}^{n} y_i \log(p_i)$$

Finally, if you are working on a binary classification problem with one output unit, or a multilabel problem where each observation can belong to multiple classes simultaneously, you should use *binary cross-entropy*:

$$-\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(p_i) + (1 - y_i)\log(1 - p_i)\right)$$

Optimizers

The *optimizer* is the algorithm that will be used to update the weights in the neural network based on the gradient of the loss function. One of the most commonly used and stable optimizers is *Adam* (Adaptive Moment Estimation).[3] In most cases, you shouldn't need to tweak the default parameters of the Adam optimizer, except the *learning rate*. The greater the learning rate, the larger the change in weights at each training step. While training is initially faster with a large learning rate, the downside is that it may result in less stable training and may not find the global minimum of the loss function. This is a parameter that you may want to tune or adjust during training.

Another common optimizer that you may come across is *RMSProp* (Root Mean Squared Propagation). Again, you shouldn't need to adjust the parameters of this optimizer too much, but it is worth reading the Keras documentation to understand the role of each parameter.

We pass both the loss function and the optimizer into the `compile` method of the model, as well as a `metrics` parameter where we can specify any additional metrics that we would like to report on during training, such as accuracy.

Training the Model

Thus far, we haven't shown the model any data. We have just set up the architecture and compiled the model with a loss function and optimizer.

To train the model against the data, we simply call the `fit` method, as shown in Example 2-8.

Example 2-8. Calling the `fit` method to train the model

```
model.fit(x_train ❶
          , y_train ❷
          , batch_size = 32 ❸
          , epochs = 10 ❹
          , shuffle = True ❺
          )
```

❶ The raw image data.

❷ The one-hot encoded class labels.

❸ The `batch_size` determines how many observations will be passed to the network at each training step.

❹ The `epochs` determine how many times the network will be shown the full training data.

❺ If `shuffle = True`, the batches will be drawn randomly without replacement from the training data at each training step.

This will start training a deep neural network to predict the category of an image from the CIFAR-10 dataset. The training process works as follows.

First, the weights of the network are initialized to small random values. Then the network performs a series of training steps. At each training step, one *batch* of images is passed through the network and the errors are backpropagated to update the weights. The `batch_size` determines how many images are in each training step batch. The larger the batch size, the more stable the gradient calculation, but the slower each training step.



It would be far too time-consuming and computationally intensive to use the entire dataset to calculate the gradient at each training step, so generally a batch size between 32 and 256 is used. It is also now recommended practice to increase the batch size as training progresses.[4]

This continues until all observations in the dataset have been seen once. This completes the first *epoch*. The data is then passed through the network again in batches as part of the second epoch. This process repeats until the specified number of epochs have elapsed.

During training, Keras outputs the progress of the procedure, as shown in Figure 2-7. We can see that the training dataset has been split into 1,563 batches (each containing 32 images) and it has been shown to the network 10 times (i.e., over 10 epochs), at a rate of approximately 2 milliseconds per batch. The categorical cross-entropy loss has fallen from 1.8377 to 1.3696, resulting in an accuracy increase from 33.69% after the first epoch to 51.67% after the tenth epoch.

```
model.fit(x_train, y_train, batch_size=32, epochs=10, shuffle=True)
Epoch 1/10
1563/1563 [==============================] - 3s 2ms/step - loss: 1.8377 - accuracy: 0.3369
Epoch 2/10
1563/1563 [==============================] - 3s 2ms/step - loss: 1.6552 - accuracy: 0.4076
Epoch 3/10
1563/1563 [==============================] - 3s 2ms/step - loss: 1.5743 - accuracy: 0.4396
Epoch 4/10
1563/1563 [==============================] - 3s 2ms/step - loss: 1.5288 - accuracy: 0.4549
Epoch 5/10
1563/1563 [==============================] - 3s 2ms/step - loss: 1.4888 - accuracy: 0.4706
Epoch 6/10
1563/1563 [==============================] - 2s 2ms/step - loss: 1.4542 - accuracy: 0.4851
Epoch 7/10
1563/1563 [==============================] - 3s 2ms/step - loss: 1.4332 - accuracy: 0.4908
Epoch 8/10
1563/1563 [==============================] - 2s 2ms/step - loss: 1.4094 - accuracy: 0.4992
Epoch 9/10
1563/1563 [==============================] - 2s 2ms/step - loss: 1.3896 - accuracy: 0.5045
Epoch 10/10
1563/1563 [==============================] - 3s 2ms/step - loss: 1.3696 - accuracy: 0.5167
```

Figure 2-7. The output from the `fit` method

Evaluating the Model

We know the model achieves an accuracy of 51.9% on the training set, but how does it perform on data it has never seen?

To answer this question we can use the `evaluate` method provided by Keras, as shown in Example 2-9.

Example 2-9. Evaluating the model performance on the test set

```
model.evaluate(x_test, y_test)
```

Figure 2-8 shows the output from this method.

```
10000/10000 [==============================] - 1s 55us/step

[1.4358007415771485, 0.4896]
```

Figure 2-8. The output from the `evaluate` method

The output is a list of the metrics we are monitoring: categorical cross-entropy and accuracy. We can see that model accuracy is still 49.0% even on images that it has never seen before. Note that if the model were guessing randomly, it would achieve approximately 10% accuracy (because there are 10 classes), so 49.0% is a good result, given that we have used a very basic neural network.

We can view some of the predictions on the test set using the `predict` method, as shown in Example 2-10.

Example 2-10. Viewing predictions on the test set using the `predict` method

```
CLASSES = np.array(['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog'
                    , 'frog', 'horse', 'ship', 'truck'])

preds = model.predict(x_test) ❶
preds_single = CLASSES[np.argmax(preds, axis = -1)] ❷
actual_single = CLASSES[np.argmax(y_test, axis = -1)]
```

❶ `preds` is an array of shape `[10000, 10]`—i.e., a vector of 10 class probabilities for each observation.

❷ We convert this array of probabilities back into a single prediction using `numpy`'s `argmax` function. Here, `axis = -1` tells the function to collapse the array over the last dimension (the classes dimension), so that the shape of `preds_single` is then `[10000, 1]`.

We can view some of the images alongside their labels and predictions with the code in Example 2-11. As expected, around half are correct.

Example 2-11. Displaying predictions of the MLP against the actual labels

```
import matplotlib.pyplot as plt

n_to_show = 10
indices = np.random.choice(range(len(x_test)), n_to_show)

fig = plt.figure(figsize=(15, 3))
fig.subplots_adjust(hspace=0.4, wspace=0.4)
```

```
for i, idx in enumerate(indices):
    img = x_test[idx]
    ax = fig.add_subplot(1, n_to_show, i+1)
    ax.axis('off')
    ax.text(0.5, -0.35, 'pred = ' + str(preds_single[idx]), fontsize=10
       , ha='center', transform=ax.transAxes)
    ax.text(0.5, -0.7, 'act = ' + str(actual_single[idx]), fontsize=10
        , ha='center', transform=ax.transAxes)
    ax.imshow(img)
```

Figure 2-9 shows a randomly chosen selection of predictions made by the model, alongside the true labels.



Figure 2-9. Some predictions made by the model, alongside the actual labels

Congratulations! You've just built a multilayer perceptron using Keras and used it to make predictions on new data. Even though this is a supervised learning problem, when we come to building generative models in future chapters many of the core ideas from this chapter (such as loss functions, activation functions, and understanding layer shapes) will still be extremely important. Next we'll look at ways of improving this model, by introducing a few new layer types.

Convolutional Neural Network (CNN)

One of the reasons our network isn't yet performing as well as it might is because there isn't anything in the network that takes into account the spatial structure of the input images. In fact, our first step is to flatten the image into a single vector, so that we can pass it to the first `Dense` layer!

To achieve this we need to use a *convolutional layer*.

Convolutional Layers

First, we need to understand what is meant by a *convolution* in the context of deep learning.

Figure 2-10 shows two different 3 × 3 × 1 portions of a grayscale image being convoluted with a 3 × 3 × 1 *filter* (or *kernel*). The convolution is performed by multiplying the filter pixelwise with the portion of the image, and summing the results. The output is more positive when the portion of the image closely matches the filter and more negative when the portion of the image is the inverse of the filter. The top example resonates strongly with the filter, so it produces a large positive value. The bottom example does not resonate much with the filter, so it produces a value near zero.

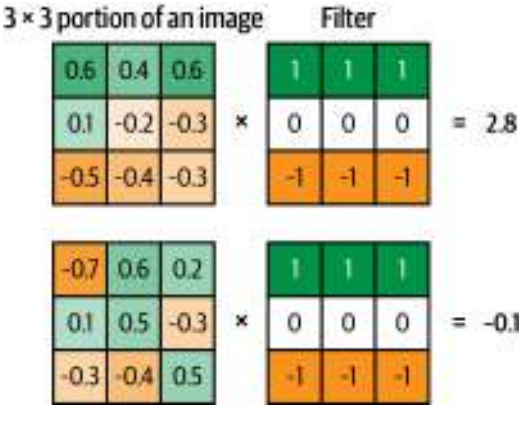


Figure 2-10. A 3 × 3 convolutional filter applied to two portions of a grayscale image

If we move the filter across the entire image from left to right and top to bottom, recording the convolutional output as we go, we obtain a new array that picks out a particular feature of the input, depending on the values in the filter. For example, Figure 2-11 shows two different filters that highlight horizontal and vertical edges.



Running the Code for This Example

You can see this convolutional process worked through manually in the Jupyter notebook located at *notebooks/02_deeplearning/02_cnn/convolutions.ipynb* in the book repository.

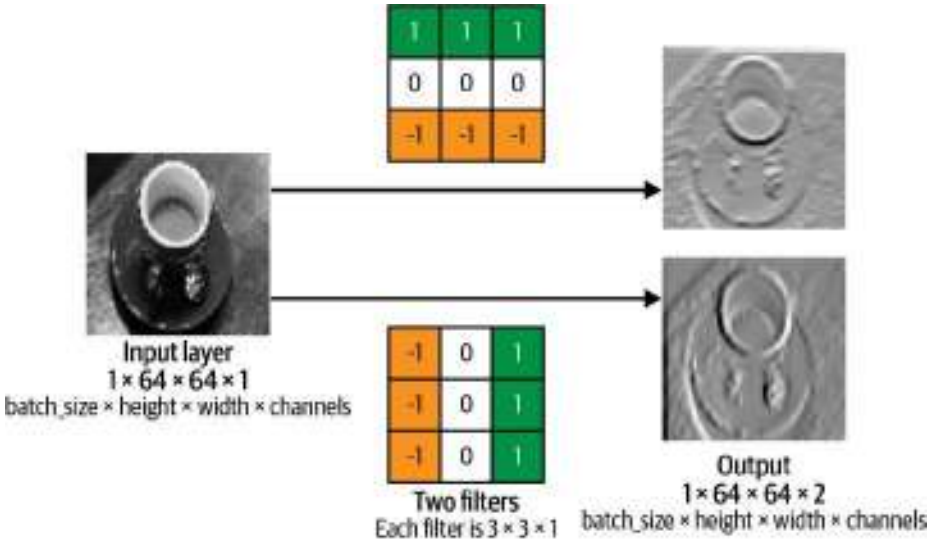


Figure 2-11. Two convolutional filters applied to a grayscale image

A convolutional layer is simply a collection of filters, where the values stored in the filters are the weights that are learned by the neural network through training. Initially these are random, but gradually the filters adapt their weights to start picking out interesting features such as edges or particular color combinations.

In Keras, the `Conv2D` layer applies convolutions to an input tensor with two spatial dimensions (such as an image). For example, the code shown in Example 2-12 builds a convolutional layer with two filters, to match the example in Figure 2-11.

Example 2-12. A `Conv2D` layer applied to grayscale input images

```
from tensorflow.keras import layers

input_layer = layers.Input(shape=(64,64,1))
conv_layer_1 = layers.Conv2D(
    filters = 2
    , kernel_size = (3,3)
    , strides = 1
    , padding = "same"
    )(input_layer)
```

Next, let's look at two of the arguments to the `Conv2D` layer in more detail—`strides` and `padding`.

Stride

The `strides` parameter is the step size used by the layer to move the filters across the input. Increasing the stride therefore reduces the size of the output tensor. For example, when `strides = 2`, the height and width of the output tensor will be half the size of the input tensor. This is useful for reducing the spatial size of the tensor as it passes through the network, while increasing the number of channels.

Padding

The `padding = "same"` input parameter pads the input data with zeros so that the output size from the layer is exactly the same as the input size when `strides = 1`.

Figure 2-12 shows a 3 × 3 kernel being passed over a 5 × 5 input image, with `padding = "same"` and `strides = 1`. The output size from this convolutional layer would also be 5 × 5, as the padding allows the kernel to extend over the edge of the image, so that it fits five times in both directions. Without padding, the kernel could only fit three times along each direction, giving an output size of 3 × 3.



Figure 2-12. A 3 × 3 × 1 kernel (gray) being passed over a 5 × 5 × 1 input image (blue), with `padding = "same"` and `strides = 1`, to generate the 5 × 5 × 1 output (green) (source: Dumoulin and Visin, 2018)[5]

Setting `padding = "same"` is a good way to ensure that you are able to easily keep track of the size of the tensor as it passes through many convolutional layers. The shape of the output from a convolutional layer with `padding = "same"` is:

$$\left(\frac{input\ height}{stride}, \frac{input\ width}{stride}, filters\right)$$

Stacking convolutional layers

The output of a `Conv2D` layer is another four-dimensional tensor, now of shape `(batch_size, height, width, filters)`, so we can stack `Conv2D` layers on top of each other to grow the depth of our neural network and make it more powerful. To demonstrate this, let's imagine we are applying `Conv2D` layers to the CIFAR-10 dataset and wish to predict the label of a given image. Note that this time, instead of one input channel (grayscale) we have three (red, green, and blue).

Example 2-13 shows how to build a simple convolutional neural network that we could train to succeed at this task.

Example 2-13. Code to build a convolutional neural network model using Keras

```
from tensorflow.keras import layers, models

input_layer = layers.Input(shape=(32,32,3))
conv_layer_1 = layers.Conv2D(
    filters = 10
    , kernel_size = (4,4)
    , strides = 2
    , padding = 'same'
    )(input_layer)
conv_layer_2 = layers.Conv2D(
    filters = 20
    , kernel_size = (3,3)
    , strides = 2
    , padding = 'same'
    )(conv_layer_1)
flatten_layer = layers.Flatten()(conv_layer_2)
output_layer = layers.Dense(units=10, activation = 'softmax')(flatten_layer)
model = models.Model(input_layer, output_layer)
```

This code corresponds to the diagram shown in Figure 2-13.

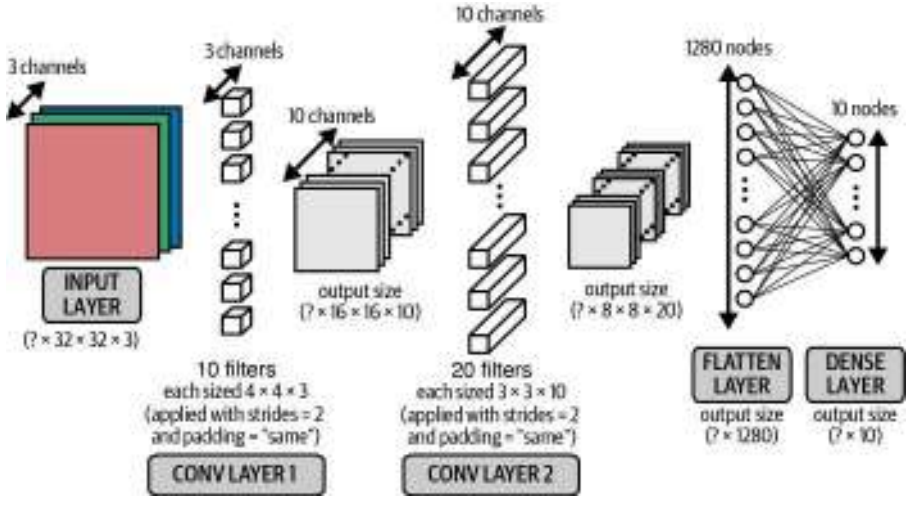


Figure 2-13. A diagram of a convolutional neural network

Note that now that we are working with color images, each filter in the first convolutional layer has a depth of 3 rather than 1 (i.e., each filter has shape 4 × 4 × 3, rather than 4 × 4 × 1). This is to match the three channels (red, green, blue) of the input

image. The same idea applies to the filters in the second convolutional layer that have a depth of 10, to match the 10 channels output by the first convolutional layer.



In general, the depth of the filters in a layer is always equal to the number of channels output by the preceding layer.

Inspecting the model

It's really informative to look at how the shape of the tensor changes as data flows through from one convolutional layer to the next. We can use the `model.summary()` method to inspect the shape of the tensor as it passes through the network (Table 2-2).

Table 2-2. CNN model summary

Layer (type)	Output shape	Param #
InputLayer	(None, 32, 32, 3)	0
Conv2D	(None, 16, 16, 10)	490
Conv2D	(None, 8, 8, 20)	1,820
Flatten	(None, 1280)	0
Dense	(None, 10)	12,810

Total params	15,120
Trainable params	15,120
Non-trainable params	0

Let's walk through our network layer by layer, noting the shape of the tensor as we go:

1. The input shape is `(None, 32, 32, 3)`—Keras uses `None` to represent the fact that we can pass any number of images through the network simultaneously. Since the network is just performing tensor algebra, we don't need to pass images through the network individually, but instead can pass them through together as a batch.
2. The shape of each of the 10 filters in the first convolutional layer is $4 \times 4 \times 3$. This is because we have chosen each filter to have a height and width of 4 (`kernel_size = (4,4)`) and there are three channels in the preceding layer (red, green, and blue). Therefore, the number of parameters (or weights) in the layer is $(4 \times 4 \times 3 + 1) \times 10 = 490$, where the $+ 1$ is due to the inclusion of a bias term attached to each of the filters. The output from each filter will be the pixelwise multiplication of the filter weights and the $4 \times 4 \times 3$ section of the image it is

covering. As `strides = 2` and `padding = "same"`, the width and height of the output are both halved to 16, and since there are 10 filters the output of the first layer is a batch of tensors each having shape `[16, 16, 10]`.

3. In the second convolutional layer, we choose the filters to be 3 × 3 and they now have depth 10, to match the number of channels in the previous layer. Since there are 20 filters in this layer, this gives a total number of parameters (weights) of (3 × 3 × 10 + 1) × 20 = 1,820. Again, we use `strides = 2` and `padding = "same"`, so the width and height both halve. This gives us an overall output shape of `(None, 8, 8, 20)`.
4. We now flatten the tensor using the Keras `Flatten` layer. This results in a set of 8 × 8 × 20 = 1,280 units. Note that there are no parameters to learn in a `Flatten` layer as the operation is just a restructuring of the tensor.
5. We finally connect these units to a 10-unit `Dense` layer with softmax activation, which represents the probability of each category in a 10-category classification task. This creates an extra 1,280 × 10 = 12,810 parameters (weights) to learn.

This example demonstrates how we can chain convolutional layers together to create a convolutional neural network. Before we see how this compares in accuracy to our densely connected neural network, we'll examine two more techniques that can also improve performance: batch normalization and dropout.

Batch Normalization

One common problem when training a deep neural network is ensuring that the weights of the network remain within a reasonable range of values—if they start to become too large, this is a sign that your network is suffering from what is known as the *exploding gradient* problem. As errors are propagated backward through the network, the calculation of the gradient in the earlier layers can sometimes grow exponentially large, causing wild fluctuations in the weight values.



If your loss function starts to return `NaN`, chances are that your weights have grown large enough to cause an overflow error.

This doesn't necessarily happen immediately as you start training the network. Sometimes it can be happily training for hours when suddenly the loss function returns `NaN` and your network has exploded. This can be incredibly annoying. To prevent it from happening, you need to understand the root cause of the exploding gradient problem.

Covariate shift

One of the reasons for scaling input data to a neural network is to ensure a stable start to training over the first few iterations. Since the weights of the network are initially randomized, unscaled input could potentially create huge activation values that immediately lead to exploding gradients. For example, instead of passing pixel values from 0–255 into the input layer, we usually scale these values to between –1 and 1.

Because the input is scaled, it's natural to expect the activations from all future layers to be relatively well scaled as well. Initially this may be true, but as the network trains and the weights move further away from their random initial values, this assumption can start to break down. This phenomenon is known as *covariate shift*.



Covariate Shift Analogy

Imagine you're carrying a tall pile of books, and you get hit by a gust of wind. You move the books in a direction opposite to the wind to compensate, but as you do so, some of the books shift, so that the tower is slightly more unstable than before. Initially, this is OK, but with every gust the pile becomes more and more unstable, until eventually the books have shifted so much that the pile collapses. This is covariate shift.

Relating this to neural networks, each layer is like a book in the pile. To remain stable, when the network updates the weights, each layer implicitly assumes that the distribution of its input from the layer beneath is approximately consistent across iterations. However, since there is nothing to stop any of the activation distributions shifting significantly in a certain direction, this can sometimes lead to runaway weight values and an overall collapse of the network.

Training using batch normalization

Batch normalization is a technique that drastically reduces this problem. The solution is surprisingly simple. During training, a batch normalization layer calculates the mean and standard deviation of each of its input channels across the batch and normalizes by subtracting the mean and dividing by the standard deviation. There are then two learned parameters for each channel, the scale (gamma) and shift (beta). The output is simply the normalized input, scaled by gamma and shifted by beta. Figure 2-14 shows the whole process.

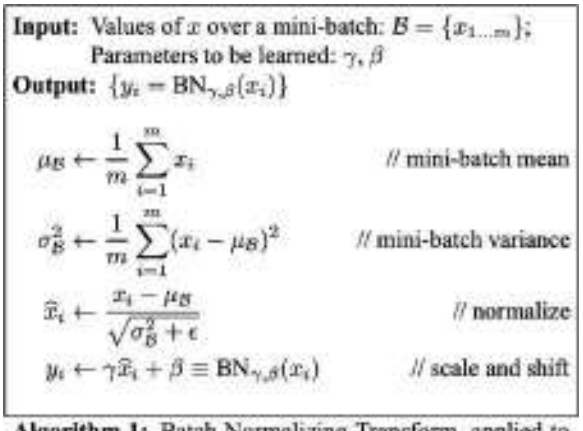


Figure 2-14. The batch normalization process (source: Ioffe and Szegedy, 2015)[6]

We can place batch normalization layers after dense or convolutional layers to normalize the output.



Referring to our previous example, it's a bit like connecting the layers of books with small sets of adjustable springs that ensure there aren't any overall huge shifts in their positions over time.

Prediction using batch normalization

You might be wondering how this layer works at prediction time. When it comes to prediction, we may only want to predict a single observation, so there is no *batch* over which to calculate the mean and standard deviation. To get around this problem, during training a batch normalization layer also calculates the moving average of the mean and standard deviation of each channel and stores this value as part of the layer to use at test time.

How many parameters are contained within a batch normalization layer? For every channel in the preceding layer, two weights need to be learned: the scale (gamma) and shift (beta). These are the *trainable* parameters. The moving average and standard deviation also need to be calculated for each channel, but since they are derived from the data passing through the layer rather than trained through backpropagation, they are called *nontrainable* parameters. In total, this gives four parameters for each channel in the preceding layer, where two are trainable and two are nontrainable.

In Keras, the `BatchNormalization` layer implements the batch normalization functionality, as shown in Example 2-14.

Example 2-14. A `BatchNormalization` layer in Keras

```
from tensorflow.keras import layers
layers.BatchNormalization(momentum = 0.9)
```

The `momentum` parameter is the weight given to the previous value when calculating the moving average and moving standard deviation.

Dropout

When studying for an exam, it is common practice for students to use past papers and sample questions to improve their knowledge of the subject material. Some students try to memorize the answers to these questions, but then come unstuck in the exam because they haven't truly understood the subject matter. The best students use the practice material to further their general understanding, so that they are still able to answer correctly when faced with new questions that they haven't seen before.

The same principle holds for machine learning. Any successful machine learning algorithm must ensure that it generalizes to unseen data, rather than simply *remembering* the training dataset. If an algorithm performs well on the training dataset, but not the test dataset, we say that it is suffering from *overfitting*. To counteract this problem, we use *regularization* techniques, which ensure that the model is penalized if it starts to overfit.

There are many ways to regularize a machine learning algorithm, but for deep learning, one of the most common is by using *dropout* layers. This idea was introduced by Hinton et al. in 2012[7] and presented in a 2014 paper by Srivastava et al.[8]

Dropout layers are very simple. During training, each dropout layer chooses a random set of units from the preceding layer and sets their output to 0, as shown in Figure 2-15.

Incredibly, this simple addition drastically reduces overfitting by ensuring that the network doesn't become overdependent on certain units or groups of units that, in effect, just remember observations from the training set. If we use dropout layers, the network cannot rely too much on any one unit and therefore knowledge is more evenly spread across the whole network.

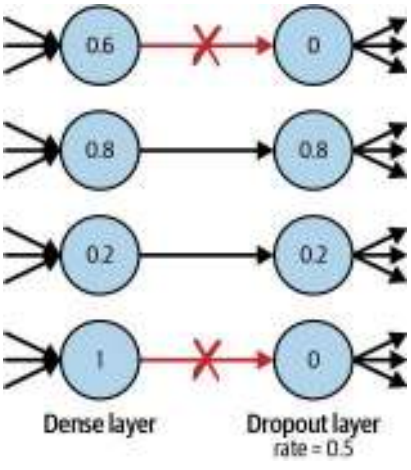


Figure 2-15. A dropout layer

This makes the model much better at generalizing to unseen data, because the network has been trained to produce accurate predictions even under unfamiliar conditions, such as those caused by dropping random units. There are no weights to learn within a dropout layer, as the units to drop are decided stochastically. At prediction time, the dropout layer doesn't drop any units, so that the full network is used to make predictions.



Dropout Analogy

Returning to our analogy, it's a bit like a math student practicing past papers with a random selection of key formulae missing from their formula book. This way, they learn how to answer questions through an understanding of the core principles, rather than always looking up the formulae in the same places in the book. When it comes to test time, they will find it much easier to answer questions that they have never seen before, due to their ability to generalize beyond the training material.

The `Dropout` layer in Keras implements this functionality, with the `rate` parameter specifying the proportion of units to drop from the preceding layer, as shown in Example 2-15.

Example 2-15. A `Dropout` layer in Keras

```
from tensorflow.keras import layers
layers.Dropout(rate = 0.25)
```

Dropout layers are used most commonly after dense layers since these are the most prone to overfitting due to the higher number of weights, though you can also use them after convolutional layers.



Batch normalization also has been shown to reduce overfitting, and therefore many modern deep learning architectures don't use dropout at all, relying solely on batch normalization for regularization. As with most deep learning principles, there is no golden rule that applies in every situation—the only way to know for sure what's best is to test different architectures and see which performs best on a holdout set of data.

Building the CNN

You've now seen three new Keras layer types: `Conv2D`, `BatchNormalization`, and `Dropout`. Let's put these pieces together into a CNN model and see how it performs on the CIFAR-10 dataset.



Running the Code for This Example

You can run the following example in the Jupyter notebook in the book repository called *notebooks/02_deeplearning/02_cnn/cnn.ipynb*.

The model architecture we shall test is shown in Example 2-16.

Example 2-16. Code to build a CNN model using Keras

```
from tensorflow.keras import layers, models

input_layer = layers.Input((32,32,3))

x = layers.Conv2D(filters = 32, kernel_size = 3
        , strides = 1, padding = 'same')(input_layer)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)

x = layers.Conv2D(filters = 32, kernel_size = 3, strides = 2, padding = 'same')(x)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)

x = layers.Conv2D(filters = 64, kernel_size = 3, strides = 1, padding = 'same')(x)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)

x = layers.Conv2D(filters = 64, kernel_size = 3, strides = 2, padding = 'same')(x)
x = layers.BatchNormalization()(x)
```

```
x = layers.LeakyReLU()(x)

x = layers.Flatten()(x)

x = layers.Dense(128)(x)
x = layers.BatchNormalization()(x)
x = layers.LeakyReLU()(x)
x = layers.Dropout(rate = 0.5)(x)

output_layer = layers.Dense(10, activation = 'softmax')(x)

model = models.Model(input_layer, output_layer)
```

We use four stacked `Conv2D` layers, each followed by a `BatchNormalization` and a `LeakyReLU` layer. After flattening the resulting tensor, we pass the data through a `Dense` layer of size 128, again followed by a `BatchNormalization` and a `LeakyReLU` layer. This is immediately followed by a `Dropout` layer for regularization, and the network is concluded with an output `Dense` layer of size 10.



The order in which to use the batch normalization and activation layers is a matter of preference. Usually batch normalization layers are placed before the activation, but some successful architectures use these layers the other way around. If you do choose to use batch normalization before activation, you can remember the order using the acronym *BAD* (batch normalization, activation, then dropout)!

The model summary is shown in Table 2-3.

Table 2-3. Model summary of the CNN for CIFAR-10

Layer (type)	Output shape	Param #
InputLayer	(None, 32, 32, 3)	0
Conv2D	(None, 32, 32, 32)	896
BatchNormalization	(None, 32, 32, 32)	128
LeakyReLU	(None, 32, 32, 32)	0
Conv2D	(None, 16, 16, 32)	9,248
BatchNormalization	(None, 16, 16, 32)	128
LeakyReLU	(None, 16, 16, 32)	0
Conv2D	(None, 16, 16, 64)	18,496
BatchNormalization	(None, 16, 16, 64)	256
LeakyReLU	(None, 16, 16, 64)	0
Conv2D	(None, 8, 8, 64)	36,928
BatchNormalization	(None, 8, 8, 64)	256

Layer (type)	Output shape	Param #
LeakyReLU	(None, 8, 8, 64)	0
Flatten	(None, 4096)	0
Dense	(None, 128)	524,416
BatchNormalization	(None, 128)	512
LeakyReLU	(None, 128)	0
Dropout	(None, 128)	0
Dense	(None, 10)	1290

Total params	592,554
Trainable params	591,914
Non-trainable params	640



Before moving on, make sure you are able to calculate the output shape and number of parameters for each layer by hand. It's a good exercise to prove to yourself that you have fully understood how each layer is constructed and how it is connected to the preceding layer! Don't forget to include the bias weights that are included as part of the `Conv2D` and `Dense` layers.

Training and Evaluating the CNN

We compile and train the model in exactly the same way as before and call the `evaluate` method to determine its accuracy on the holdout set (Figure 2-16).

```
model.evaluate(x_test, y_test, batch_size=1000)
 10000/10000 [==============================] - 15s 1ms/step
[0.8423407137393951, 0.7155999958515167]
```

Figure 2-16. CNN performance

As you can see, this model is now achieving 71.5% accuracy, up from 49.0% previously. Much better! Figure 2-17 shows some predictions from our new convolutional model.

This improvement has been achieved simply by changing the architecture of the model to include convolutional, batch normalization, and dropout layers. Notice that the number of parameters is actually fewer in our new model than the previous model, even though the number of layers is far greater. This demonstrates the importance of being experimental with your model design and being comfortable with how the different layer types can be used to your advantage. When building generative

models, it becomes even more important to understand the inner workings of your model since it is the middle layers of your network that capture the high-level features that you are most interested in.



Figure 2-17. CNN predictions

Summary

This chapter introduced the core deep learning concepts that you will need to start building deep generative models. We started by building a multilayer perceptron (MLP) using Keras and trained the model to predict the category of a given image from the CIFAR-10 dataset. Then, we improved upon this architecture by introducing convolutional, batch normalization, and dropout layers to create a convolutional neural network (CNN).

A really important point to take away from this chapter is that deep neural networks are completely flexible by design, and there really are no fixed rules when it comes to model architecture. There are guidelines and best practices, but you should feel free to experiment with layers and the order in which they appear. Don't feel constrained to only use the architectures that you have read about in this book or elsewhere! Like a child with a set of building blocks, the design of your neural network is only limited by your own imagination.

In the next chapter, we shall see how we can use these building blocks to design a network that can generate images.

References

1. Kaiming He et al., "Deep Residual Learning for Image Recognition," December 10, 2015, *https://arxiv.org/abs/1512.03385*.

2. Alex Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," April 8, 2009, *https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf*.

3. Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization," December 22, 2014, *https://arxiv.org/abs/1412.6980v8*.

4. Samuel L. Smith et al., "Don't Decay the Learning Rate, Increase the Batch Size," November 1, 2017, *https://arxiv.org/abs/1711.00489*.

5. Vincent Dumoulin and Francesco Visin, "A Guide to Convolution Arithmetic for Deep Learning," January 12, 2018, *https://arxiv.org/abs/1603.07285*.

6. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," February 11, 2015, *https://arxiv.org/abs/1502.03167*.

7. Hinton et al., "Networks by Preventing Co-Adaptation of Feature Detectors," July 3, 2012, *https://arxiv.org/abs/1207.0580*.

8. Nitish Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research* 15 (2014): 1929–1958, *http://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf*.

PART II
Methods

In Part II we will dive into the six families of generative models, including the theory behind how they work and practical examples of how to build each type of model.

In Chapter 3 we shall take a look at our first generative deep learning model, the *variational autoencoder*. This technique will allow us to not only generate realistic faces, but also alter existing images—for example, by adding a smile or changing the color of someone's hair.

Chapter 4 explores one of the most successful generative modeling techniques of recent years, the *generative adversarial network*. We shall see the ways that GAN training has been fine-tuned and adapted to continually push the boundaries of what generative modeling is able to achieve.

In Chapter 5 we will delve into several examples of *autoregressive models*, including LSTMs and PixelCNN. This family of models treats the generation process as a sequence prediction problem—it underpins today's state-of-the-art text generation models and can also be used for image generation.

In Chapter 6 we will cover the family of *normalizing flow models*, including RealNVP. This model is based on a change of variables formula, which allows the transformation of a simple distribution, such as a Gaussian distribution, into a more complex distribution in way that preserves tractability.

Chapter 7 introduces the family of *energy-based models*. These models train a scalar energy function to score the validity of a given input. We will explore a technique for training energy-based models called contrastive divergence and a technique for sampling new observations called Langevin dynamics.

Finally, in Chapter 8 we shall explore the family of *diffusion models*. This technique is based on the idea of iteratively adding noise to an image and then training a model to remove the noise, giving us the ability to transform pure noise into realistic samples.

By the end of Part II you will have built practical examples of generative models from each of the six generative modeling families and be able to explain how each works from a theoretical perspective.

CHAPTER 3
Variational Autoencoders

Chapter Goals

In this chapter you will:

- Learn how the architectural design of autoencoders makes them perfectly suited to generative modeling.
- Build and train an autoencoder from scratch using Keras.
- Use autoencoders to generate new images, but understand the limitations of this approach.
- Learn about the architecture of the variational autoencoder and how it solves many of the problems associated with standard autoencoders.
- Build a variational autoencoder from scratch using Keras.
- Use variational autoencoders to generate new images.
- Use variational autoencoders to manipulate generated images using latent space arithmetic.

In 2013, Diederik P. Kingma and Max Welling published a paper that laid the foundations for a type of neural network known as a *variational autoencoder* (VAE).[1] This is now one of the most fundamental and well-known deep learning architectures for generative modeling and an excellent place to start our journey into generative deep learning.

In this chapter, we shall start by building a standard autoencoder and then see how we can extend this framework to develop a variational autoencoder. Along the way, we will pick apart both types of models, to understand how they work at a granular level. By the end of the chapter you should have a complete understanding of how to

build and manipulate autoencoder-based models and, in particular, how to build a variational autoencoder from scratch to generate images based on your own dataset.

Introduction

Let's start with a simple story that will help to explain the fundamental problem that an autoencoder is trying to solve.

Brian, the Stitch, and the Wardrobe

Imagine that on the floor in front of you is a pile of all the clothing you own—trousers, tops, shoes, and coats, all of different styles. Your stylist, Brian, is becoming increasingly frustrated with how long it takes him to find the items you require, so he devises a clever plan.

He tells you to organize your clothes into a wardrobe that is infinitely high and wide (Figure 3-1). When you want to request a particular item, you simply need to tell Brian its location and he will sew the item from scratch using his trusty sewing machine. It soon becomes obvious that you will need to place similar items near to each other, so that Brian can accurately re-create each item given only its location.



Figure 3-1. A man standing in front of an infinite 2D wardrobe (created with Midjourney)

After several weeks of practice, you and Brian have adjusted to each other's understandings of the wardrobe layout. It is now possible for you to tell Brian the location of any item of clothing that you desire, and he can accurately sew it from scratch!

This gives you an idea—what would happen if you gave Brian a wardrobe location that was empty? To your amazement, you find that Brian is able to generate entirely

new items of clothing that haven't existed before! The process isn't perfect, but you now have limitless options for generating new clothing, just by picking an empty location in the infinite wardrobe and letting Brian work his magic with the sewing machine.

Let's now explore how this story relates to building autoencoders.

Autoencoders

A diagram of the process described by the story is shown in Figure 3-2. You play the part of the *encoder*, moving each item of clothing to a location in the wardrobe. This process is called *encoding*. Brian plays the part of the *decoder*, taking a location in the wardrobe and attempting to re-create the item. This process is called *decoding*.

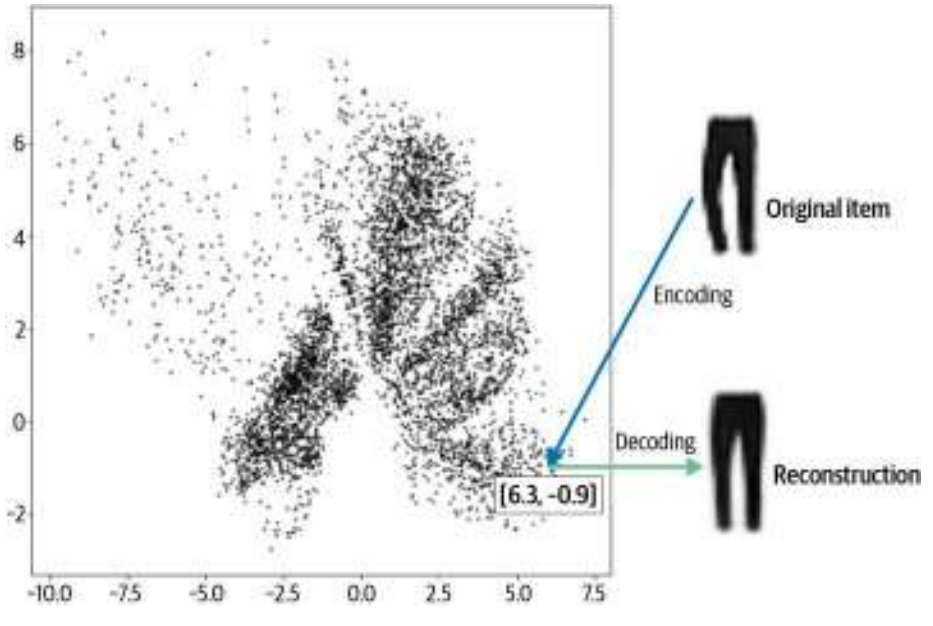


Figure 3-2. Items of clothing in the infinite wardrobe—each black dot represents an item of clothing

Each location in the wardrobe is represented by two numbers (i.e., a 2D vector). For example, the trousers in Figure 3-2 are encoded to the point [6.3, –0.9]. This vector is also known as an *embedding* because the encoder attempts to embed as much information into it as possible, so that the decoder can produce an accurate reconstruction.

An *autoencoder* is simply a neural network that is trained to perform the task of encoding and decoding an item, such that the output from this process is as close to the original item as possible. Crucially, it can be used as a generative model, because we can decode any point in the 2D space that we want (in particular, those that are not embeddings of original items) to produce a novel item of clothing.

Let's now see how we can build an autoencoder using Keras and apply it to a real dataset!



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/03_vae/01_autoencoder/autoencoder.ipynb* in the book repository.

The Fashion-MNIST Dataset

For this example, we'll be using the Fashion-MNIST dataset—a collection of grayscale images of clothing items, each of size 28 × 28 pixels. Some example images from the dataset are shown in Figure 3-3.



Figure 3-3. Examples of images from the Fashion-MNIST dataset

The dataset comes prepackaged with TensorFlow, so it can be downloaded as shown in Example 3-1.

Example 3-1. Loading the Fashion-MNIST dataset

```
from tensorflow.keras import datasets
(x_train,y_train), (x_test,y_test) = datasets.fashion_mnist.load_data()
```

These are 28 × 28 grayscale images (pixel values between 0 and 255) out of the box, which we need to preprocess to ensure that the pixel values are scaled between 0 and 1. We will also pad each image to 32 × 32 for easier manipulation of the tensor shape as it passes through the network, as shown in Example 3-2.

Example 3-2. Preprocessing the data

```
def preprocess(imgs):
    imgs = imgs.astype("float32") / 255.0
    imgs = np.pad(imgs, ((0, 0), (2, 2), (2, 2)), constant_values=0.0)
    imgs = np.expand_dims(imgs, -1)
    return imgs

x_train = preprocess(x_train)
x_test = preprocess(x_test)
```

Next, we need to understand the overall structure of an autoencoder, so that we can code it up using TensorFlow and Keras.

The Autoencoder Architecture

An *autoencoder* is a neural network made up of two parts:

- An *encoder* network that compresses high-dimensional input data such as an image into a lower-dimensional embedding vector
- A *decoder* network that decompresses a given embedding vector back to the original domain (e.g., back to an image)

A diagram of the network architecture is shown in Figure 3-4. An input image is encoded to a latent embedding vector z, which is then decoded back to the original pixel space.

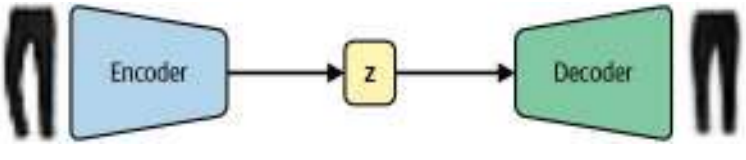


Figure 3-4. Autoencoder architecture diagram

The autoencoder is trained to reconstruct an image, after it has passed through the encoder and back out through the decoder. This may seem strange at first—why would you want to reconstruct a set of images that you already have available to you? However, as we shall see, it is the embedding space (also called the *latent space*) that is the interesting part of the autoencoder, as sampling from this space will allow us to generate new images.

Let's first define what we mean by an embedding. The embedding (z) is a compression of the original image into a lower-dimensional latent space. The idea is that by choosing any point in the latent space, we can generate novel images by passing this point through the decoder, since the decoder has learned how to convert points in the latent space into viable images.

In our example, we will embed images into a two-dimensional latent space. This will help us to visualize the latent space, since we can easily plot points in 2D. In practice, the latent space of an autoencoder will usually have more than two dimensions in order to have more freedom to capture greater nuance in the images.



Autoencoders as Denoising Models

Autoencoders can be used to clean noisy images, since the encoder learns that it is not useful to capture the position of the random noise inside the latent space in order to reconstruct the original. For tasks such as this, a 2D latent space is probably too small to encode sufficient relevant information from the input. However, as we shall see, increasing the dimensionality of the latent space quickly leads to problems if we want to use the autoencoder as a generative model.

Let's now see how to build the encoder and decoder.

The Encoder

In an autoencoder, the encoder's job is to take the input image and map it to an embedding vector in the latent space. The architecture of the encoder we will be building is shown in Table 3-1.

Table 3-1. Model summary of the encoder

Layer (type)	Output shape	Param #
InputLayer	(None, 32, 32, 1)	0
Conv2D	(None, 16, 16, 32)	320
Conv2D	(None, 8, 8, 64)	18,496
Conv2D	(None, 4, 4, 128)	73,856
Flatten	(None, 2048)	0
Dense	(None, 2)	4,098

Total params	96,770
Trainable params	96,770
Non-trainable params	0

To achieve this, we first create an `Input` layer for the image and pass this through three `Conv2D` layers in sequence, each capturing increasingly high-level features. We use a stride of 2 to halve the size of the output of each layer, while increasing the number of channels. The last convolutional layer is flattened and connected to a `Dense` layer of size 2, which represents our two-dimensional latent space.

Example 3-3 shows how to build this in Keras.

Example 3-3. The encoder

```
encoder_input = layers.Input(
    shape=(32, 32, 1), name = "encoder_input"
) ❶
x = layers.Conv2D(32, (3, 3), strides = 2, activation = 'relu', padding="same")(
    encoder_input
) ❷
x = layers.Conv2D(64, (3, 3), strides = 2, activation = 'relu', padding="same")(x)
x = layers.Conv2D(128, (3, 3), strides = 2, activation = 'relu', padding="same")(x)
shape_before_flattening = K.int_shape(x)[1:]

x = layers.Flatten()(x) ❸
encoder_output = layers.Dense(2, name="encoder_output")(x) ❹

encoder = models.Model(encoder_input, encoder_output) ❺
```

❶ Define the `Input` layer of the encoder (the image).

❷ Stack `Conv2D` layers sequentially on top of each other.

❸ Flatten the last convolutional layer to a vector.

❹ Connect this vector to the 2D embeddings with a `Dense` layer.

❺ The Keras `Model` that defines the encoder—a model that takes an input image and encodes it into a 2D embedding.



I strongly encourage you to experiment with the number of convolutional layers and filters to understand how the architecture affects the overall number of model parameters, model performance, and model runtime.

The Decoder

The decoder is a mirror image of the encoder—instead of convolutional layers, we use *convolutional transpose* layers, as shown in Table 3-2.

Table 3-2. Model summary of the decoder

Layer (type)	Output shape	Param #
InputLayer	(None, 2)	0
Dense	(None, 2048)	6,144

Layer (type)	Output shape	Param #
Reshape	(None, 4, 4, 128)	0
Conv2DTranspose	(None, 8, 8, 128)	147,584
Conv2DTranspose	(None, 16, 16, 64)	73,792
Conv2DTranspose	(None, 32, 32, 32)	18,464
Conv2D	(None, 32, 32, 1)	289

Total params	246,273
Trainable params	246,273
Non-trainable params	0

Convolutional Transpose Layers

Standard convolutional layers allow us to halve the size of an input tensor in both dimensions (height and width), by setting `strides = 2`.

The convolutional transpose layer uses the same principle as a standard convolutional layer (passing a filter across the image), but is different in that setting `strides = 2` *doubles* the size of the input tensor in both dimensions.

In a convolutional transpose layer, the `strides` parameter determines the internal zero padding between pixels in the image, as shown in Figure 3-5. Here, a $3 \times 3 \times 1$ filter (gray) is being passed across a $3 \times 3 \times 1$ image (blue) with `strides = 2`, to produce a $6 \times 6 \times 1$ output tensor (green).

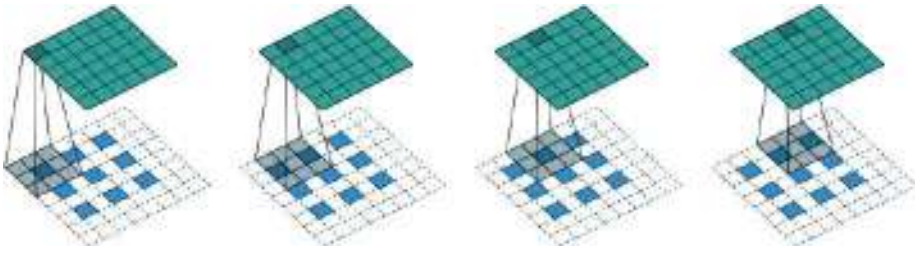


Figure 3-5. A convolutional transpose layer example (source: Dumoulin and Visin, 2018)[2]

In Keras, the `Conv2DTranspose` layer allows us to perform convolutional transpose operations on tensors. By stacking these layers, we can gradually expand the size of each layer, using strides of 2, until we get back to the original image dimension of 32×32.

Example 3-4 shows how we build the decoder in Keras.

Example 3-4. The decoder

```
decoder_input = layers.Input(shape=(2,), name="decoder_input") ❶
x = layers.Dense(np.prod(shape_before_flattening))(decoder_input) ❷
x = layers.Reshape(shape_before_flattening)(x) ❸
x = layers.Conv2DTranspose(
    128, (3, 3), strides=2, activation = 'relu', padding="same"
)(x) ❹
x = layers.Conv2DTranspose(
    64, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
x = layers.Conv2DTranspose(
    32, (3, 3), strides=2, activation = 'relu', padding="same"
)(x)
decoder_output = layers.Conv2D(
    1,
    (3, 3),
    strides = 1,
    activation="sigmoid",
    padding="same",
    name="decoder_output"
)(x)

decoder = models.Model(decoder_input, decoder_output) ❺
```

❶ Define the `Input` layer of the decoder (the embedding).

❷ Connect the input to a `Dense` layer.

❸ `Reshape` this vector into a tensor that can be fed as input into the first `Conv2DTranspose` layer.

❹ Stack `Conv2DTranspose` layers on top of each other.

❺ The Keras `Model` that defines the decoder—a model that takes an embedding in the latent space and decodes it into the original image domain.

Joining the Encoder to the Decoder

To train the encoder and decoder simultaneously, we need to define a model that will represent the flow of an image through the encoder and back out through the decoder. Luckily, Keras makes it extremely easy to do this, as you can see in Example 3-5. Notice the way in which we specify that the output from the autoencoder is simply the output from the encoder after it has been passed through the decoder.

Example 3-5. The full autoencoder

```
autoencoder = Model(encoder_input, decoder(encoder_output)) ❶
```

❶ The Keras `Model` that defines the full autoencoder—a model that takes an image and passes it through the encoder and back out through the decoder to generate a reconstruction of the original image.

Now that we've defined our model, we just need to compile it with a loss function and optimizer, as shown in Example 3-6. The loss function is usually chosen to be either the root mean squared error (RMSE) or binary cross-entropy between the individual pixels of the original image and the reconstruction.

Example 3-6. Compiling the autoencoder

```
# Compile the autoencoder
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
```

Choosing the Loss Function

Optimizing for RMSE means that your generated output will be symmetrically distributed around the average pixel values (because an overestimation is penalized equivalently to an underestimation).

On the other hand, binary cross-entropy loss is asymmetrical—it penalizes errors toward the extremes more heavily than errors toward the center. For example, if the true pixel value is high (say 0.7), then generating a pixel with value 0.8 is penalized more heavily than generating a pixel with value 0.6. If the true pixel value is low (say 0.3), then generating a pixel with value 0.2 is penalized more heavily than generating a pixel with value 0.4.

This has the effect of binary cross-entropy loss producing slightly blurrier images than RMSE loss (as it tends to push predictions toward 0.5), but sometimes this is desirable as RMSE can lead to obviously pixelized edges.

There is no right or wrong choice—you should choose whichever works best for your use case after experimentation.

We can now train the autoencoder by passing in the input images as both the input and output, as shown in Example 3-7.

Example 3-7. Training the autoencoder

```
autoencoder.fit(
    x_train,
    x_train,
    epochs=5,
    batch_size=100,
    shuffle=True,
    validation_data=(x_test, x_test),
)
```

Now that our autoencoder is trained, the first thing we need to check is that it is able to accurately reconstruct the input images.

Reconstructing Images

We can test the ability to reconstruct images by passing images from the test set through the autoencoder and comparing the output to the original images. The code for this is shown in Example 3-8.

Example 3-8. Reconstructing images using the autoencoder

```
example_images = x_test[:5000]
predictions = autoencoder.predict(example_images)
```

In Figure 3-6 you can see some examples of original images (top row), the 2D vectors after encoding, and the reconstructed items after decoding (bottom row).

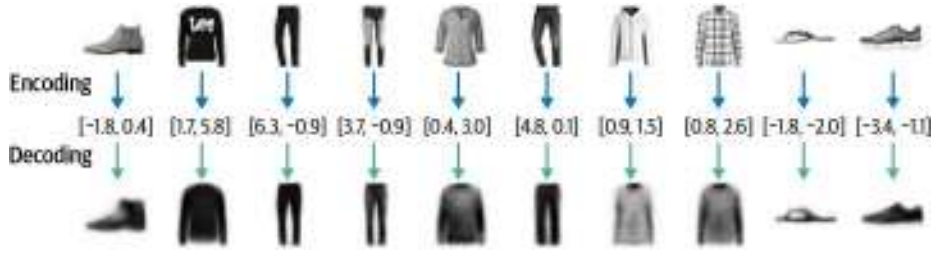


Figure 3-6. Examples of encoding and decoding items of clothing

Notice how the reconstruction isn't perfect—there are still some details of the original images that aren't captured by the decoding process, such as logos. This is because by reducing each image to just two numbers, we naturally lose some information.

Let's now investigate how the encoder is representing images in the latent space.

Visualizing the Latent Space

We can visualize how images are embedded into the latent space by passing the test set through the encoder and plotting the resulting embeddings, as shown in Example 3-9.

Example 3-9. Embedding images using the encoder

```
embeddings = encoder.predict(example_images)

plt.figure(figsize=(8, 8))
plt.scatter(embeddings[:, 0], embeddings[:, 1], c="black", alpha=0.5, s=3)
plt.show()
```

The resulting plot is the scatter plot shown in Figure 3-2—each black point represents an image that has been embedded into the latent space.

In order to better understand how this latent space is structured, we can make use of the labels that come with the Fashion-MNIST dataset, describing the type of item in each image. There are 10 groups altogether, shown in Table 3-3.

Table 3-3. The Fashion-MNIST labels

ID	Clothing label
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

We can color each point based on the label of the corresponding image to produce the plot in Figure 3-7. Now the structure becomes very clear! Even though the clothing labels were never shown to the model during training, the autoencoder has naturally grouped items that look alike into the same parts of the latent space. For example, the dark blue cloud of points in the bottom-right corner of the latent space are all different images of trousers and the red cloud of points toward the center are all ankle boots.

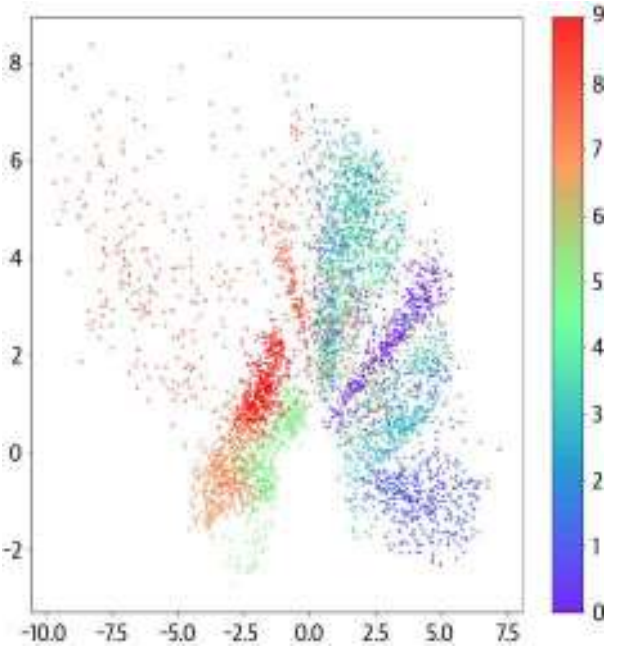


Figure 3-7. Plot of the latent space, colored by clothing label

Generating New Images

We can generate novel images by sampling some points in the latent space and using the decoder to convert these back into pixel space, as shown in Example 3-10.

Example 3-10. Generating novel images using the decoder

```
mins, maxs = np.min(embeddings, axis=0), np.max(embeddings, axis=0)
sample = np.random.uniform(mins, maxs, size=(18, 2))
reconstructions = decoder.predict(sample)
```

Some examples of generated images are shown in Figure 3-8, alongside their embeddings in the latent space.

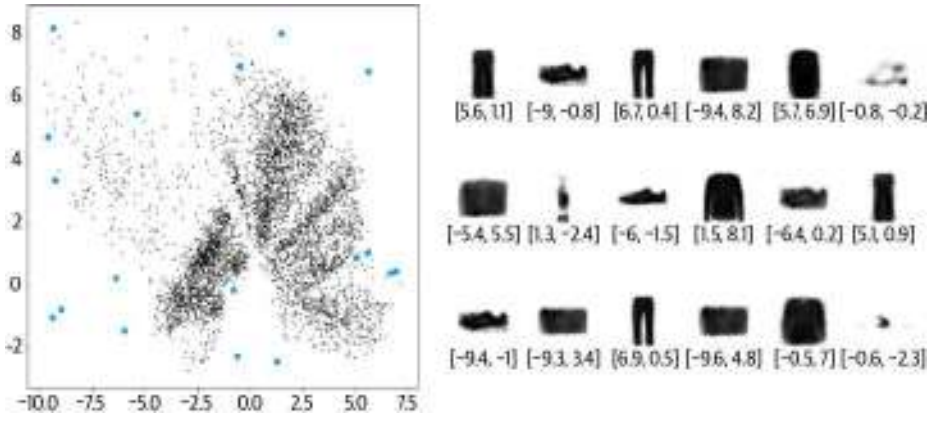


Figure 3-8. Generated items of clothing

Each blue dot maps to one of the images shown on the right of the diagram, with the embedding vector shown underneath. Notice how some of the generated items are more realistic than others. Why is this?

To answer this, let's first make a few observations about the overall distribution of points in the latent space, referring back to Figure 3-7:

- Some clothing items are represented over a very small area and others over a much larger area.
- The distribution is not symmetrical about the point (0, 0), or bounded. For example, there are far more points with positive y-axis values than negative, and some points even extend to a y-axis value > 8.
- There are large gaps between colors containing few points.

These observations actually make sampling from the latent space quite challenging. If we overlay the latent space with images of decoded points on a grid, as shown in Figure 3-9, we can begin to understand why the decoder may not always generate images to a satisfactory standard.

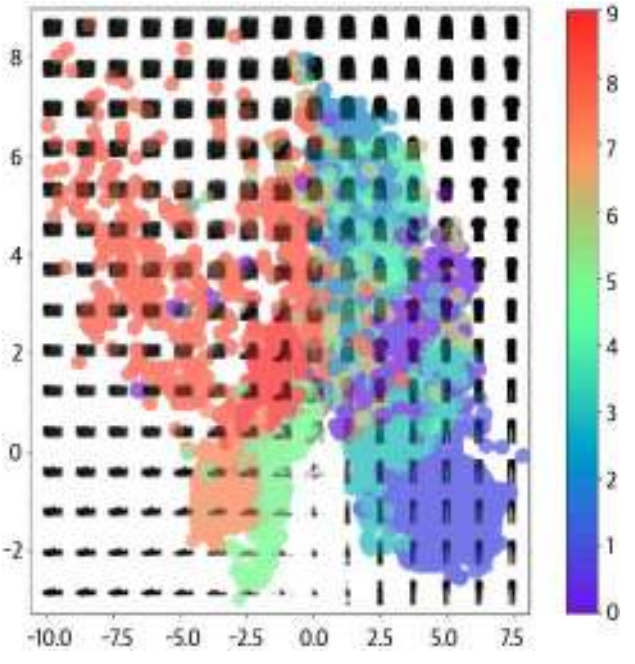


Figure 3-9. A grid of decoded embeddings, overlaid with the embeddings from the original images in the dataset, colored by item type

Firstly, we can see that if we pick points uniformly in a bounded space that we define, we're more likely to sample something that decodes to look like a bag (ID 8) than an ankle boot (ID 9) because the part of the latent space carved out for bags (orange) is larger than the ankle boot area (red).

Secondly, it is not obvious how we should go about choosing a *random* point in the latent space, since the distribution of these points is undefined. Technically, we would be justified in choosing any point in the 2D plane! It's not even guaranteed that points will be centered around (0, 0). This makes sampling from our latent space problematic.

Lastly, we can see holes in the latent space where none of the original images are encoded. For example, there are large white spaces at the edges of the domain—the autoencoder has no reason to ensure that points here are decoded to recognizable clothing items as very few images in the training set are encoded here.

Even points that are central may not be decoded into well-formed images. This is because the autoencoder is not forced to ensure that the space is continuous. For example, even though the point (–1, –1) might be decoded to give a satisfactory

image of a sandal, there is no mechanism in place to ensure that the point (–1.1, –1.1) also produces a satisfactory image of a sandal.

In two dimensions this issue is subtle; the autoencoder only has a small number of dimensions to work with, so naturally it has to squash clothing groups together, resulting in the space between clothing groups being relatively small. However, as we start to use more dimensions in the latent space to generate more complex images such as faces, this problem becomes even more apparent. If we give the autoencoder free rein over how it uses the latent space to encode images, there will be huge gaps between groups of similar points with no incentive for the spaces in between to generate well-formed images.

In order to solve these three problems, we need to convert our autoencoder into a *variational autoencoder*.

Variational Autoencoders

To explain, let's revisit the infinite wardrobe and make a few changes…

Revisiting the Infinite Wardrobe

Suppose now, instead of placing every item of clothing at a single point in the wardrobe, you decide to allocate a general area where the item is more likely to be found. You reason that this more relaxed approach to item location will help to solve the current issue around local discontinuities in the wardrobe.

Also, in order to ensure you do not become too careless with the new placement system, you agree with Brian that you will try to place the center of each item's area as close to the middle of the wardrobe as possible and that deviation of the item from the center should be as close to one meter as possible (not smaller and not larger). The further you stray from this rule, the more you have to pay Brian as your stylist.

After several months of operating with these two simple changes, you step back and admire the new wardrobe layout, alongside some examples of new clothing items that Brian has generated. Much better! There is plenty of diversity in the generated items, and this time there are no examples of poor-quality garments. It seems the two changes have made all the difference!

Let's now try to understand what we need to do to our autoencoder model to convert it into a variational autoencoder and thus make it a more sophisticated generative model.

The two parts that we need to change are the encoder and the loss function.

The Encoder

In an autoencoder, each image is mapped directly to one point in the latent space. In a variational autoencoder, each image is instead mapped to a multivariate normal distribution around a point in the latent space, as shown in Figure 3-10.

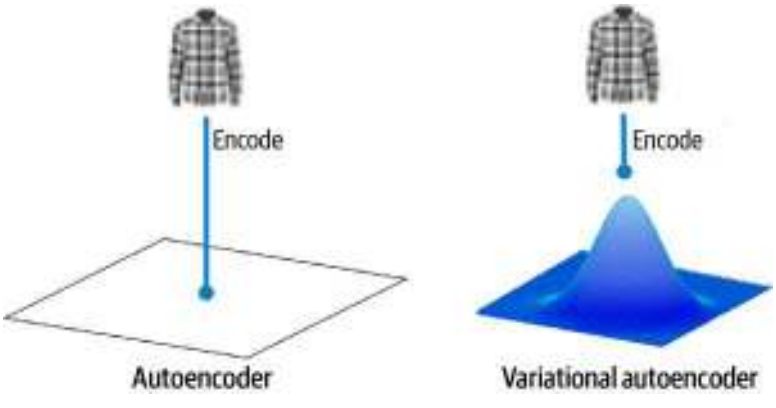


Figure 3-10. The difference between the encoders in an autoencoder and a variational autoencoder

The Multivariate Normal Distribution

A *normal distribution* (or *Gaussian distribution*) $\mathcal{N}(\mu, \sigma)$ is a probability distribution characterized by a distinctive *bell curve* shape, defined by two variables: the *mean* (μ) and the *variance* (σ^2). The *standard deviation* (\(\sigma\)) is the square root of the variance.

The probability density function of the normal distribution in one dimension is:

$$f\left(x \mid \mu, \sigma^2\right) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Figure 3-11 shows several normal distributions in one dimension, for different values of the mean and variance. The red curve is the *standard normal* (or *unit normal*) $\mathcal{N}(0, 1)$—the normal distribution with mean equal to 0 and variance equal to 1.

We can sample a point z from a normal distribution with mean μ and standard deviation σ using the following equation:

$$z = \mu + \sigma\epsilon$$

where ϵ is sampled from a standard normal distribution.

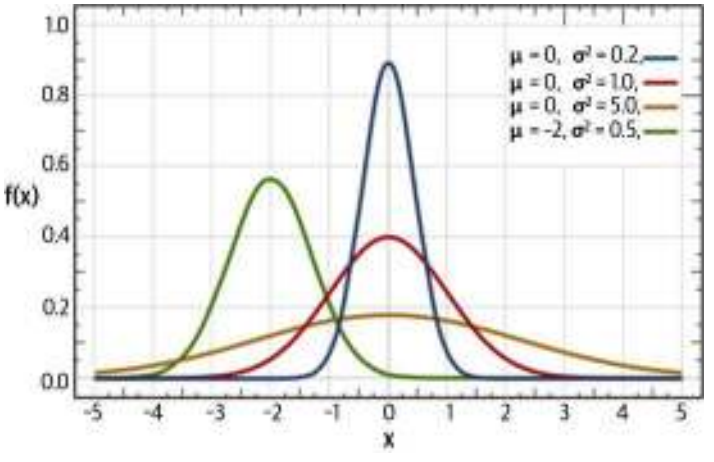


Figure 3-11. The normal distribution in one dimension (source: Wikipedia)

The concept of a normal distribution extends to more than one dimension—the probability density function for a *multivariate normal distribution* (or *multivariate Gaussian distribution*) $\mathcal{N}(\mu, \Sigma)$ in k dimensions with mean vector μ and symmetric covariance matrix Σ is as follows:

$$f(x_1, ..., x_k) = \frac{\exp\left(-\frac{1}{2}(\mathbf{x}-\mu)^{\mathrm{T}}\Sigma^{-1}(\mathbf{x}-\mu)\right)}{\sqrt{(2\pi)^k|\Sigma|}}$$

In this book, we will typically be using *isotropic* multivariate normal distributions, where the covariance matrix is diagonal. This means that the distribution is independent in each dimension (i.e., we can sample a vector where each element is normally distributed with independent mean and variance). This is the case for the multivariate normal distribution that we will use in our variational autoencoder.

A *multivariate standard normal distribution* $\mathcal{N}(0, \mathbf{I})$ is a multivariate distribution with a zero-valued mean vector and identity covariance matrix.



Normal Versus Gaussian

In this book, the terms *normal* and *Gaussian* are used interchangeably and the isotropic and multivariate nature of the distribution is usually implied. For example, "we sample from a Gaussian distribution" can be interpreted to mean "we sample from an isotropic, multivariate Gaussian distribution."

The encoder only needs to map each input to a mean vector and a variance vector and does not need to worry about covariance between dimensions. Variational autoencoders assume that there is no correlation between dimensions in the latent space.

Variance values are always positive, so we actually choose to map to the *logarithm* of the variance, as this can take any real number in the range $(-\infty, \infty)$. This way we can use a neural network as the encoder to perform the mapping from the input image to the mean and log variance vectors.

To summarize, the encoder will take each input image and encode it to two vectors that together define a multivariate normal distribution in the latent space:

`z_mean`
: The mean point of the distribution

`z_log_var`
: The logarithm of the variance of each dimension

We can sample a point `z` from the distribution defined by these values using the following equation:

```
z = z_mean + z_sigma * epsilon
```

where:

```
z_sigma = exp(z_log_var * 0.5)
epsilon ~ N(0,I)
```



The derivation of the relationship between `z_sigma` (σ) and `z_log_var` ($\log\left(\sigma^2\right)$) is as follows:

$$\sigma = \exp\left(\log\left(\sigma\right)\right) = \exp\left(2\log\left(\sigma\right)/2\right) = \exp\left(\log\left(\sigma^2\right)/2\right)$$

The decoder of a variational autoencoder is identical to the decoder of a plain autoencoder, giving the overall architecture shown in Figure 3-12.

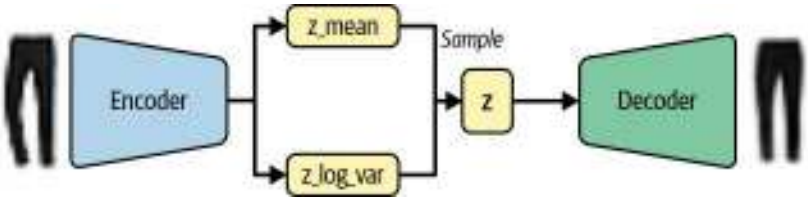


Figure 3-12. VAE architecture diagram

Why does this small change to the encoder help?

Previously, we saw that there was no requirement for the latent space to be continuous—even if the point (–2, 2) decodes to a well-formed image of a sandal, there's no requirement for (–2.1, 2.1) to look similar. Now, since we are sampling a random point from an area around `z_mean`, the decoder must ensure that all points in the same neighborhood produce very similar images when decoded, so that the reconstruction loss remains small. This is a very nice property that ensures that even when we choose a point in the latent space that has never been seen by the decoder, it is likely to decode to an image that is well formed.

Building the VAE encoder

Let's now see how we build this new version of the encoder in Keras.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/03_vae/02_vae_fashion/vae_fashion.ipynb* in the book repository.

The code has been adapted from the excellent VAE tutorial created by Francois Chollet, available on the Keras website.

First, we need to create a new type of `Sampling` layer that will allow us to sample from the distribution defined by `z_mean` and `z_log_var`, as shown in Example 3-11.

Example 3-11. The `Sampling` layer

```
class Sampling(layers.Layer): ❶
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = K.random_normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon ❷
```

❶ We create a new layer by subclassing the Keras base `Layer` class (see the "Subclassing the Layer Class" sidebar).

❷ We use the reparameterization trick (see "The Reparameterization Trick" sidebar) to build a sample from the normal distribution parameterized by `z_mean` and `z_log_var`.

Subclassing the Layer Class

You can create new layers in Keras by subclassing the abstract `Layer` class and defining the `call` method, which describes how a tensor is transformed by the layer.

For example, in the variational autoencoder, we can create a `Sampling` layer that can handle the sampling of `z` from a normal distribution with parameters defined by `z_mean` and `z_log_var`.

This is useful when you want to apply a transformation to a tensor that isn't already included as one of the out-of-the-box Keras layer types.

The Reparameterization Trick

Rather than sample directly from a normal distribution with parameters `z_mean` and `z_log_var`, we can sample `epsilon` from a standard normal and then manually adjust the sample to have the correct mean and variance.

This is known as the *reparameterization trick*, and it's important as it means gradients can backpropagate freely through the layer. By keeping all of the randomness of the layer contained within the variable `epsilon`, the partial derivative of the layer output with respect to its input can be shown to be deterministic (i.e., independent of the random `epsilon`), which is essential for backpropagation through the layer to be possible.

The complete code for the encoder, including the new `Sampling` layer, is shown in Example 3-12.

Example 3-12. The encoder

```
encoder_input = layers.Input(
    shape=(32, 32, 1), name="encoder_input"
)
x = layers.Conv2D(32, (3, 3), strides=2, activation="relu", padding="same")(
    encoder_input
)
x = layers.Conv2D(64, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2D(128, (3, 3), strides=2, activation="relu", padding="same")(x)
shape_before_flattening = K.int_shape(x)[1:]

x = layers.Flatten()(x)
z_mean = layers.Dense(2, name="z_mean")(x) ❶
z_log_var = layers.Dense(2, name="z_log_var")(x)
z = Sampling()([z_mean, z_log_var]) ❷
```

```
encoder = models.Model(encoder_input, [z_mean, z_log_var, z], name="encoder") ❸
```

❶ Instead of connecting the `Flatten` layer directly to the 2D latent space, we connect it to layers `z_mean` and `z_log_var`.

❷ The `Sampling` layer samples a point `z` in the latent space from the normal distribution defined by the parameters `z_mean` and `z_log_var`.

❸ The Keras `Model` that defines the encoder—a model that takes an input image and outputs `z_mean`, `z_log_var`, and a sampled point `z` from the normal distribution defined by these parameters.

A summary of the encoder is shown in Table 3-4.

Table 3-4. Model summary of the VAE encoder

Layer (type)	Output shape	Param #	Connected to
InputLayer (input)	(None, 32, 32, 1)	0	[]
Conv2D (conv2d_1)	(None, 16, 16, 32)	320	[input]
Conv2D (conv2d_2)	(None, 8, 8, 64)	18,496	[conv2d_1]
Conv2D (conv2d_3)	(None, 4, 4, 128)	73,856	[conv2d_2]
Flatten (flatten)	(None, 2048)	0	[conv2d_3]
Dense (z_mean)	(None, 2)	4,098	[flatten]
Dense (z_log_var)	(None, 2)	4,098	[flatten]
Sampling (z)	(None, 2)	0	[z_mean, z_log_var]

Total params	100,868
Trainable params	100,868
Non-trainable params	0

The only other part of the original autoencoder that we need to change is the loss function.

The Loss Function

Previously, our loss function only consisted of the *reconstruction loss* between images and their attempted copies after being passed through the encoder and decoder. The reconstruction loss also appears in a variational autoencoder, but we now require one extra component: the *Kullback–Leibler (KL) divergence* term.

KL divergence is a way of measuring how much one probability distribution differs from another. In a VAE, we want to measure how much our normal distribution with parameters `z_mean` and `z_log_var` differs from a standard normal distribution. In this special case, it can be shown that the KL divergence has the following closed form:

```
kl_loss = -0.5 * sum(1 + z_log_var - z_mean ^ 2 - exp(z_log_var))
```

or in mathematical notation:

$$D_{KL}[N(\mu, \sigma \parallel N(0,1)] = -\frac{1}{2}\sum\left(1 + log\left(\sigma^2\right) - \mu^2 - \sigma^2\right)$$

The sum is taken over all the dimensions in the latent space. `kl_loss` is minimized to 0 when `z_mean = 0` and `z_log_var = 0` for all dimensions. As these two terms start to differ from 0, `kl_loss` increases.

In summary, the KL divergence term penalizes the network for encoding observations to `z_mean` and `z_log_var` variables that differ significantly from the parameters of a standard normal distribution, namely `z_mean = 0` and `z_log_var = 0`.

Why does this addition to the loss function help?

Firstly, we now have a well-defined distribution that we can use for choosing points in the latent space—the standard normal distribution. Secondly, since this term tries to force all encoded distributions toward the standard normal distribution, there is less chance that large gaps will form between point clusters. Instead, the encoder will try to use the space around the origin symmetrically and efficiently.

In the original VAE paper, the loss function for a VAE was simply the addition of the reconstruction loss and the KL divergence loss term. A variant on this (the β-VAE) includes a factor that weights the KL divergence to ensure that it is well balanced with the reconstruction loss. If we weight the reconstruction loss too heavily, the KL loss will not have the desired regulatory effect and we will see the same problems that we experienced with the plain autoencoder. If the KL divergence term is weighted too heavily, the KL divergence loss will dominate and the reconstructed images will be poor. This weighting term is one of the parameters to tune when you're training your VAE.

Training the Variational Autoencoder

Example 3-13 shows how we build the overall VAE model as a subclass of the abstract Keras `Model` class. This allows us to include the calculation of the KL divergence term of the loss function in a custom `train_step` method.

Example 3-13. Training the VAE

```
class VAE(models.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super(VAE, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = metrics.Mean(
            name="reconstruction_loss"
        )
        self.kl_loss_tracker = metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker,
        ]

    def call(self, inputs): ❶
        z_mean, z_log_var, z = encoder(inputs)
        reconstruction = decoder(z)
        return z_mean, z_log_var, reconstruction

    def train_step(self, data): ❷
        with tf.GradientTape() as tape:
            z_mean, z_log_var, reconstruction = self(data)
            reconstruction_loss = tf.reduce_mean(
                500
                * losses.binary_crossentropy(
                    data, reconstruction, axis=(1, 2, 3)
                )
            ) ❸
            kl_loss = tf.reduce_mean(
                tf.reduce_sum(
                    -0.5
                    * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)),
                    axis = 1,
                )
            )
            total_loss = reconstruction_loss + kl_loss ❹

        grads = tape.gradient(total_loss, self.trainable_weights)
```

```
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(reconstruction_loss)
        self.kl_loss_tracker.update_state(kl_loss)

        return {m.name: m.result() for m in self.metrics}

vae = VAE(encoder, decoder)
vae.compile(optimizer="adam")
vae.fit(
    train,
    epochs=5,
    batch_size=100
)
```

❶ This function describes what we would like returned what we call the VAE on a particular input image.

❷ This function describes one training step of the VAE, including the calculation of the loss function.

❸ A beta value of 500 is used in the reconstruction loss.

❹ The total loss is the sum of the reconstruction loss and the KL divergence loss.



Gradient Tape

TensorFlow's *Gradient Tape* is a mechanism that allows the computation of gradients of operations executed during a forward pass of a model. To use it, you need to wrap the code that performs the operations you want to differentiate in a `tf.GradientTape()` context. Once you have recorded the operations, you can compute the gradient of the loss function with respect to some variables by calling `tape.gradient()`. The gradients can then be used to update the variables with the optimizer.

This mechanism is useful for calculating the gradient of custom loss functions (as we have done here) and also for creating custom training loops, as we shall see in Chapter 4.

Analysis of the Variational Autoencoder

Now that we have trained our VAE, we can use the encoder to encode the images in the test set and plot the `z_mean` values in the latent space. We can also sample from a standard normal distribution to generate points in the latent space and use the decoder to decode these points back into pixel space to see how the VAE performs.

Figure 3-13 shows the structure of the new latent space, alongside some sampled points and their decoded images. We can immediately see several changes in how the latent space is organized.

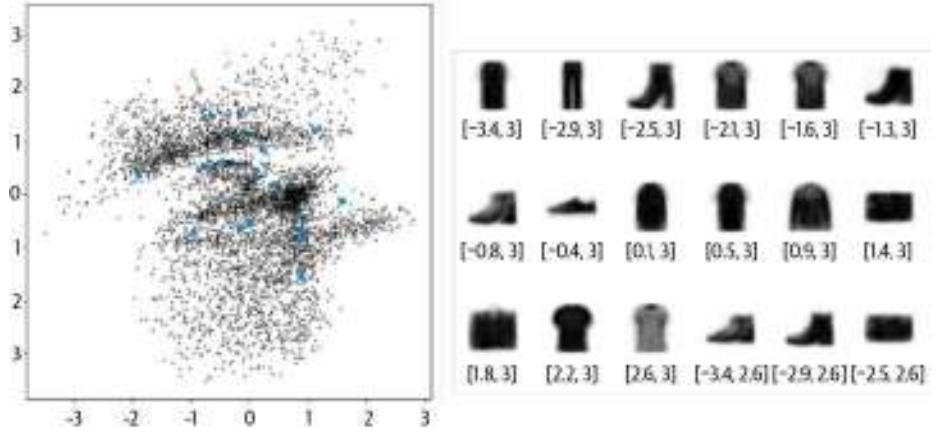


Figure 3-13. The new latent space: the black dots show the `z_mean` value of each encoded image, while blue dots show some sampled points in the latent space (with their decoded images on the right)

Firstly, the KL divergence loss term ensures that the `z_mean` and `z_log_var` values of the encoded images never stray too far from a standard normal distribution. Secondly, there are not so many poorly formed images as the latent space is now much more continuous, due to fact that the encoder is now stochastic, rather than deterministic.

Finally, by coloring points in the latent space by clothing type (Figure 3-14), we can see that there is no preferential treatment of any one type. The righthand plot shows the space transformed into *p*-values—we can see that each color is approximately equally represented. Again, it's important to remember that the labels were not used at all during training; the VAE has learned the various forms of clothing by itself in order to help minimize reconstruction loss.

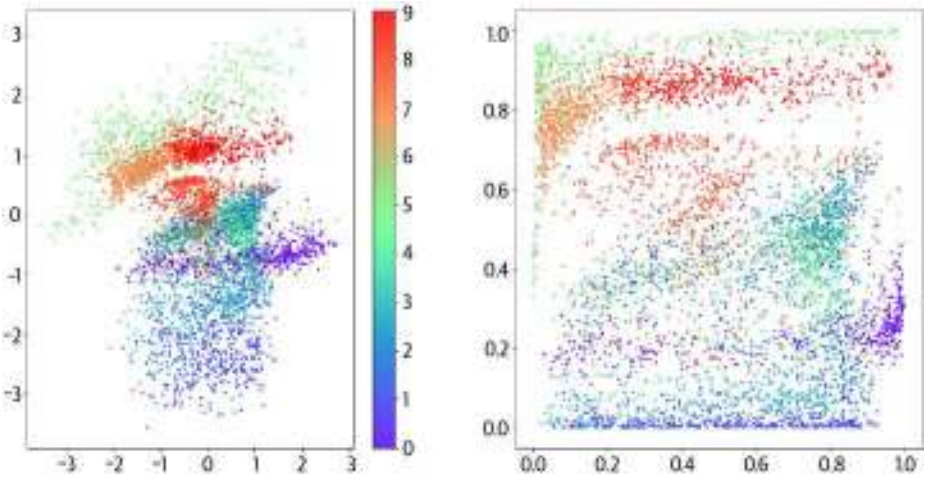


Figure 3-14. The latent space of the VAE colored by clothing type

Exploring the Latent Space

So far, all of our work on autoencoders and variational autoencoders has been limited to a latent space with two dimensions. This has helped us to visualize the inner workings of a VAE on the page and understand why the small tweaks that we made to the architecture of the autoencoder helped transform it into a more powerful class of network that can be used for generative modeling.

Let's now turn our attention to a more complex dataset and see the amazing things that variational autoencoders can achieve when we increase the dimensionality of the latent space.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/03_vae/03_faces/vae_faces.ipynb* in the book repository.

The CelebA Dataset

We shall be using the CelebFaces Attributes (CelebA) dataset to train our next variational autoencoder. This is a collection of over 200,000 color images of celebrity faces, each annotated with various labels (e.g., *wearing hat*, *smiling*, etc.). A few examples are shown in Figure 3-15.



Figure 3-15. Some examples from the CelebA dataset (source: Liu et al., 2015)[3]

Of course, we don't need the labels to train the VAE, but these will be useful later when we start exploring how these features are captured in the multidimensional latent space. Once our VAE is trained, we can sample from the latent space to generate new examples of celebrity faces.

The CelebA dataset is also available through Kaggle, so you can download the dataset by running the Kaggle dataset downloader script in the book repository, as shown in Example 3-14. This will save the images and accompanying metadata locally to the */data* folder.

Example 3-14. Downloading the CelebA dataset

```
bash scripts/download_kaggle_data.sh jessicali9530 celeba-dataset
```

We use the Keras function `image_dataset_from_directory` to create a TensorFlow Dataset pointed at the directory where the images are stored, as shown in Example 3-15. This allows us to read batches of images into memory only when required (e.g., during training), so that we can work with large datasets and not worry about having to fit the entire dataset into memory. It also resizes the images to 64 × 64, interpolating between pixel values.

Example 3-15. Preprocessing the CelebA dataset

```
train_data = utils.image_dataset_from_directory(
    "/app/data/celeba-dataset/img_align_celeba/img_align_celeba",
    labels=None,
    color_mode="rgb",
    image_size=(64, 64),
    batch_size=128,
    shuffle=True,
    seed=42,
    interpolation="bilinear",
)
```

The original data is scaled in the range [0, 255] to denote the pixel intensity, which we rescale to the range [0, 1] as shown in Example 3-16.

Example 3-16. Preprocessing the CelebA dataset

```
def preprocess(img):
    img = tf.cast(img, "float32") / 255.0
    return img

train = train_data.map(lambda x: preprocess(x))
```

Training the Variational Autoencoder

The network architecture for the faces model is similar to the Fashion-MNIST example, with a few slight differences:

- Our data now has three input channels (RGB) instead of one (grayscale). This means we need to change the number of channels in the final convolutional transpose layer of the decoder to 3.
- We shall be using a latent space with 200 dimensions instead of 2. Since faces are much more complex than the Fashion-MNIST images, we increase the dimensionality of the latent space so that the network can encode a satisfactory amount of detail from the images.
- There are batch normalization layers after each convolutional layer to stabilize training. Even though each batch takes a longer time to run, the number of batches required to reach the same loss is greatly reduced.
- We increase the β factor for the KL divergence to 2,000. This is a parameter that requires tuning; for this dataset and architecture this value was found to generate good results.

The full architectures of the encoder and decoder are shown in Tables 3-5 and 3-6, respectively.

Table 3-5. Model summary of the VAE faces encoder

Layer (type)	Output shape	Param #	Connected to
InputLayer (input)	(None, 32, 32, 3)	0	[]
Conv2D (conv2d_1)	(None, 16, 16, 128)	3,584	[input]
BatchNormalization (bn_1)	(None, 16, 16, 128)	512	[conv2d_1]
LeakyReLU (lr_1)	(None, 16, 16, 128)	0	[bn_1]
Conv2D (conv2d_2)	(None, 8, 8, 128)	147,584	[lr_1]
BatchNormalization (bn_2)	(None, 8, 8, 128)	512	[conv2d_2]
LeakyReLU (lr_2)	(None, 8, 8, 128)	0	[bn_2]
Conv2D (conv2d_3)	(None, 4, 4, 128)	147,584	[lr_2]
BatchNormalization (bn_3)	(None, 4, 4, 128)	512	[conv2d_3]
LeakyReLU (lr_3)	(None, 4, 4, 128)	0	[bn_3]
Conv2D (conv2d_4)	(None, 2, 2, 128)	147,584	[lr_3]
BatchNormalization (bn_4)	(None, 2, 2, 128)	512	[conv2d_4]
LeakyReLU (lr_4)	(None, 2, 2, 128)	0	[bn_4]
Flatten (flatten)	(None, 512)	0	[lr_4]
Dense (z_mean)	(None, 200)	102,600	[flatten]
Dense (z_log_var)	(None, 200)	102,600	[flatten]
Sampling (z)	(None, 200)	0	[z_mean, z_log_var]

Total params	653,584
Trainable params	652,560
Non-trainable params	1,024

Table 3-6. Model summary of the VAE faces decoder

Layer (type)	Output shape	Param #
InputLayer	(None, 200)	0
Dense	(None, 512)	102,912
BatchNormalization	(None, 512)	2,048
LeakyReLU	(None, 512)	0
Reshape	(None, 2, 2, 128)	0
Conv2DTranspose	(None, 4, 4, 128)	147,584
BatchNormalization	(None, 4, 4, 128)	512
LeakyReLU	(None, 4, 4, 128)	0
Conv2DTranspose	(None, 8, 8, 128)	147,584

Layer (type)	Output shape	Param #
BatchNormalization	(None, 8, 8, 128)	512
LeakyReLU	(None, 8, 8, 128)	0
Conv2DTranspose	(None, 16, 16, 128)	147,584
BatchNormalization	(None, 16, 16, 128)	512
LeakyReLU	(None, 16, 16, 128)	0
Conv2DTranspose	(None, 32, 32, 128)	147,584
BatchNormalization	(None, 32, 32, 128)	512
LeakyReLU	(None, 32, 32, 128)	0
Conv2DTranspose	(None, 32, 32, 3)	3,459

Total params	700,803
Trainable params	698,755
Non-trainable params	2,048

After around five epochs of training, our VAE should be able to produce novel images of celebrity faces!

Analysis of the Variational Autoencoder

First, let's take a look at a sample of reconstructed faces. The top row in Figure 3-16 shows the original images and the bottom row shows the reconstructions once they have passed through the encoder and decoder.



Figure 3-16. Reconstructed faces, after passing through the encoder and decoder

We can see that the VAE has successfully captured the key features of each face—the angle of the head, the hairstyle, the expression, etc. Some of the fine detail is missing, but it is important to remember that the aim of building variational autoencoders isn't to achieve perfect reconstruction loss. Our end goal is to sample from the latent space in order to generate new faces.

For this to be possible we must check that the distribution of points in the latent space approximately resembles a multivariate standard normal distribution. If we see any dimensions that are significantly different from a standard normal distribution,

we should probably reduce the reconstruction loss factor, since the KL divergence term isn't having enough effect.

The first 50 dimensions in our latent space are shown in Figure 3-17. There aren't any distributions that stand out as being significantly different from the standard normal, so we can move on to generating some faces!

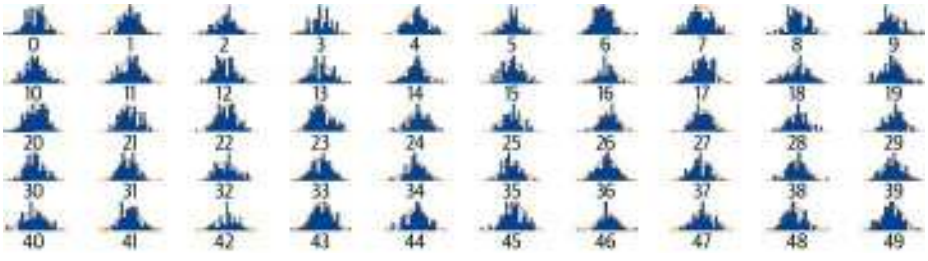


Figure 3-17. Distributions of points for the first 50 dimensions in the latent space

Generating New Faces

To generate new faces, we can use the code in Example 3-17.

Example 3-17. Generating new faces from the latent space

```
grid_width, grid_height = (10,3)
z_sample = np.random.normal(size=(grid_width * grid_height, 200)) ❶

reconstructions = decoder.predict(z_sample) ❷

fig = plt.figure(figsize=(18, 5))
fig.subplots_adjust(hspace=0.4, wspace=0.4)
for i in range(grid_width * grid_height):
    ax = fig.add_subplot(grid_height, grid_width, i + 1)
    ax.axis("off")
    ax.imshow(reconstructions[i, :, :]) ❸
```

❶ Sample 30 points from a standard multivariate normal distribution with 200 dimensions.

❷ Decode the sampled points.

❸ Plot the images!

The output is shown in Figure 3-18.



Figure 3-18. New generated faces

Amazingly, the VAE is able to take the set of points that we sampled from a standard normal distribution and convert each into a convincing image of a person's face. This is our first glimpse of the true power of generative models!

Next, let's see if we can start to use the latent space to perform some interesting operations on generated images.

Latent Space Arithmetic

One benefit of mapping images into a lower-dimensional latent space is that we can perform arithmetic on vectors in this latent space that has a visual analogue when decoded back into the original image domain.

For example, suppose we want to take an image of somebody who looks sad and give them a smile. To do this we first need to find a vector in the latent space that points in the direction of increasing smile. Adding this vector to the encoding of the original image in the latent space will give us a new point which, when decoded, should give us a more smiley version of the original image.

So how can we find the *smile* vector? Each image in the CelebA dataset is labeled with attributes, one of which is `Smiling`. If we take the average position of encoded images in the latent space with the attribute `Smiling` and subtract the average position of encoded images that do not have the attribute `Smiling`, we will obtain the vector that points in the direction of `Smiling`, which is exactly what we need.

Conceptually, we are performing the following vector arithmetic in the latent space, where `alpha` is a factor that determines how much of the feature vector is added or subtracted:

```
z_new = z + alpha * feature_vector
```

Let's see this in action. Figure 3-19 shows several images that have been encoded into the latent space. We then add or subtract multiples of a certain vector (e.g., `Smiling`, `Black_Hair`, `Eyeglasses`, `Young`, `Male`, `Blond_Hair`) to obtain different versions of the image, with only the relevant feature changed.

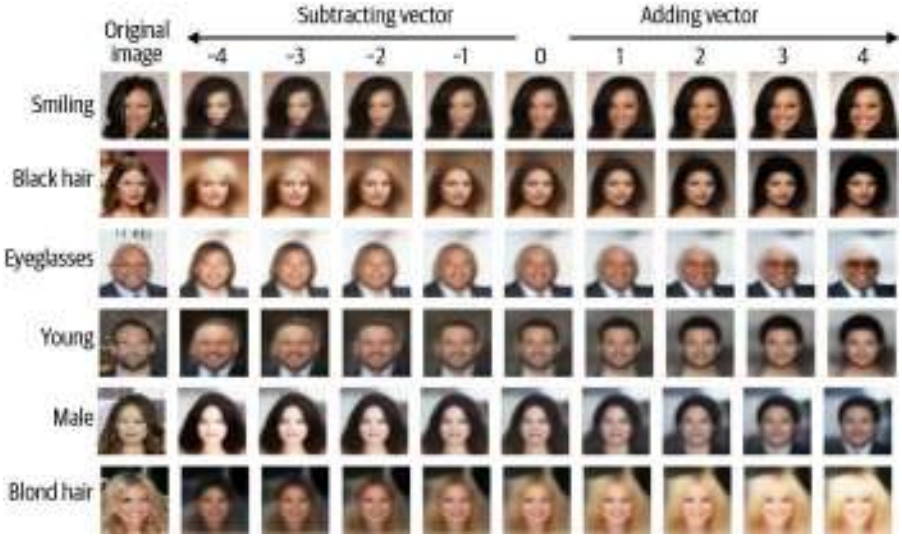


Figure 3-19. Adding and subtracting features to and from faces

It is remarkable that even though we are moving the point a significantly large distance in the latent space, the core image remains approximately the same, except for the one feature that we want to manipulate. This demonstrates the power of variational autoencoders for capturing and adjusting high-level features in images.

Morphing Between Faces

We can use a similar idea to morph between two faces. Imagine two points in the latent space, A and B, that represent two images. If you started at point A and walked toward point B in a straight line, decoding each point on the line as you went, you would see a gradual transition from the starting face to the end face.

Mathematically, we are traversing a straight line, which can be described by the following equation:

```
z_new = z_A * (1- alpha) + z_B * alpha
```

Here, `alpha` is a number between 0 and 1 that determines how far along the line we are, away from point A.

Figure 3-20 shows this process in action. We take two images, encode them into the latent space, and then decode points along the straight line between them at regular intervals.

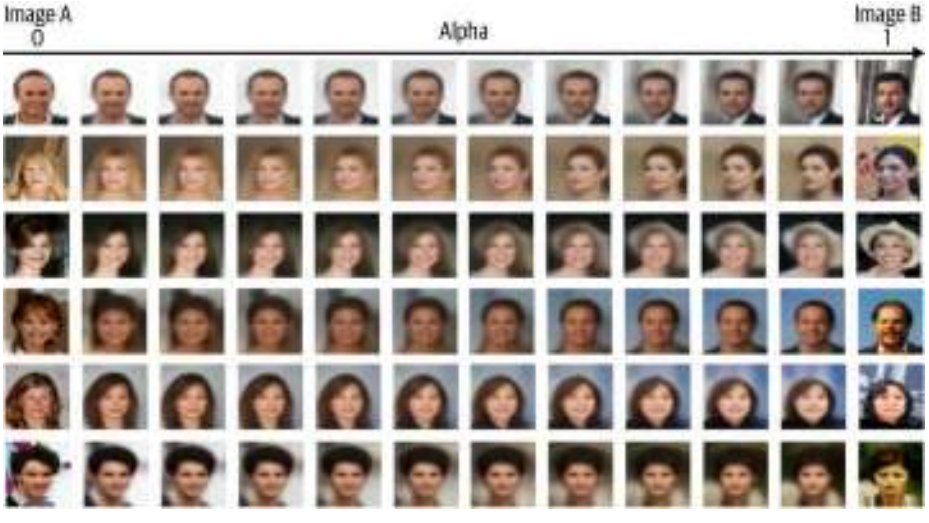


Figure 3-20. Morphing between two faces

It is worth noting the smoothness of the transition—even where there are multiple features to change simultaneously (e.g., removal of glasses, hair color, gender), the VAE manages to achieve this fluidly, showing that the latent space of the VAE is truly a continuous space that can be traversed and explored to generate a multitude of different human faces.

Summary

In this chapter we have seen how variational autoencoders are a powerful tool in the generative modeling toolbox. We started by exploring how plain autoencoders can be used to map high-dimensional images into a low-dimensional latent space, so that high-level features can be extracted from the individually uninformative pixels. However, we quickly found that there were some drawbacks to using plain autoencoders as a generative model—sampling from the learned latent space was problematic, for example.

Variational autoencoders solve these problems by introducing randomness into the model and constraining how points in the latent space are distributed. We saw that with a few minor adjustments, we can transform our autoencoder into a variational autoencoder, thus giving it the power to be a true generative model.

Finally, we applied our new technique to the problem of face generation and saw how we can simply decode points from a standard normal distribution to generate new faces. Moreover, by performing vector arithmetic within the latent space, we can achieve some amazing effects, such as face morphing and feature manipulation.

In the next chapter, we shall explore a different kind of model that remains a popular choice for generative image modeling: the generative adversarial network.

References

1. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," December 20, 2013, *https://arxiv.org/abs/1312.6114*.

2. Vincent Dumoulin and Francesco Visin, "A Guide to Convolution Arithmetic for Deep Learning," January 12, 2018, *https://arxiv.org/abs/1603.07285*.

3. Ziwei Liu et al., "Large-Scale CelebFaces Attributes (CelebA) Dataset," 2015, *http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html*.

CHAPTER 4

Generative Adversarial Networks

Chapter Goals

In this chapter you will:

- Learn about the architectural design of a generative adversarial network (GAN).
- Build and train a deep convolutional GAN (DCGAN) from scratch using Keras.
- Use the DCGAN to generate new images.
- Understand some of the common problems faced when training a DCGAN.
- Learn how the Wasserstein GAN (WGAN) architecture addresses these problems.
- Understand additional enhancements that can be made to the WGAN, such as incorporating a gradient penalty (GP) term into the loss function.
- Build a WGAN-GP from scratch using Keras.
- Use the WGAN-GP to generate faces.
- Learn how a conditional GAN (CGAN) gives you the ability to condition generated output on a given label.
- Build and train a CGAN in Keras and use it to manipulate a generated image.

In 2014, Ian Goodfellow et al. presented a paper entitled "Generative Adversarial Nets"[1] at the Neural Information Processing Systems conference (NeurIPS) in Montreal. The introduction of generative adversarial networks (or GANs, as they are more commonly known) is now regarded as a key turning point in the history of generative modeling, as the core ideas presented in this paper have spawned some of the most successful and impressive generative models ever created.

This chapter will first lay out the theoretical underpinning of GANs, then we will see how to build our own GAN using Keras.

Introduction

Let's start with a short story to illustrate some of the fundamental concepts used in the GAN training process.

Brickki Bricks and the Forgers

It's your first day at your new job as head of quality control for Brickki, a company that specializes in producing high-quality building blocks of all shapes and sizes (Figure 4-1).



Figure 4-1. The production line of a company making bricks of many different shapes and sizes (created with Midjourney)

You are immediately alerted to a problem with some of the items coming off the production line. A competitor has started to make counterfeit copies of Brickki bricks and has found a way to mix them into the bags received by your customers. You decide to become an expert at telling the difference between the counterfeit bricks and the real thing, so that you can intercept the forged bricks on the production line before they are given to customers. Over time, by listening to customer feedback, you gradually become more adept at spotting the fakes.

The forgers are not happy about this—they react to your improved detection abilities by making some changes to their forgery process so that now, the difference between the real bricks and the fakes is even harder for you to spot.

Not one to give up, you retrain yourself to identify the more sophisticated fakes and try to keep one step ahead of the forgers. This process continues, with the forgers iteratively updating their brick creation technologies while you try to become increasingly more accomplished at intercepting their fakes.

With every week that passes, it becomes more and more difficult to tell the difference between the real Brickki bricks and those created by the forgers. It seems that this simple game of cat and mouse is enough to drive significant improvement in both the quality of the forgery and the quality of the detection.

The story of Brickki bricks and the forgers describes the training process of a generative adversarial network.

A GAN is a battle between two adversaries, the *generator* and the *discriminator*. The generator tries to convert random noise into observations that look as if they have been sampled from the original dataset, and the discriminator tries to predict whether an observation comes from the original dataset or is one of the generator's forgeries. Examples of the inputs and outputs to the two networks are shown in Figure 4-2.

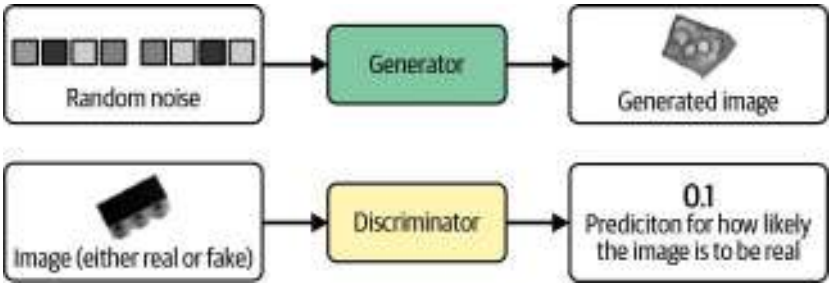


Figure 4-2. Inputs and outputs of the two networks in a GAN

At the start of the process, the generator outputs noisy images and the discriminator predicts randomly. The key to GANs lies in how we alternate the training of the two networks, so that as the generator becomes more adept at fooling the discriminator, the discriminator must adapt in order to maintain its ability to correctly identify which observations are fake. This drives the generator to find new ways to fool the discriminator, and so the cycle continues.

Deep Convolutional GAN (DCGAN)

To see this in action, let's start building our first GAN in Keras, to generate pictures of bricks.

We will be closely following one of the first major papers on GANs, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial

Networks."[2] In this 2015 paper, the authors show how to build a deep convolutional GAN to generate realistic images from a variety of datasets. They also introduce several changes that significantly improve the quality of the generated images.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/04_gan/01_dcgan/dcgan.ipynb* in the book repository.

The Bricks Dataset

First, you'll need to download the training data. We'll be using the Images of LEGO Bricks dataset that is available through Kaggle. This is a computer-rendered collection of 40,000 photographic images of 50 different toy bricks, taken from multiple angles. Some example images of Brickki products are shown in Figure 4-3.

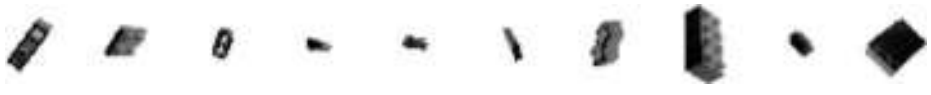


Figure 4-3. Examples of images from the Bricks dataset

You can download the dataset by running the Kaggle dataset downloader script in the book repository, as shown in Example 4-1. This will save the images and accompanying metadata locally to the */data* folder.

Example 4-1. Downloading the Bricks dataset

```
bash scripts/download_kaggle_data.sh joosthazelzet lego-brick-images
```

We use the Keras function `image_dataset_from_directory` to create a TensorFlow Dataset pointed at the directory where the images are stored, as shown in Example 4-2. This allows us to read batches of images into memory only when required (e.g., during training), so that we can work with large datasets and not worry about having to fit the entire dataset into memory. It also resizes the images to 64 × 64, interpolating between pixel values.

Example 4-2. Creating a TensorFlow Dataset from image files in a directory

```
train_data = utils.image_dataset_from_directory(
    "/app/data/lego-brick-images/dataset/",
    labels=None,
    color_mode="grayscale",
    image_size=(64, 64),
    batch_size=128,
```

```
    shuffle=True,
    seed=42,
    interpolation="bilinear",
)
```

The original data is scaled in the range [0, 255] to denote the pixel intensity. When training GANs we rescale the data to the range [–1, 1] so that we can use the tanh activation function on the final layer of the generator, which tends to provide stronger gradients than the sigmoid function (Example 4-3).

Example 4-3. Preprocessing the Bricks dataset

```
def preprocess(img):
    img = (tf.cast(img, "float32") - 127.5) / 127.5
    return img

train = train_data.map(lambda x: preprocess(x))
```

Let's now take a look at how we build the discriminator.

The Discriminator

The goal of the discriminator is to predict if an image is real or fake. This is a supervised image classification problem, so we can use a similar architecture to those we worked with in Chapter 2: stacked convolutional layers, with a single output node.

The full architecture of the discriminator we will be building is shown in Table 4-1.

Table 4-1. Model summary of the discriminator

Layer (type)	Output shape	Param #
InputLayer	(None, 64, 64, 1)	0
Conv2D	(None, 32, 32, 64)	1,024
LeakyReLU	(None, 32, 32, 64)	0
Dropout	(None, 32, 32, 64)	0
Conv2D	(None, 16, 16, 128)	131,072
BatchNormalization	(None, 16, 16, 128)	512
LeakyReLU	(None, 16, 16, 128)	0
Dropout	(None, 16, 16, 128)	0
Conv2D	(None, 8, 8, 256)	524,288
BatchNormalization	(None, 8, 8, 256)	1,024
LeakyReLU	(None, 8, 8, 256)	0
Dropout	(None, 8, 8, 256)	0
Conv2D	(None, 4, 4, 512)	2,097,152
BatchNormalization	(None, 4, 4, 512)	2,048

Layer (type)	Output shape	Param #
LeakyReLU	(None, 4, 4, 512)	0
Dropout	(None, 4, 4, 512)	0
Conv2D	(None, 1, 1, 1)	8,192
Flatten	(None, 1)	0

Total params	2,765,312
Trainable params	2,763,520
Non-trainable params	1,792

The Keras code to build the discriminator is provided in Example 4-4.

Example 4-4. The discriminator

```
discriminator_input = layers.Input(shape=(64, 64, 1)) ❶
x = layers.Conv2D(64, kernel_size=4, strides=2, padding="same", use_bias = False)(
    discriminator_input
) ❷
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    128, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    256, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    512, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    1,
    kernel_size=4,
    strides=1,
    padding="valid",
    use_bias = False,
    activation = 'sigmoid'
)(x)
discriminator_output = layers.Flatten()(x) ❸
```

```
discriminator = models.Model(discriminator_input, discriminator_output) ❹
```

❶ Define the `Input` layer of the discriminator (the image).

❷ Stack `Conv2D` layers on top of each other, with `BatchNormalization`, `LeakyReLU` activation, and `Dropout` layers sandwiched in between.

❸ Flatten the last convolutional layer—by this point, the shape of the tensor is $1 \times 1 \times 1$, so there is no need for a final `Dense` layer.

❹ The Keras model that defines the discriminator—a model that takes an input image and outputs a single number between 0 and 1.

Notice how we use a stride of 2 in some of the `Conv2D` layers to reduce the spatial shape of the tensor as it passes through the network (64 in the original image, then 32, 16, 8, 4, and finally 1), while increasing the number of channels (1 in the grayscale input image, then 64, 128, 256, and finally 512), before collapsing to a single prediction.

We use a sigmoid activation on the final `Conv2D` layer to output a number between 0 and 1.

The Generator

Now let's build the generator. The input to the generator will be a vector drawn from a multivariate standard normal distribution. The output is an image of the same size as an image in the original training data.

This description may remind you of the decoder in a variational autoencoder. In fact, the generator of a GAN fulfills exactly the same purpose as the decoder of a VAE: converting a vector in the latent space to an image. The concept of mapping from a latent space back to the original domain is very common in generative modeling, as it gives us the ability to manipulate vectors in the latent space to change high-level features of images in the original domain.

The architecture of the generator we will be building is shown in Table 4-2.

Table 4-2. Model summary of the generator

Layer (type)	Output shape	Param #
InputLayer	(None, 100)	0
Reshape	(None, 1, 1, 100)	0
Conv2DTranspose	(None, 4, 4, 512)	819,200
BatchNormalization	(None, 4, 4, 512)	2,048

Layer (type)	Output shape	Param #
ReLU	(None, 4, 4, 512)	0
Conv2DTranspose	(None, 8, 8, 256)	2,097,152
BatchNormalization	(None, 8, 8, 256)	1,024
ReLU	(None, 8, 8, 256)	0
Conv2DTranspose	(None, 16, 16, 128)	524,288
BatchNormalization	(None, 16, 16, 128)	512
ReLU	(None, 16, 16, 128)	0
Conv2DTranspose	(None, 32, 32, 64)	131,072
BatchNormalization	(None, 32, 32, 64)	256
ReLU	(None, 32, 32, 64)	0
Conv2DTranspose	(None, 64, 64, 1)	1,024

Total params	3,576,576
Trainable params	3,574,656
Non-trainable params	1,920

The code for building the generator is given in Example 4-5.

Example 4-5. The generator

```
generator_input = layers.Input(shape=(100,)) ❶
x = layers.Reshape((1, 1, 100))(generator_input) ❷
x = layers.Conv2DTranspose(
    512, kernel_size=4, strides=1, padding="valid", use_bias = False
)(x) ❸
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    256, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    128, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    64, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
generator_output = layers.Conv2DTranspose(
    1,
```

```
    kernel_size=4,
    strides=2,
    padding="same",
    use_bias = False,
    activation = 'tanh'
)(x) ❹
generator = models.Model(generator_input, generator_output) ❺
```

❶ Define the `Input` layer of the generator—a vector of length 100.

❷ We use a `Reshape` layer to give a $1 \times 1 \times 100$ tensor, so that we can start applying convolutional transpose operations.

❸ We pass this through four `Conv2DTranspose` layers, with `BatchNormalization` and `LeakyReLU` layers sandwiched in between.

❹ The final `Conv2DTranspose` layer uses a tanh activation function to transform the output to the range [–1, 1], to match the original image domain.

❺ The Keras model that defines the generator—a model that accepts a vector of length 100 and outputs a tensor of shape `[64, 64, 1]`.

Notice how we use a stride of 2 in some of the `Conv2DTranspose` layers to increase the spatial shape of the tensor as it passes through the network (1 in the original vector, then 4, 8, 16, 32, and finally 64), while decreasing the number of channels (512 then 256, 128, 64, and finally 1 to match the grayscale output).

Upsampling Versus Conv2DTranspose

An alternative to using `Conv2DTranspose` layers is to instead use an `UpSampling2D` layer followed by a normal `Conv2D` layer with stride 1, as shown in Example 4-6.

Example 4-6. Upsampling example

```
x = layers.UpSampling2D(size = 2)(x)
x = layers.Conv2D(256, kernel_size=4, strides=1, padding="same")(x)
```

The `UpSampling2D` layer simply repeats each row and column of its input in order to double the size. The `Conv2D` layer with stride 1 then performs the convolution operation. It is a similar idea to convolutional transpose, but instead of filling the gaps between pixels with zeros, upsampling just repeats the existing pixel values.

It has been shown that the `Conv2DTranspose` method can lead to *artifacts*, or small checkerboard patterns in the output image (see Figure 4-4) that spoil the quality of the output. However, they are still used in many of the most impressive GANs in the

literature and have proven to be a powerful tool in the deep learning practitioner's toolbox.



Figure 4-4. Artifacts when using convolutional transpose layers (source: Odena et al., 2016)[3]

Both of these methods—`UpSampling2D` + `Conv2D` and `Conv2DTranspose`—are acceptable ways to transform back to the original image domain. It really is a case of testing both methods in your own problem setting and seeing which produces better results.

Training the DCGAN

As we have seen, the architectures of the generator and discriminator in a DCGAN are very simple and not so different from the VAE models that we looked at in Chapter 3. The key to understanding GANs lies in understanding the training process for the generator and discriminator.

We can train the discriminator by creating a training set where some of the images are *real* observations from the training set and some are *fake* outputs from the generator. We then treat this as a supervised learning problem, where the labels are 1 for the real images and 0 for the fake images, with binary cross-entropy as the loss function.

How should we train the generator? We need to find a way of scoring each generated image so that it can optimize toward high-scoring images. Luckily, we have a discriminator that does exactly that! We can generate a batch of images and pass these through the discriminator to get a score for each image. The loss function for the generator is then simply the binary cross-entropy between these probabilities and a

vector of ones, because we want to train the generator to produce images that the discriminator thinks are real.

Crucially, we must alternate the training of these two networks, making sure that we only update the weights of one network at a time. For example, during the generator training process, only the generator's weights are updated. If we allowed the discriminator's weights to change as well, the discriminator would just adjust so that it is more likely to predict the generated images to be real, which is not the desired outcome. We want generated images to be predicted close to 1 (real) because the generator is strong, not because the discriminator is weak.

A diagram of the training process for the discriminator and generator is shown in Figure 4-5.

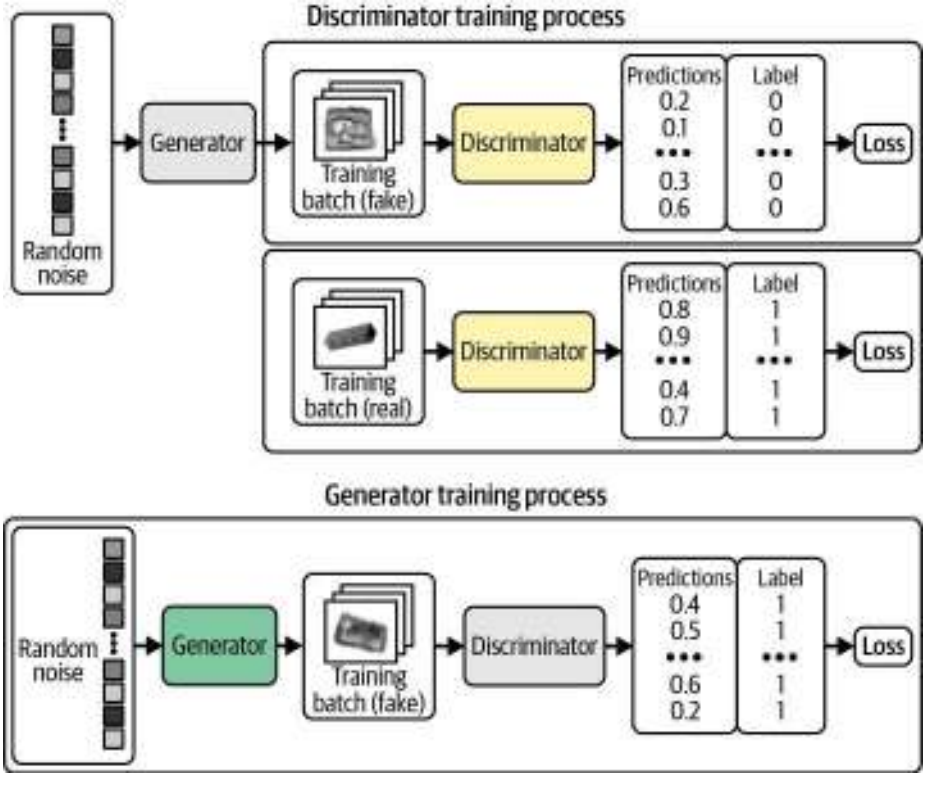


Figure 4-5. Training the DCGAN—gray boxes indicate that the weights are frozen during training

Keras provides us with the ability to create a custom `train_step` function to implement this logic. Example 4-7 shows the full `DCGAN` model class.

Example 4-7. Compiling the DCGAN

```
class DCGAN(models.Model):
    def __init__(self, discriminator, generator, latent_dim):
        super(DCGAN, self).__init__()
        self.discriminator = discriminator
        self.generator = generator
        self.latent_dim = latent_dim

    def compile(self, d_optimizer, g_optimizer):
        super(DCGAN, self).compile()
        self.loss_fn = losses.BinaryCrossentropy() ❶
        self.d_optimizer = d_optimizer
        self.g_optimizer = g_optimizer
        self.d_loss_metric = metrics.Mean(name="d_loss")
        self.g_loss_metric = metrics.Mean(name="g_loss")

    @property
    def metrics(self):
        return [self.d_loss_metric, self.g_loss_metric]

    def train_step(self, real_images):
        batch_size = tf.shape(real_images)[0]
        random_latent_vectors = tf.random.normal(
            shape=(batch_size, self.latent_dim)
        ) ❷

        with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
            generated_images = self.generator(
                random_latent_vectors, training = True
            ) ❸
            real_predictions = self.discriminator(real_images, training = True) ❹
            fake_predictions = self.discriminator(
                generated_images, training = True
            ) ❺

            real_labels = tf.ones_like(real_predictions)
            real_noisy_labels = real_labels + 0.1 * tf.random.uniform(
                tf.shape(real_predictions)
            )
            fake_labels = tf.zeros_like(fake_predictions)
            fake_noisy_labels = fake_labels - 0.1 * tf.random.uniform(
                tf.shape(fake_predictions)
            )

            d_real_loss = self.loss_fn(real_noisy_labels, real_predictions)
            d_fake_loss = self.loss_fn(fake_noisy_labels, fake_predictions)
            d_loss = (d_real_loss + d_fake_loss) / 2.0 ❻

            g_loss = self.loss_fn(real_labels, fake_predictions) ❼

        gradients_of_discriminator = disc_tape.gradient(
```

```
            d_loss, self.discriminator.trainable_variables
        )
        gradients_of_generator = gen_tape.gradient(
            g_loss, self.generator.trainable_variables
        )

        self.d_optimizer.apply_gradients(
            zip(gradients_of_discriminator, discriminator.trainable_variables)
        ) ❽
        self.g_optimizer.apply_gradients(
            zip(gradients_of_generator, generator.trainable_variables)
        )

        self.d_loss_metric.update_state(d_loss)
        self.g_loss_metric.update_state(g_loss)

        return {m.name: m.result() for m in self.metrics}

dcgan = DCGAN(
    discriminator=discriminator, generator=generator, latent_dim=100
)

dcgan.compile(
    d_optimizer=optimizers.Adam(
        learning_rate=0.0002, beta_1 = 0.5, beta_2 = 0.999
    ),
    g_optimizer=optimizers.Adam(
        learning_rate=0.0002, beta_1 = 0.5, beta_2 = 0.999
    ),
)

dcgan.fit(train, epochs=300)
```

❶ The loss function for the generator and discriminator is `BinaryCrossentropy`.

❷ To train the network, first sample a batch of vectors from a multivariate standard normal distribution.

❸ Next, pass these through the generator to produce a batch of generated images.

❹ Now ask the discriminator to predict the realness of the batch of real images…

❺ …and the batch of generated images.

❻ The discriminator loss is the average binary cross-entropy across both the real images (with label 1) and the fake images (with label 0).

❼ The generator loss is the binary cross-entropy between the discriminator predictions for the generated images and a label of 1.

❽ Update the weights of the discriminator and generator separately.

The discriminator and generator are constantly fighting for dominance, which can make the DCGAN training process unstable. Ideally, the training process will find an equilibrium that allows the generator to learn meaningful information from the discriminator and the quality of the images will start to improve. After enough epochs, the discriminator tends to end up dominating, as shown in Figure 4-6, but this may not be a problem as the generator may have already learned to produce sufficiently high-quality images by this point.

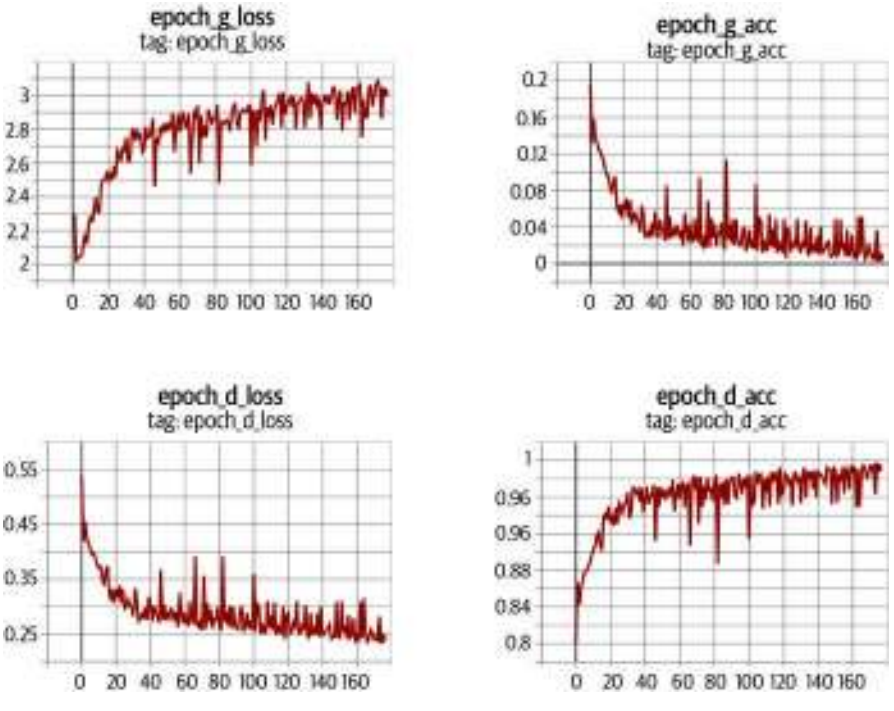


Figure 4-6. Loss and accuracy of the discriminator and generator during training



Adding Noise to the Labels

A useful trick when training GANs is to add a small amount of random noise to the training labels. This helps to improve the stability of the training process and sharpen the generated images. This *label smoothing* acts as way to tame the discriminator, so that it is presented with a more challenging task and doesn't overpower the generator.

Analysis of the DCGAN

By observing images produced by the generator at specific epochs during training (Figure 4-7), it is clear that the generator is becoming increasingly adept at producing images that could have been drawn from the training set.

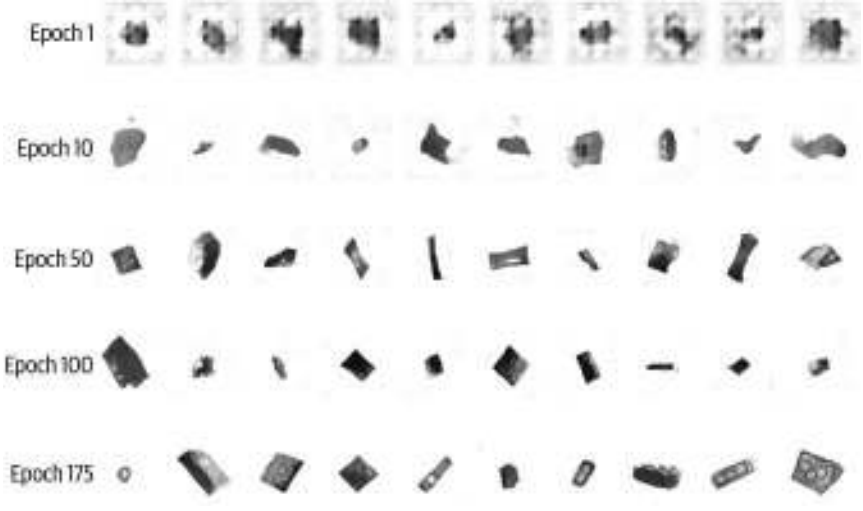


Figure 4-7. Output from the generator at specific epochs during training

It is somewhat miraculous that a neural network is able to convert random noise into something meaningful. It is worth remembering that we haven't provided the model with any additional features beyond the raw pixels, so it has to work out high-level concepts such as how to draw shadows, cuboids, and circles entirely by itself.

Another requirement of a successful generative model is that it doesn't only reproduce images from the training set. To test this, we can find the image from the training set that is closest to a particular generated example. A good measure for distance is the *L1 distance*, defined as:

```
def compare_images(img1, img2):
    return np.mean(np.abs(img1 - img2))
```

Figure 4-8 shows the closest observations in the training set for a selection of generated images. We can see that while there is some degree of similarity between the generated images and the training set, they are not identical. This shows that the generator has understood these high-level features and can generate examples that are distinct from those it has already seen.

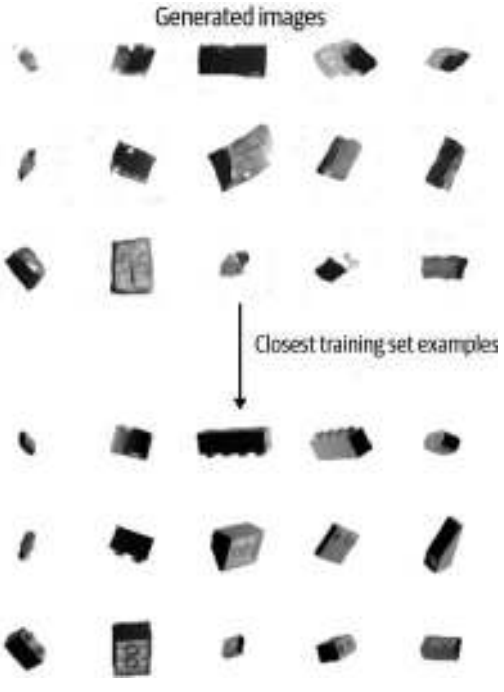


Figure 4-8. Closest matches of generated images from the training set

GAN Training: Tips and Tricks

While GANs are a major breakthrough for generative modeling, they are also notoriously difficult to train. We will explore some of the most common problems and challenges encountered when training GANs in this section, alongside potential solutions. In the next section, we will look at some more fundamental adjustments to the GAN framework that we can make to remedy many of these problems.

Discriminator overpowers the generator

If the discriminator becomes too strong, the signal from the loss function becomes too weak to drive any meaningful improvements in the generator. In the worst-case scenario, the discriminator perfectly learns to separate real images from fake images and the gradients vanish completely, leading to no training whatsoever, as can be seen in Figure 4-9.

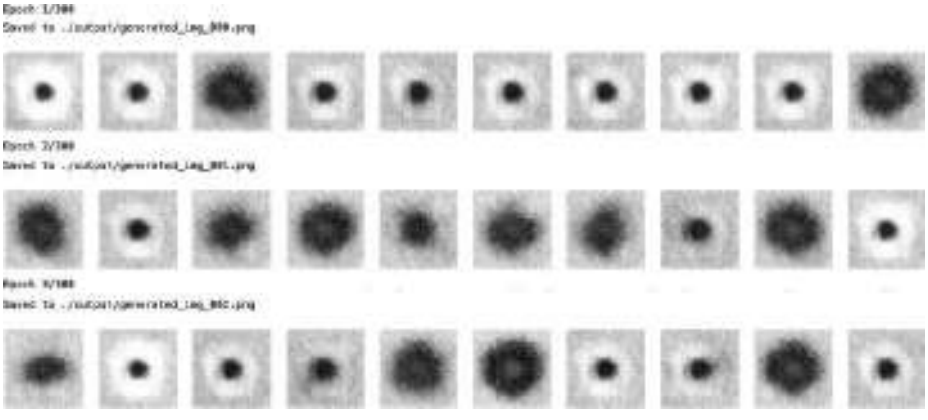


Figure 4-9. Example output when the discriminator overpowers the generator

If you find your discriminator loss function collapsing, you need to find ways to weaken the discriminator. Try the following suggestions:

- Increase the `rate` parameter of the `Dropout` layers in the discriminator to dampen the amount of information that flows through the network.
- Reduce the learning rate of the discriminator.
- Reduce the number of convolutional filters in the discriminator.
- Add noise to the labels when training the discriminator.
- Flip the labels of some images at random when training the discriminator.

Generator overpowers the discriminator

If the discriminator is not powerful enough, the generator will find ways to easily trick the discriminator with a small sample of nearly identical images. This is known as *mode collapse*.

For example, suppose we were to train the generator over several batches without updating the discriminator in between. The generator would be inclined to find a single observation (also known as a *mode*) that always fools the discriminator and would start to map every point in the latent input space to this image. Moreover, the gradients of the loss function would collapse to near 0, so it wouldn't be able to recover from this state.

Even if we then tried to retrain the discriminator to stop it being fooled by this one point, the generator would simply find another mode that fools the discriminator, since it has already become numb to its input and therefore has no incentive to diversify its output.

The effect of mode collapse can be seen in Figure 4-10.

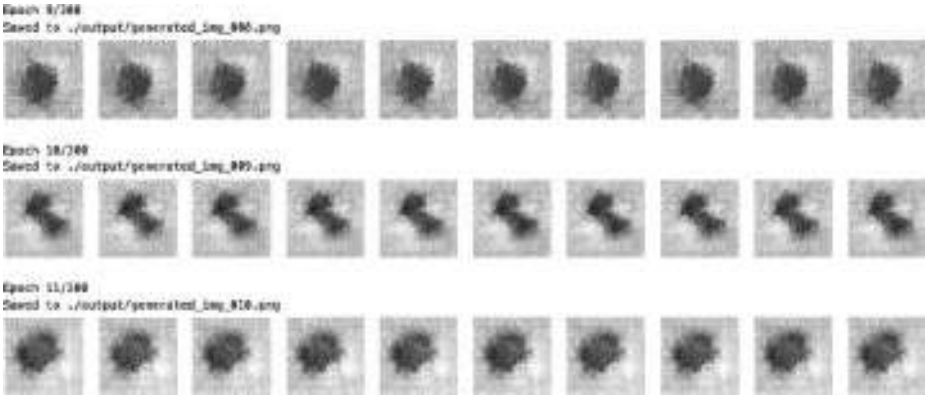


Figure 4-10. Example of mode collapse when the generator overpowers the discriminator

If you find that your generator is suffering from mode collapse, you can try strengthening the discriminator using the opposite suggestions to those listed in the previous section. Also, you can try reducing the learning rate of both networks and increasing the batch size.

Uninformative loss

Since the deep learning model is compiled to minimize the loss function, it would be natural to think that the smaller the loss function of the generator, the better the quality of the images produced. However, since the generator is only graded against the current discriminator and the discriminator is constantly improving, we cannot compare the loss function evaluated at different points in the training process. Indeed, in Figure 4-6, the loss function of the generator actually increases over time, even though the quality of the images is clearly improving. This lack of correlation between the generator loss and image quality sometimes makes GAN training difficult to monitor.

Hyperparameters

As we have seen, even with simple GANs, there are a large number of hyperparameters to tune. As well as the overall architecture of both the discriminator and the generator, there are the parameters that govern batch normalization, dropout, learning rate, activation layers, convolutional filters, kernel size, striding, batch size, and latent space size to consider. GANs are highly sensitive to very slight changes in all of these parameters, and finding a set of parameters that works is often a case of educated trial and error, rather than following an established set of guidelines.

This is why it is important to understand the inner workings of the GAN and know how to interpret the loss function—so that you can identify sensible adjustments to the hyperparameters that might improve the stability of the model.

Tackling GAN challenges

In recent years, several key advancements have drastically improved the overall stability of GAN models and diminished the likelihood of some of the problems listed earlier, such as mode collapse.

In the remainder of this chapter we shall examine the Wasserstein GAN with Gradient Penalty (WGAN-GP), which makes several key adjustments to the GAN framework we have explored thus far to improve the stability and quality of the image generation process.

Wasserstein GAN with Gradient Penalty (WGAN-GP)

In this section we will build a WGAN-GP to generate faces from the CelebA dataset that we utilized in Chapter 3.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/04_gan/02_wgan_gp/wgan_gp.ipynb* in the book repository.

The code has been adapted from the excellent WGAN-GP tutorial created by Aakash Kumar Nain, available on the Keras website.

The Wasserstein GAN (WGAN), introduced in a 2017 paper by Arjovsky et al.,[4] was one of the first big steps toward stabilizing GAN training. With a few changes, the authors were able to show how to train GANs that have the following two properties (quoted from the paper):

- A meaningful loss metric that correlates with the generator's convergence and sample quality
- Improved stability of the optimization process

Specifically, the paper introduces the *Wasserstein loss function* for both the discriminator and the generator. Using this loss function instead of binary cross-entropy results in a more stable convergence of the GAN.

In this section we'll define the Wasserstein loss function and then see what other changes we need to make to the model architecture and training process to incorporate our new loss function.

You can find the full model class in the Jupyter notebook located at *chapter05/wgan-gp/faces/train.ipynb* in the book repository.

Wasserstein Loss

Let's first remind ourselves of the definition of binary cross-entropy loss—the function that we are currently using to train the discriminator and generator of the GAN (Equation 4-1).

Equation 4-1. Binary cross-entropy loss

$$-\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log\left(p_i\right) + \left(1-y_i\right)\log\left(1-p_i\right)\right)$$

To train the GAN discriminator D, we calculate the loss when comparing predictions for real images $p_i = D(x_i)$ to the response $y_i = 1$ and predictions for generated images $p_i = D(G(z_i))$ to the response $y_i = 0$. Therefore, for the GAN discriminator, minimizing the loss function can be written as shown in Equation 4-2.

Equation 4-2. GAN discriminator loss minimization

$$\min_D \; -\left(\mathbb{E}_{x \sim p_X}[\log D(x)] + \mathbb{E}_{z \sim p_Z}[\log\left(1 - D(G(z))\right)]\right)$$

To train the GAN generator G, we calculate the loss when comparing predictions for generated images $p_i = D(G(z_i))$ to the response $y_i = 1$. Therefore, for the GAN generator, minimizing the loss function can be written as shown in Equation 4-3.

Equation 4-3. GAN generator loss minimization

$$\min_G \; -\left(\mathbb{E}_{z \sim p_Z}[\log D(G(z))]\right)$$

Now let's compare this to the Wasserstein loss function.

First, the Wasserstein loss requires that we use $y_i = 1$ and $y_i = -1$ as labels, rather than 1 and 0. We also remove the sigmoid activation from the final layer of the discriminator, so that predictions p_i are no longer constrained to fall in the range [0, 1] but instead can now be any number in the range $(-\infty, \infty)$. For this reason, the discriminator in a WGAN is usually referred to as a *critic* that outputs a *score* rather than a probability.

The Wasserstein loss function is defined as follows:

$$-\frac{1}{n}\sum_{i=1}^{n}\left(y_i p_i\right)$$

To train the WGAN critic D, we calculate the loss when comparing predictions for real images $p_i = D(x_i)$ to the response $y_i = 1$ and predictions for generated images $p_i = D\left(G\left(z_i\right)\right)$ to the response $y_i = -1$. Therefore, for the WGAN critic, minimizing the loss function can be written as follows:

$$\min_D \; -\left(\mathbb{E}_{x \sim p_X}[D(x)] - \mathbb{E}_{z \sim p_Z}[D(G(z))]\right)$$

In other words, the WGAN critic tries to maximize the difference between its predictions for real images and generated images.

To train the WGAN generator, we calculate the loss when comparing predictions for generated images $p_i = D\left(G\left(z_i\right)\right)$ to the response $y_i = 1$. Therefore, for the WGAN generator, minimizing the loss function can be written as follows:

$$\min_G \; -\left(\mathbb{E}_{z \sim p_Z}[D(G(z))]\right)$$

In other words, the WGAN generator tries to produce images that are scored as highly as possible by the critic (i.e., the critic is fooled into thinking they are real).

The Lipschitz Constraint

It may surprise you that we are now allowing the critic to output any number in the range $(-\infty, \infty)$, rather than applying a sigmoid function to restrict the output to the usual [0, 1] range. The Wasserstein loss can therefore be very large, which is unsettling—usually, large numbers in neural networks are to be avoided!

In fact, the authors of the WGAN paper show that for the Wasserstein loss function to work, we also need to place an additional constraint on the critic. Specifically, it is required that the critic is a *1-Lipschitz continuous function*. Let's pick this apart to understand what it means in more detail.

The critic is a function D that converts an image into a prediction. We say that this function is 1-Lipschitz if it satisfies the following inequality for any two input images, x_1 and x_2:

$$\frac{\left|D(x_1) - D(x_2)\right|}{\left|x_1 - x_2\right|} \leq 1$$

Here, $|x_1 - x_2|$ is the average pixelwise absolute difference between two images and $|D(x_1) - D(x_2)|$ is the absolute difference between the critic predictions. Essentially, we require a limit on the rate at which the predictions of the critic can change between two images (i.e., the absolute value of the gradient must be at most 1 everywhere). We can see this applied to a Lipschitz continuous 1D function in Figure 4-11—at no point does the line enter the cone, wherever you place the cone on the line. In other words, there is a limit on the rate at which the line can rise or fall at any point.

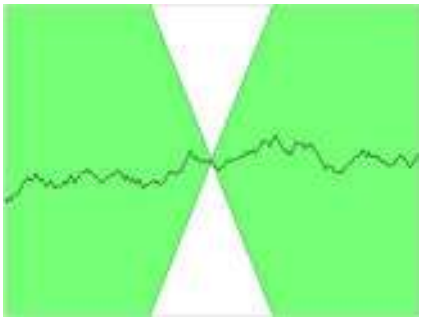


Figure 4-11. A Lipschitz continuous function (source: Wikipedia)



For those who want to delve deeper into the mathematical rationale behind why the Wasserstein loss only works when this constraint is enforced, Jonathan Hui offers an excellent explanation.

Enforcing the Lipschitz Constraint

In the original WGAN paper, the authors show how it is possible to enforce the Lipschitz constraint by clipping the weights of the critic to lie within a small range, [–0.01, 0.01], after each training batch.

One of the criticisms of this approach is that the capacity of the critic to learn is greatly diminished, since we are clipping its weights. In fact, even in the original WGAN paper the authors write, "Weight clipping is a clearly terrible way to enforce a Lipschitz constraint." A strong critic is pivotal to the success of a WGAN, since without accurate gradients, the generator cannot learn how to adapt its weights to produce better samples.

Therefore, other researchers have looked for alternative ways to enforce the Lipschitz constraint and improve the capacity of the WGAN to learn complex features. One such method is the Wasserstein GAN with Gradient Penalty.

In the paper introducing this variant,[5] the authors show how the Lipschitz constraint can be enforced directly by including a *gradient penalty* term in the loss function for

the critic that penalizes the model if the gradient norm deviates from 1. This results in a far more stable training process.

In the next section, we'll see how to build this extra term into the loss function for our critic.

The Gradient Penalty Loss

Figure 4-12 is a diagram of the training process for the critic of a WGAN-GP. If we compare this to the original discriminator training process from Figure 4-5, we can see that the key addition is the gradient penalty loss included as part of the overall loss function, alongside the Wasserstein loss from the real and fake images.

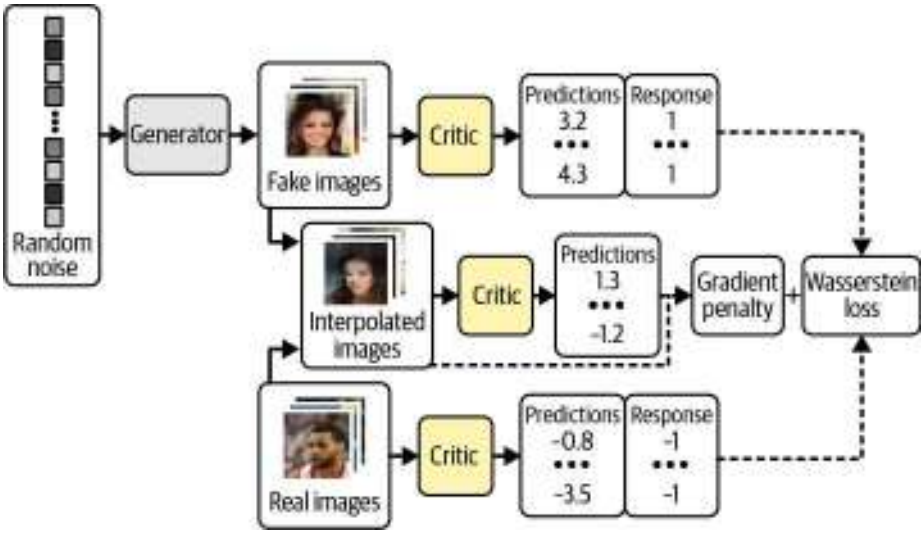


Figure 4-12. The WGAN-GP critic training process

The gradient penalty loss measures the squared difference between the norm of the gradient of the predictions with respect to the input images and 1. The model will naturally be inclined to find weights that ensure the gradient penalty term is minimized, thereby encouraging the model to conform to the Lipschitz constraint.

It is intractable to calculate this gradient everywhere during the training process, so instead the WGAN-GP evaluates the gradient at only a handful of points. To ensure a balanced mix, we use a set of interpolated images that lie at randomly chosen points along lines connecting the batch of real images to the batch of fake images pairwise, as shown in Figure 4-13.

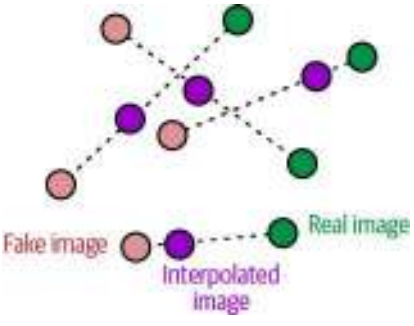


Figure 4-13. Interpolating between images

In Example 4-8, we show how the gradient penalty is calculated in code.

Example 4-8. The gradient penalty loss function

```
def gradient_penalty(self, batch_size, real_images, fake_images):
    alpha = tf.random.normal([batch_size, 1, 1, 1], 0.0, 1.0) ❶
    diff = fake_images - real_images
    interpolated = real_images + alpha * diff ❷

    with tf.GradientTape() as gp_tape:
        gp_tape.watch(interpolated)
        pred = self.critic(interpolated, training=True) ❸

    grads = gp_tape.gradient(pred, [interpolated])[0] ❹
    norm = tf.sqrt(tf.reduce_sum(tf.square(grads), axis=[1, 2, 3])) ❺
    gp = tf.reduce_mean((norm - 1.0) ** 2) ❻
    return gp
```

❶ Each image in the batch gets a random number, between 0 and 1, stored as the vector `alpha`.

❷ A set of interpolated images is calculated.

❸ The critic is asked to score each of these interpolated images.

❹ The gradient of the predictions is calculated with respect to the input images.

❺ The L2 norm of this vector is calculated.

❻ The function returns the average squared distance between the L2 norm and 1.

Training the WGAN-GP

A key benefit of using the Wasserstein loss function is that we no longer need to worry about balancing the training of the critic and the generator—in fact, when using the Wasserstein loss, the critic must be trained to convergence before updating the generator, to ensure that the gradients for the generator update are accurate. This is in contrast to a standard GAN, where it is important not to let the discriminator get too strong.

Therefore, with Wasserstein GANs, we can simply train the critic several times between generator updates, to ensure it is close to convergence. A typical ratio used is three to five critic updates per generator update.

We have now introduced both of the key concepts behind the WGAN-GP—the Wasserstein loss and the gradient penalty term that is included in the critic loss function. The training step of the WGAN model that incorporates all of these ideas is shown in Example 4-9.

Example 4-9. Training the WGAN-GP

```
def train_step(self, real_images):
    batch_size = tf.shape(real_images)[0]

    for i in range(3): ❶
        random_latent_vectors = tf.random.normal(
            shape=(batch_size, self.latent_dim)
        )

        with tf.GradientTape() as tape:
            fake_images = self.generator(
                random_latent_vectors, training = True
            )
            fake_predictions = self.critic(fake_images, training = True)
            real_predictions = self.critic(real_images, training = True)

            c_wass_loss = tf.reduce_mean(fake_predictions) - tf.reduce_mean(
                real_predictions
            ) ❷
            c_gp = self.gradient_penalty(
                batch_size, real_images, fake_images
            ) ❸
            c_loss = c_wass_loss + c_gp * self.gp_weight ❹

        c_gradient = tape.gradient(c_loss, self.critic.trainable_variables)
```

```
            self.c_optimizer.apply_gradients(
                zip(c_gradient, self.critic.trainable_variables)
            ) ❺

        random_latent_vectors = tf.random.normal(
            shape=(batch_size, self.latent_dim)
        )
        with tf.GradientTape() as tape:
            fake_images = self.generator(random_latent_vectors, training=True)
            fake_predictions = self.critic(fake_images, training=True)
            g_loss = -tf.reduce_mean(fake_predictions) ❻

        gen_gradient = tape.gradient(g_loss, self.generator.trainable_variables)
        self.g_optimizer.apply_gradients(
            zip(gen_gradient, self.generator.trainable_variables)
        ) ❼

        self.c_loss_metric.update_state(c_loss)
        self.c_wass_loss_metric.update_state(c_wass_loss)
        self.c_gp_metric.update_state(c_gp)
        self.g_loss_metric.update_state(g_loss)

        return {m.name: m.result() for m in self.metrics}
```

❶ Perform three critic updates.

❷ Calculate the Wasserstein loss for the critic—the difference between the average prediction for the fake images and the real images.

❸ Calculate the gradient penalty term (see Example 4-8).

❹ The critic loss function is a weighted sum of the Wasserstein loss and the gradient penalty.

❺ Update the weights of the critic.

❻ Calculate the Wasserstein loss for the generator.

❼ Update the weights of the generator.



Batch Normalization in a WGAN-GP

One last consideration we should note before training a WGAN-GP is that batch normalization shouldn't be used in the critic. This is because batch normalization creates correlation between images in the same batch, which makes the gradient penalty loss less effective. Experiments have shown that WGAN-GPs can still produce excellent results even without batch normalization in the critic.

We have now covered all of the key differences between a standard GAN and a WGAN-GP. To recap:

- A WGAN-GP uses the Wasserstein loss.
- The WGAN-GP is trained using labels of 1 for real and –1 for fake.
- There is no sigmoid activation in the final layer of the critic.
- Include a gradient penalty term in the loss function for the critic.
- Train the critic multiple times for each update of the generator.
- There are no batch normalization layers in the critic.

Analysis of the WGAN-GP

Let's take a look at some example outputs from the generator, after 25 epochs of training (Figure 4-14).



Figure 4-14. WGAN-GP face examples

The model has learned the significant high-level attributes of a face, and there is no sign of mode collapse.

We can also see how the loss functions of the model evolve over time (Figure 4-15)—the loss functions of both the critic and generator are highly stable and convergent.

If we compare the WGAN-GP output to the VAE output from the previous chapter, we can see that the GAN images are generally sharper—especially the definition between the hair and the background. This is true in general; VAEs tend to produce softer images that blur color boundaries, whereas GANs are known to produce sharper, more well-defined images.

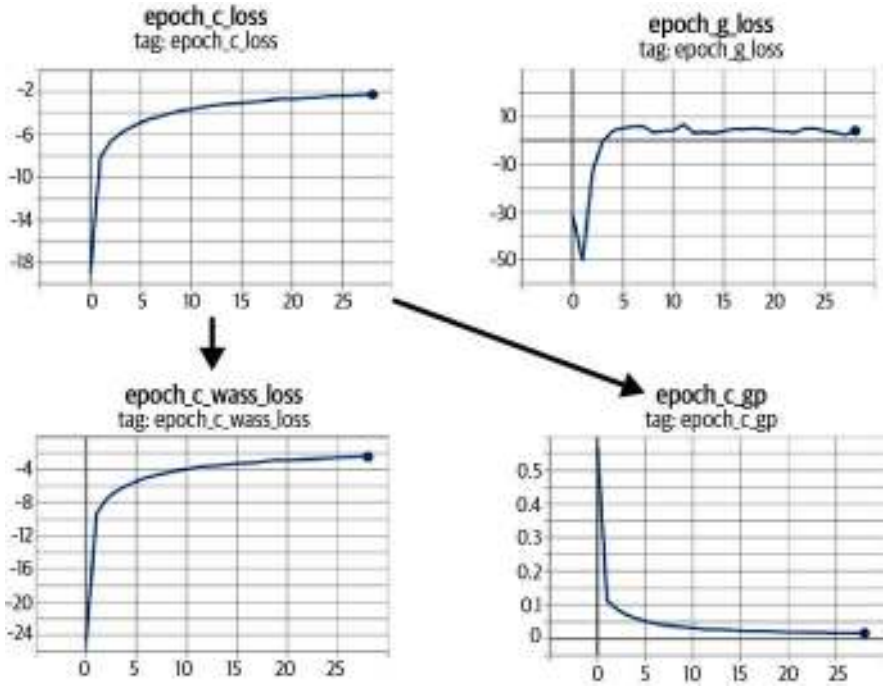


Figure 4-15. WGAN-GP loss curves: the critic loss (`epoch_c_loss`) is broken down into the Wasserstein loss (`epoch_c_wass`) and the gradient penalty loss (`epoch_c_gp`)

It is also true that GANs are generally more difficult to train than VAEs and take longer to reach a satisfactory quality. However, many state-of-the-art generative models today are GAN-based, as the rewards for training large-scale GANs on GPUs over a longer period of time are significant.

Conditional GAN (CGAN)

So far in this chapter, we have built GANs that are able to generate realistic images from a given training set. However, we haven't been able to control the type of image we would like to generate—for example, a male or female face, or a large or small brick. We can sample a random point from the latent space, but we do not have the ability to easily understand what kind of image will be produced given the choice of latent variable.

In the final part of this chapter we shall turn our attention to building a GAN where we are able to control the output—a so called *conditional GAN*. This idea, first introduced in "Conditional Generative Adversarial Nets" by Mirza and Osindero in 2014,[6] is a relatively simple extension to the GAN architecture.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/04_gan/03_cgan/cgan.ipynb* in the book repository.

The code has been adapted from the excellent CGAN tutorial created by Sayak Paul, available on the Keras website.

CGAN Architecture

In this example, we will condition our CGAN on the *blond hair* attribute of the faces dataset. That is, we will be able to explicitly specify whether we want to generate an image with blond hair or not. This label is provided as part of the CelebA dataset.

The high-level CGAN architecture is shown in Figure 4-16.

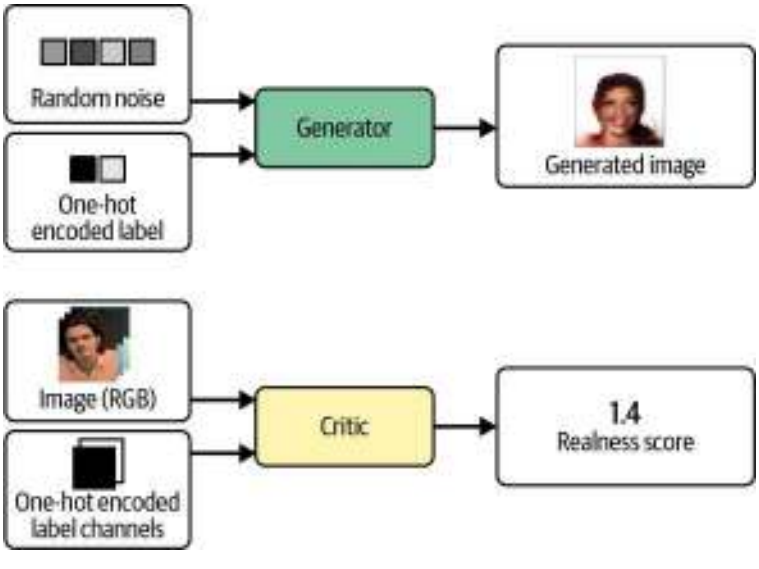


Figure 4-16. Inputs and outputs of the generator and critic in a CGAN

The key difference between a standard GAN and a CGAN is that in a CGAN we pass in extra information to the generator and critic relating to the label. In the generator, this is simply appended to the latent space sample as a one-hot encoded vector. In the critic, we add the label information as extra channels to the RGB image. We do this by repeating the one-hot encoded vector to fill the same shape as the input images.

CGANs work because the critic now has access to extra information regarding the content of the image, so the generator must ensure that its output agrees with the provided label, in order to keep fooling the critic. If the generator produced perfect

images that disagreed with the image label the critic would be able to tell that they were fake simply because the images and labels did not match.



In our example, our one-hot encoded label will have length 2, because there are two classes (Blonde and Not Blond). However, you can have as many labels as you like—for example, you could train a CGAN on the Fashion-MNIST dataset to output one of the 10 different fashion items, by incorporating a one-hot encoded label vector of length 10 into the input of the generator and 10 additional one-hot encoded label channels into the input of the critic.

The only change we need to make to the architecture is to concatenate the label information to the existing inputs of the generator and the critic, as shown in Example 4-10.

Example 4-10. Input layers in the CGAN

```
critic_input = layers.Input(shape=(64, 64, 3)) ❶
label_input = layers.Input(shape=(64, 64, 2))
x = layers.Concatenate(axis = -1)([critic_input, label_input])
...
generator_input = layers.Input(shape=(32,)) ❷
label_input = layers.Input(shape=(2,))
x = layers.Concatenate(axis = -1)([generator_input, label_input])
x = layers.Reshape((1,1, 34))(x)
...
```

❶ The image channels and label channels are passed in separately to the critic and concatenated.

❷ The latent vector and the label classes are passed in separately to the generator and concatenated before being reshaped.

Training the CGAN

We must also make some changes to the `train_step` of the CGAN to match the new input formats of the generator and critic, as shown in Example 4-11.

Example 4-11. The `train_step` of the CGAN

```
def train_step(self, data):
    real_images, one_hot_labels = data ❶

    image_one_hot_labels = one_hot_labels[:, None, None, :] ❷
    image_one_hot_labels = tf.repeat(
```

```
        image_one_hot_labels, repeats=64, axis = 1
    )
    image_one_hot_labels = tf.repeat(
        image_one_hot_labels, repeats=64, axis = 2
    )

    batch_size = tf.shape(real_images)[0]

    for i in range(self.critic_steps):
        random_latent_vectors = tf.random.normal(
            shape=(batch_size, self.latent_dim)
        )

        with tf.GradientTape() as tape:
            fake_images = self.generator(
                [random_latent_vectors, one_hot_labels], training = True
            ) ❸

            fake_predictions = self.critic(
                [fake_images, image_one_hot_labels], training = True
            ) ❹
            real_predictions = self.critic(
                [real_images, image_one_hot_labels], training = True
            )

            c_wass_loss = tf.reduce_mean(fake_predictions) - tf.reduce_mean(
                real_predictions
            )
            c_gp = self.gradient_penalty(
                batch_size, real_images, fake_images, image_one_hot_labels
            ) ❺
            c_loss = c_wass_loss + c_gp * self.gp_weight

        c_gradient = tape.gradient(c_loss, self.critic.trainable_variables)
        self.c_optimizer.apply_gradients(
            zip(c_gradient, self.critic.trainable_variables)
        )

    random_latent_vectors = tf.random.normal(
        shape=(batch_size, self.latent_dim)
    )

    with tf.GradientTape() as tape:
        fake_images = self.generator(
            [random_latent_vectors, one_hot_labels], training=True
        ) ❻
        fake_predictions = self.critic(
            [fake_images, image_one_hot_labels], training=True
        )
        g_loss = -tf.reduce_mean(fake_predictions)

    gen_gradient = tape.gradient(g_loss, self.generator.trainable_variables)
```

```
        self.g_optimizer.apply_gradients(
            zip(gen_gradient, self.generator.trainable_variables)
        )
```

❶ The images and labels are unpacked from the input data.

❷ The one-hot encoded vectors are expanded to one-hot encoded images that have the same spatial size as the input images (64 × 64).

❸ The generator is now fed with a list of two inputs—the random latent vectors and the one-hot encoded label vectors.

❹ The critic is now fed with a list of two inputs—the fake/real images and the one-hot encoded label channels.

❺ The gradient penalty function also requires the one-hot encoded label channels to be passed through as it uses the critic.

❻ The changes made to the critic training step also apply to the generator training step.

Analysis of the CGAN

We can control the CGAN output by passing a particular one-hot encoded label into the input of the generator. For example, to generate a face with nonblond hair, we pass in the vector `[1, 0]`. To generate a face with blond hair, we pass in the vector `[0, 1]`.

The output from the CGAN can be seen in Figure 4-17. Here, we keep the random latent vectors the same across the examples and change only the conditional label vector. It is clear that the CGAN has learned to use the label vector to control only the hair color attribute of the images. It is impressive that the rest of the image barely changes—this is proof that GANs are able to organize points in the latent space in such a way that individual features can be decoupled from each other.

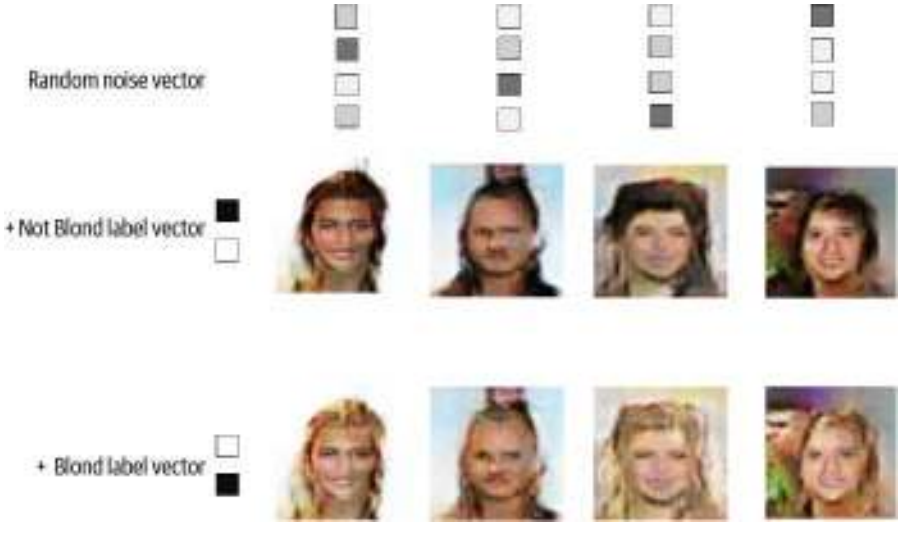


Figure 4-17. Output from the CGAN when the Blond and Not Blond vectors are appended to the latent sample



If labels are available for your dataset, it is generally a good idea to include them as input to your GAN even if you do not necessarily need to condition the generated output on the label, as they tend to improve the quality of images generated. You can think of the labels as just a highly informative extension to the pixel input.

Summary

In this chapter we explored three different generative adversarial network (GAN) models: the deep convolutional GAN (DCGAN), the more sophisticated Wasserstein GAN with Gradient Penalty (WGAN-GP), and the conditional GAN (CGAN).

All GANs are characterized by a generator versus discriminator (or critic) architecture, with the discriminator trying to "spot the difference" between real and fake images and the generator aiming to fool the discriminator. By balancing how these two adversaries are trained, the GAN generator can gradually learn how to produce similar observations to those in the training set.

We first saw how to train a DCGAN to generate images of toy bricks. It was able to learn how to realistically represent 3D objects as images, including accurate representations of shadow, shape, and texture. We also explored the different ways in which GAN training can fail, including mode collapse and vanishing gradients.

We then explored how the Wasserstein loss function remedied many of these problems and made GAN training more predictable and reliable. The WGAN-GP places the 1-Lipschitz requirement at the heart of the training process by including a term in the loss function to pull the gradient norm toward 1.

We applied the WGAN-GP to the problem of face generation and saw how by simply choosing points from a standard normal distribution, we can generate new faces. This sampling process is very similar to a VAE, though the faces produced by a GAN are quite different—often sharper, with greater distinction between different parts of the image.

Finally, we built a CGAN that allowed us to control the type of image that is generated. This works by passing in the label as input to the critic and generator, thereby giving the network the additional information it needs in order to condition the generated output on a given label.

Overall, we have seen how the GAN framework is extremely flexible and able to be adapted to many interesting problem domains. In particular, GANs have driven significant progress in the field of image generation with many interesting extensions to the underlying framework, as we shall see in Chapter 10.

In the next chapter, we will explore a different family of generative model that is ideal for modeling sequential data—autoregressive models.

References

1. Ian J. Goodfellow et al., "Generative Adversarial Nets," June 10, 2014, *https://arxiv.org/abs/1406.2661*

2. Alec Radford et al., "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," January 7, 2016, *https://arxiv.org/abs/1511.06434*.

3. Augustus Odena et al., "Deconvolution and Checkerboard Artifacts," October 17, 2016, *https://distill.pub/2016/deconv-checkerboard*.

4. Martin Arjovsky et al., "Wasserstein GAN," January 26, 2017, *https://arxiv.org/abs/1701.07875*.

5. Ishaan Gulrajani et al., "Improved Training of Wasserstein GANs," March 31, 2017, *https://arxiv.org/abs/1704.00028*.

6. Mehdi Mirza and Simon Osindero, "Conditional Generative Adversarial Nets," November 6, 2014, *https://arxiv.org/abs/1411.1784*.

CHAPTER 5

Autoregressive Models

Chapter Goals

In this chapter you will:

- Learn why autoregressive models are well suited to generating sequential data such as text.
- Learn how to process and tokenize text data.
- Learn about the architectural design of recurrent neural networks (RNNs).
- Build and train a long short-term memory network (LSTM) from scratch using Keras.
- Use the LSTM to generate new text.
- Learn about other variations of RNNs, including gated recurrent units (GRUs) and bidirectional cells.
- Understand how image data can be treated as a sequence of pixels.
- Learn about the architectural design of a PixelCNN.
- Build a PixelCNN from scratch using Keras.
- Use the PixelCNN to generate images.

So far, we have explored two different families of generative models that have both involved latent variables—variational autoencoders (VAEs) and generative adversarial networks (GANs). In both cases, a new variable is introduced with a distribution that is easy to sample from and the model learns how to *decode* this variable back into the original domain.

We will now turn our attention to *autoregressive models*—a family of models that simplify the generative modeling problem by treating it as a sequential process. Autoregressive models condition predictions on previous values in the sequence, rather than on a latent random variable. Therefore, they attempt to explicitly model the data-generating distribution rather than an approximation of it (as in the case of VAEs).

In this chapter we shall explore two different autoregressive models: long short-term memory networks and PixelCNN. We will apply the LSTM to text data and the PixelCNN to image data. We will cover another highly successful autoregressive model, the Transformer, in detail in Chapter 9.

Introduction

To understand how an LSTM works, we will first pay a visit to a strange prison, where the inmates have formed a literary society…

The Literary Society for Troublesome Miscreants

Edward Sopp hated his job as a prison warden. He spent his days watching over the prisoners and had no time to follow his true passion of writing short stories. He was running low on inspiration and needed to find a way to generate new content.

One day, he came up with a brilliant idea that would allow him to produce new works of fiction in his style, while also keeping the inmates occupied—he would get the inmates to collectively write the stories for him! He branded the new society the Literary Society for Troublesome Miscreants, or LSTM (Figure 5-1).



Figure 5-1. A large cell of prisoners reading books (created with Midjourney)

The prison is particularly strange because it only consists of one large cell, containing 256 prisoners. Each prisoner has an opinion on how Edward's current story should continue. Every day, Edward posts the latest word from his novel into the cell, and it is the job of the inmates to individually update their opinions on the current state of the story, based on the new word and the opinions of the inmates from the previous day.

Each prisoner uses a specific thought process to update their own opinion, which involves balancing information from the new incoming word and other prisoners' opinions with their own prior beliefs. First, they decide how much of yesterday's opinion they wish to forget, taking into account the information from the new word and the opinions of other prisoners in the cell. They also use this information to form new thoughts and decide to what extent they want to mix these into the old beliefs that they have chosen to carry forward from the previous day. This then forms the prisoner's new opinion for the day.

However, the prisoners are secretive and don't always tell their fellow inmates all of their opinions. They each also use the latest chosen word and the opinions of the other inmates to decide how much of their opinion they wish to disclose.

When Edward wants the cell to generate the next word in the sequence, the prisoners each tell their disclosable opinions to the guard at the door, who combines this information to ultimately decide on the next word to be appended to the end of the novel. This new word is then fed back into the cell, and the process continues until the full story is completed.

To train the inmates and the guard, Edward feeds short sequences of words that he has written previously into the cell and monitors whether the inmates' chosen next word is correct. He updates them on their accuracy, and gradually they begin to learn how to write stories in his own unique style.

After many iterations of this process, Edward finds that the system has become quite accomplished at generating realistic-looking text. Satisfied with the results, he publishes a collection of the generated tales in his new book, entitled *E. Sopp's Fables*.

The story of Mr. Sopp and his crowdsourced fables is an analogy for one of the most notorious autoregressive techniques for sequential data such as text: the long short-term memory network.

Long Short-Term Memory Network (LSTM)

An LSTM is a particular type of recurrent neural network (RNN). RNNs contain a recurrent layer (or *cell*) that is able to handle sequential data by making its own output at a particular timestep form part of the input to the next timestep.

When RNNs were first introduced, recurrent layers were very simple and consisted solely of a tanh operator that ensured that the information passed between timesteps was scaled between –1 and 1. However, this approach was shown to suffer from the vanishing gradient problem and didn't scale well to long sequences of data.

LSTM cells were first introduced in 1997 in a paper by Sepp Hochreiter and Jürgen Schmidhuber.[1] In the paper, the authors describe how LSTMs do not suffer from the same vanishing gradient problem experienced by vanilla RNNs and can be trained on sequences that are hundreds of timesteps long. Since then, the LSTM architecture has been adapted and improved, and variations such as gated recurrent units (discussed later in this chapter) are now widely utilized and available as layers in Keras.

LSTMs have been applied to a wide range of problems involving sequential data, including time series forecasting, sentiment analysis, and audio classification. In this chapter we will be using LSTMs to tackle the challenge of text generation.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/05_autoregressive/01_lstm/lstm.ipynb* in the book repository.

The Recipes Dataset

We'll be using the Epicurious Recipes dataset that is available through Kaggle. This is a set of over 20,000 recipes, with accompanying metadata such as nutritional information and ingredient lists.

You can download the dataset by running the Kaggle dataset downloader script in the book repository, as shown in Example 5-1. This will save the recipes and accompanying metadata locally to the */data* folder.

Example 5-1. Downloading the Epicurious Recipe dataset

```
bash scripts/download_kaggle_data.sh hugodarwood epirecipes
```

Example 5-2 shows how the data can be loaded and filtered so that only recipes with a title and a description remain. An example of a recipe text string is given in Example 5-3.

Example 5-2. Loading the data

```
with open('/app/data/epirecipes/full_format_recipes.json') as json_data:
    recipe_data = json.load(json_data)

filtered_data = [
```

```
    'Recipe for ' + x['title']+ ' | ' + ' '.join(x['directions'])
    for x in recipe_data
    if 'title' in x
    and x['title'] is not None
    and 'directions' in x
    and x['directions'] is not None
]
```

Example 5-3. A text string from the Recipes dataset

```
Recipe for Ham Persillade with Mustard Potato Salad and Mashed Peas  | Chop enough
parsley leaves to measure 1 tablespoon; reserve. Chop remaining leaves and stems
and simmer with broth and garlic in a small saucepan, covered, 5 minutes.
Meanwhile, sprinkle gelatin over water in a medium bowl and let soften 1 minute.
Strain broth through a fine-mesh sieve into bowl with gelatin and stir to dissolve.
Season with salt and pepper. Set bowl in an ice bath and cool to room temperature,
stirring. Toss ham with reserved parsley and divide among jars. Pour gelatin on top
and chill until set, at least 1 hour. Whisk together mayonnaise, mustard, vinegar,
1/4 teaspoon salt, and 1/4 teaspoon pepper in a large bowl. Stir in celery,
cornichons, and potatoes. Pulse peas with marjoram, oil, 1/2 teaspoon pepper, and
1/4 teaspoon salt in a food processor to a coarse mash. Layer peas, then potato
salad, over ham.
```

Before taking a look at how to build an LSTM network in Keras, we must first take a quick detour to understand the structure of text data and how it is different from the image data that we have seen so far in this book.

Working with Text Data

There are several key differences between text and image data that mean that many of the methods that work well for image data are not so readily applicable to text data. In particular:

- Text data is composed of discrete chunks (either characters or words), whereas pixels in an image are points in a continuous color spectrum. We can easily make a green pixel more blue, but it is not obvious how we should go about making the word *cat* more like the word *dog*, for example. This means we can easily apply backpropagation to image data, as we can calculate the gradient of our loss function with respect to individual pixels to establish the direction in which pixel colors should be changed to minimize the loss. With discrete text data, we can't obviously apply backpropagation in the same way, so we need to find a way around this problem.
- Text data has a time dimension but no spatial dimension, whereas image data has two spatial dimensions but no time dimension. The order of words is highly important in text data and words wouldn't make sense in reverse, whereas images can usually be flipped without affecting the content. Furthermore, there are often

long-term sequential dependencies between words that need to be captured by the model: for example, the answer to a question or carrying forward the context of a pronoun. With image data, all pixels can be processed simultaneously.

- Text data is highly sensitive to small changes in the individual units (words or characters). Image data is generally less sensitive to changes in individual pixel units—a picture of a house would still be recognizable as a house even if some pixels were altered—but with text data, changing even a few words can drastically alter the meaning of the passage, or make it nonsensical. This makes it very difficult to train a model to generate coherent text, as every word is vital to the overall meaning of the passage.
- Text data has a rules-based grammatical structure, whereas image data doesn't follow set rules about how the pixel values should be assigned. For example, it wouldn't make grammatical sense in any context to write "The cat sat on the having." There are also semantic rules that are extremely difficult to model; it wouldn't make sense to say "I am in the beach," even though grammatically, there is nothing wrong with this statement.



Advances in Text-Based Generative Deep Learning

Until recently, most of the most sophisticated generative deep learning models have focused on image data, because many of the challenges presented in the preceding list were beyond the reach of even the most advanced techniques. However, in the last five years astonishing progress has been made in the field of text-based generative deep learning, thanks to the introduction of the Transformer model architecture, which we will explore in Chapter 9.

With these points in mind, let's now take a look at the steps we need to take in order to get the text data into the right shape to train an LSTM network.

Tokenization

The first step is to clean up and tokenize the text. *Tokenization* is the process of splitting the text up into individual units, such as words or characters.

How you tokenize your text will depend on what you are trying to achieve with your text generation model. There are pros and cons to using both word and character tokens, and your choice will affect how you need to clean the text prior to modeling and the output from your model.

If you use word tokens:

- All text can be converted to lowercase, to ensure capitalized words at the start of sentences are tokenized the same way as the same words appearing in the middle of a sentence. In some cases, however, this may not be desirable; for example, some proper nouns, such as names or places, may benefit from remaining capitalized so that they are tokenized independently.
- The text *vocabulary* (the set of distinct words in the training set) may be very large, with some words appearing very sparsely or perhaps only once. It may be wise to replace sparse words with a token for *unknown word*, rather than including them as separate tokens, to reduce the number of weights the neural network needs to learn.
- Words can be *stemmed*, meaning that they are reduced to their simplest form, so that different tenses of a verb remained tokenized together. For example, *browse*, *browsing*, *browses*, and *browsed* would all be stemmed to *brows*.
- You will need to either tokenize the punctuation, or remove it altogether.
- Using word tokenization means that the model will never be able to predict words outside of the training vocabulary.

If you use character tokens:

- The model may generate sequences of characters that form new words outside of the training vocabulary—this may be desirable in some contexts, but not in others.
- Capital letters can either be converted to their lowercase counterparts, or remain as separate tokens.
- The vocabulary is usually much smaller when using character tokenization. This is beneficial for model training speed as there are fewer weights to learn in the final output layer.

For this example, we'll use lowercase word tokenization, without word stemming. We'll also tokenize punctuation marks, as we would like the model to predict when it should end sentences or use commas, for example.

The code in Example 5-4 cleans and tokenizes the text.

Example 5-4. Tokenization

```
def pad_punctuation(s):
    s = re.sub(f"([{string.punctuation}])", r' \1 ', s)
    s = re.sub(' +', ' ', s)
    return s
```

```
text_data = [pad_punctuation(x) for x in filtered_data] ❶

text_ds = tf.data.Dataset.from_tensor_slices(text_data).batch(32).shuffle(1000) ❷

vectorize_layer = layers.TextVectorization( ❸
    standardize = 'lower',
    max_tokens = 10000,
    output_mode = "int",
    output_sequence_length = 200 + 1,
)

vectorize_layer.adapt(text_ds) ❹
vocab = vectorize_layer.get_vocabulary() ❺
```

❶ Pad the punctuation marks, to treat them as separate words.

❷ Convert to a TensorFlow Dataset.

❸ Create a Keras `TextVectorization` layer to convert text to lowercase, give the most prevalent 10,000 words a corresponding integer token, and trim or pad the sequence to 201 tokens long.

❹ Apply the `TextVectorization` layer to the training data.

❺ The `vocab` variable stores a list of the word tokens.

An example of a recipe after tokenization is shown in Example 5-5. The sequence length that we use to train the model is a parameter of the training process. In this example we choose to use a sequence length of 200, so we pad or clip the recipe to one more than this length, to allow us to create the target variable (more on this in the next section). To achieve this desired length, the end of the vector is padded with zeros.



Stop Tokens

The `0` token is known as a the *stop token*, signifying that the text string has come to an end.

Example 5-5. The recipe from Example 5-3 tokenized

```
[  26   16  557    1    8  298  335  189    4 1054  494   27  332  228
  235  262    5  594   11  133   22  311    2  332   45  262    4  671
    4   70    8  171    4   81    6    9   65   80    3  121    3   59
   12    2  299    3   88  650   20   39    6    9   29   21    4   67
  529   11  164    2  320  171  102    9  374   13  643  306   25   21
    8  650    4   42    5  931    2   63    8   24    4   33    2  114
   21    6  178  181 1245    4   60    5  140  112    3   48    2  117
```

```
  557    8  285  235    4  200  292  980    2  107  650   28   72    4
  108   10  114    3   57  204   11  172    2   73  110  482    3  298
    3  190    3   11   23   32  142   24    3    4   11   23   32  142
   33    6    9   30   21    2   42    6  353    3 3224    3    4  150
    2  437  494    8 1281    3   37    3   11   23   15  142   33    3
    4   11   23   32  142   24    6    9  291  188    5    9  412  572
    2  230  494    3   46  335  189    3   20  557    2    0    0    0
    0    0    0    0    0]
```

In Example 5-6, we can see a subset of the list of tokens mapped to their respective indices. The layer reserves the `0` token for padding (i.e., it is the stop token) and the `1` token for unknown words that fall outside the top 10,000 words (e.g., persillade). The other words are assigned tokens in order of frequency. The number of words to include in the vocabulary is also a parameter of the training process. The more words included, the fewer *unknown* tokens you will see in the text; however, your model will need to be larger to accommodate the larger vocabulary size.

Example 5-6. The vocabulary of the `TextVectorization` layer

```
0:
1: [UNK]
2: .
3: ,
4: and
5: to
6: in
7: the
8: with
9: a
```

Creating the Training Set

Our LSTM will be trained to predict the next word in a sequence, given a sequence of words preceding this point. For example, we could feed the model the tokens for *grilled chicken with boiled* and would expect the model to output a suitable next word (e.g., *potatoes*, rather than *bananas*).

We can therefore simply shift the entire sequence by one token in order to create our target variable.

The dataset generation step can be achieved with the code in Example 5-7.

Example 5-7. Creating the training dataset

```
def prepare_inputs(text):
    text = tf.expand_dims(text, -1)
    tokenized_sentences = vectorize_layer(text)
    x = tokenized_sentences[:, :-1]
```

```
    y = tokenized_sentences[:, 1:]
    return x, y

train_ds = text_ds.map(prepare_inputs) ❶
```

❶ Create the training set consisting of recipe tokens (the input) and the same vector shifted by one token (the target).

The LSTM Architecture

The architecture of the overall LSTM model is shown in Table 5-1. The input to the model is a sequence of integer tokens and the output is the probability of each word in the 10,000-word vocabulary appearing next in the sequence. To understand how this works in detail, we need to introduce two new layer types, `Embedding` and `LSTM`.

Table 5-1. Model summary of the LSTM

Layer (type)	Output shape	Param #
InputLayer	(None, None)	0
Embedding	(None, None, 100)	1,000,000
LSTM	(None, None, 128)	117,248
Dense	(None, None, 10000)	1,290,000

Total params	2,407,248
Trainable params	2,407,248
Non-trainable params	0



The Input Layer of the LSTM

Notice that the `Input` layer does not need us to specify the sequence length in advance. Both the batch size and the sequence length are flexible (hence the `(None, None)` shape). This is because all downstream layers are agnostic to the length of the sequence being passed through.

The Embedding Layer

An *embedding layer* is essentially a lookup table that converts each integer token into a vector of length `embedding_size`, as shown in Figure 5-2. The lookup vectors are learned by the model as *weights*. Therefore, the number of weights learned by this layer is equal to the size of the vocabulary multiplied by the dimension of the embedding vector (i.e., 10,000 × 100 = 1,000,000).

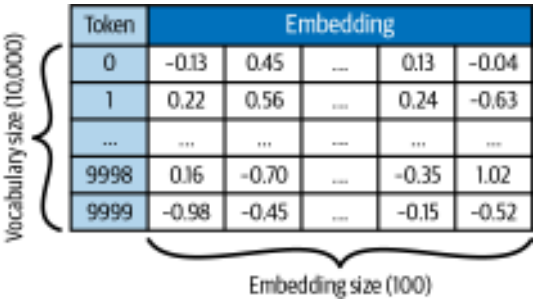


Figure 5-2. An embedding layer is a lookup table for each integer token

We embed each integer token into a continuous vector because it enables the model to learn a representation for each word that is able to be updated through backpropagation. We could also just one-hot encode each input token, but using an embedding layer is preferred because it makes the embedding itself trainable, thus giving the model more flexibility in deciding how to embed each token to improve its performance.

Therefore, the `Input` layer passes a tensor of integer sequences of shape `[batch_size, seq_length]` to the `Embedding` layer, which outputs a tensor of shape `[batch_size, seq_length, embedding_size]`. This is then passed on to the `LSTM` layer (Figure 5-3).

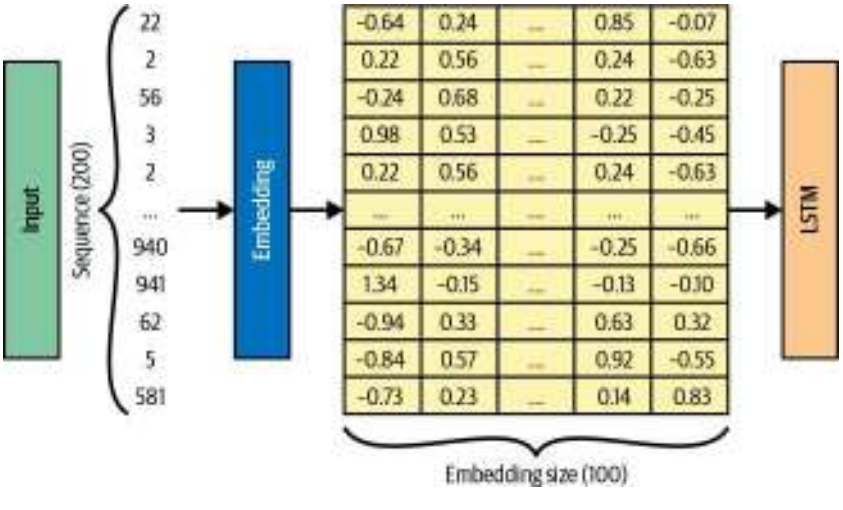


Figure 5-3. A single sequence as it flows through an embedding layer

The LSTM Layer

To understand the LSTM layer, we must first look at how a general recurrent layer works.

A recurrent layer has the special property of being able to process sequential input data $x_1, \cdots, x_n$. It consists of a cell that updates its *hidden state*, h_t, as each element of the sequence x_t is passed through it, one timestep at a time.

The hidden state is a vector with length equal to the number of *units* in the cell—it can be thought of as the cell's current understanding of the sequence. At timestep t, the cell uses the previous value of the hidden state, h_{t-1}, together with the data from the current timestep x_t to produce an updated hidden state vector, h_t. This recurrent process continues until the end of the sequence. Once the sequence is finished, the layer outputs the final hidden state of the cell, h_n, which is then passed on to the next layer of the network. This process is shown in Figure 5-4.

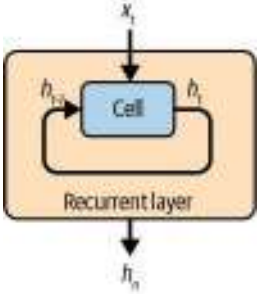


Figure 5-4. A simple diagram of a recurrent layer

To explain this in more detail, let's unroll the process so that we can see exactly how a single sequence is fed through the layer (Figure 5-5).



Cell Weights

It's important to remember that all of the cells in this diagram share the same weights (as they are really the same cell). There is no difference between this diagram and Figure 5-4; it's just a different way of drawing the mechanics of a recurrent layer.

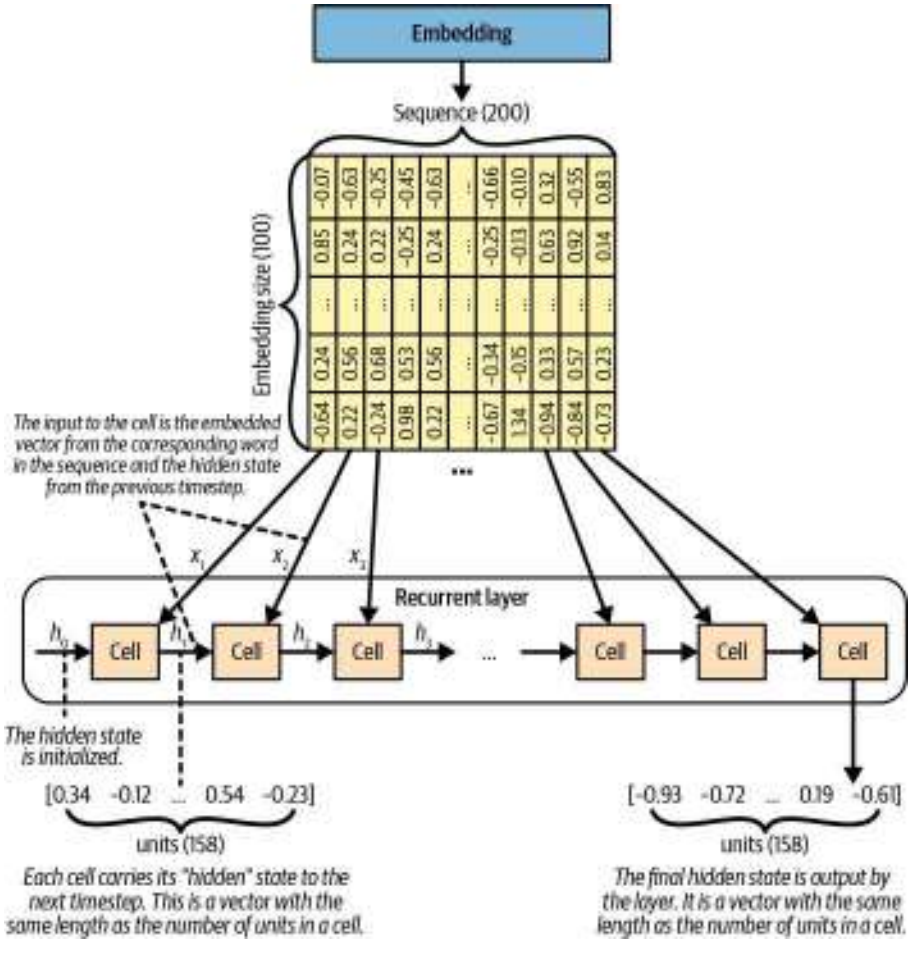


Figure 5-5. How a single sequence flows through a recurrent layer

Here, we represent the recurrent process by drawing a copy of the cell at each timestep and show how the hidden state is constantly being updated as it flows through the cells. We can clearly see how the previous hidden state is blended with the current sequential data point (i.e., the current embedded word vector) to produce the next hidden state. The output from the layer is the final hidden state of the cell, after each word in the input sequence has been processed.



The fact that the output from the cell is called a *hidden* state is an unfortunate naming convention—it's not really hidden, and you shouldn't think of it as such. Indeed, the last hidden state is the overall output from the layer, and we will be making use of the fact that we can access the hidden state at each individual timestep later in this chapter.

The LSTM Cell

Now that we have seen how a generic recurrent layer works, let's take a look inside an individual LSTM cell.

The job of the LSTM cell is to output a new hidden state, h_t, given its previous hidden state, h_{t-1}, and the current word embedding, x_t. To recap, the length of h_t is equal to the number of units in the LSTM. This is a parameter that is set when you define the layer and has nothing to do with the length of the sequence.



Make sure you do not confuse the term *cell* with *unit*. There is one cell in an LSTM layer that is defined by the number of units it contains, in the same way that the prisoner cell from our earlier story contained many prisoners. We often draw a recurrent layer as a chain of cells unrolled, as it helps to visualize how the hidden state is updated at each timestep.

An LSTM cell maintains a cell state, C_t, which can be thought of as the cell's internal beliefs about the current status of the sequence. This is distinct from the hidden state, h_t, which is ultimately output by the cell after the final timestep. The cell state is the same length as the hidden state (the number of units in the cell).

Let's look more closely at a single cell and how the hidden state is updated (Figure 5-6).

The hidden state is updated in six steps:

1. The hidden state of the previous timestep, h_{t-1}, and the current word embedding, x_t, are concatenated and passed through the *forget* gate. This gate is simply a dense layer with weights matrix W_f, bias b_f, and a sigmoid activation function. The resulting vector, f_t, has length equal to the number of units in the cell and contains values between 0 and 1 that determine how much of the previous cell state, C_{t-1}, should be retained.

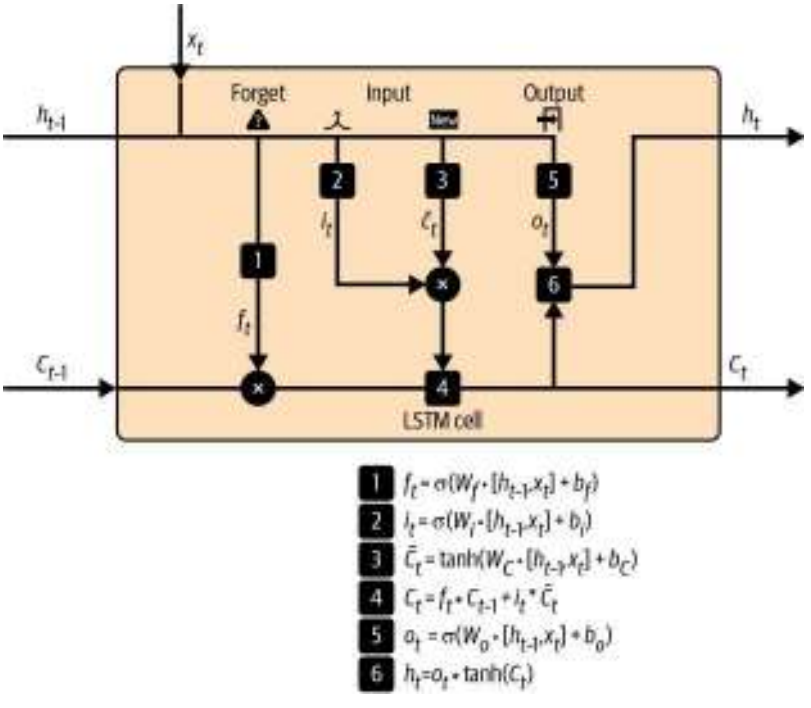


Figure 5-6. An LSTM cell

2. The concatenated vector is also passed through an *input* gate that, like the forget gate, is a dense layer with weights matrix W_i, bias b_i, and a sigmoid activation function. The output from this gate, i_t, has length equal to the number of units in the cell and contains values between 0 and 1 that determine how much new information will be added to the previous cell state, C_{t-1}.
3. The concatenated vector is passed through a dense layer with weights matrix W_C, bias b_C, and a tanh activation function to generate a vector $\tilde{C}_t$ that contains the new information that the cell wants to consider keeping. It also has length equal to the number of units in the cell and contains values between –1 and 1.
4. f_t and C_{t-1} are multiplied element-wise and added to the element-wise multiplication of i_t and $\tilde{C}_t$. This represents forgetting parts of the previous cell state and then adding new relevant information to produce the updated cell state, C_t.
5. The concatenated vector is passed through an *output* gate: a dense layer with weights matrix W_o, bias b_o, and a sigmoid activation. The resulting vector, o_t, has length equal to the number of units in the cell and stores values between 0 and 1 that determine how much of the updated cell state, C_t, to output from the cell.

6. o_t is multiplied element-wise with the updated cell state, C_t, after a tanh activation has been applied to produce the new hidden state, h_t.



The Keras LSTM Layer

All of this complexity is wrapped up within the `LSTM` layer type in Keras, so you don't have to worry about implementing it yourself!

Training the LSTM

The code to build, compile, and train the LSTM is given in Example 5-8.

Example 5-8. Building, compiling, and training the LSTM

```
inputs = layers.Input(shape=(None,), dtype="int32") ❶
x = layers.Embedding(10000, 100)(inputs) ❷
x = layers.LSTM(128, return_sequences=True)(x) ❸
outputs = layers.Dense(10000, activation = 'softmax')(x) ❹
lstm = models.Model(inputs, outputs) ❺

loss_fn = losses.SparseCategoricalCrossentropy()
lstm.compile("adam", loss_fn) ❻
lstm.fit(train_ds, epochs=25) ❼
```

❶ The `Input` layer does not need us to specify the sequence length in advance (it can be flexible), so we use `None` as a placeholder.

❷ The `Embedding` layer requires two parameters, the size of the vocabulary (10,000 tokens) and the dimensionality of the embedding vector (100).

❸ The LSTM layers require us to specify the dimensionality of the hidden vector (128). We also choose to return the full sequence of hidden states, rather than just the hidden state at the final timestep.

❹ The `Dense` layer transforms the hidden states at each timestep into a vector of probabilities for the next token.

❺ The overall `Model` predicts the next token, given an input sequence of tokens. It does this for each token in the sequence.

❻ The model is compiled with `SparseCategoricalCrossentropy` loss—this is the same as categorical cross-entropy, but is used when the labels are integers rather than one-hot encoded vectors.

❼ The model is fit to the training dataset.

In Figure 5-7 you can see the first few epochs of the LSTM training process—notice how the example output becomes more comprehensible as the loss metric falls. Figure 5-8 shows the cross-entropy loss metric falling throughout the training process.

```
Epoch 1/25
628/629 [============================>.] - ETA: 0s - loss: 4.4536
generated text:
recipe for mold salad are high 0 pickled to fold cook the dish into and warm in baking reduced but halves beans
and cut

629/629 [==============================] - 29s 43ms/step - loss: 4.4527
Epoch 2/25
628/629 [============================>.] - ETA: 0s - loss: 3.2339
generated text:
recipe for racks - up-don with herb fizz | serve checking thighs onto sanding butter and baking surface in a hea
vy heavy large saucepan over blender ; stand overnight . [UNK] over moderate heat until very blended , garlic ,
about 8 minutes . cook airtight until cooked are soft seeds , about 1 45 minutes . sugar , until s is brown , 5
to sliced , parmesan , until browned and add extract . wooden crumb to outside of out sheets . flatten and preha
ated return to the paste . add in pecans oval and let transfer day .

629/629 [==============================] - 30s 48ms/step - loss: 3.3336
Epoch 3/25
628/629 [============================>.] - ETA: 0s - loss: 2.6229
generated text:
recipe for grilled chicken | preheat oven to 400°f . cook in large 8 - caramel grinder or until desired are firm
, about 6 minutes

629/629 [==============================] - 27s 42ms/step - loss: 2.6229
Epoch 4/25
628/629 [============================>.] - ETA: 0s - loss: 2.3426
generated text:
recipe for pizza salad with sweet red pepper and star fruit | combine all ingredients except lowest ingredients
in a large skillet . working with batches and deglaze , cook until just cooked through , about 1 minute . meanwh
ile , boil potatoes and paprika in a little oil over medium - high heat , stirring it just until crisp , about 3
minutes . stir in bell pepper , onion and cooked paste and jalapeño until clams well after most - reggiano , abo
ut 5 minutes . transfer warm 2 tablespoons flesh of eggplants to medium bowl . serve .
```

Figure 5-7. The first few epochs of the LSTM training process

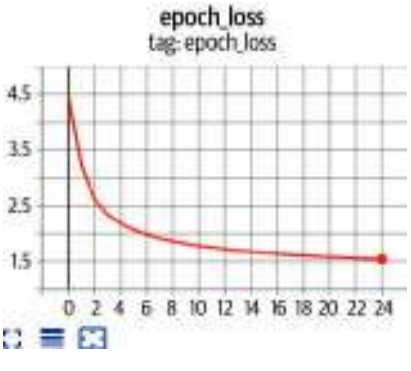


Figure 5-8. The cross-entropy loss metric of the LSTM training process by epoch

Analysis of the LSTM

Now that we have compiled and trained the LSTM, we can start to use it to generate long strings of text by applying the following process:

1. Feed the network with an existing sequence of words and ask it to predict the following word.
2. Append this word to the existing sequence and repeat.

The network will output a set of probabilities for each word that we can sample from. Therefore, we can make the text generation stochastic, rather than deterministic. Moreover, we can introduce a *temperature* parameter to the sampling process to indicate how deterministic we would like the process to be.



The Temperature Parameter

A temperature close to 0 makes the sampling more deterministic (i.e., the word with the highest probability is very likely to be chosen), whereas a temperature of 1 means each word is chosen with the probability output by the model.

This is achieved with the code in Example 5-9, which creates a callback function that can be used to generate text at the end of each training epoch.

Example 5-9. The `TextGenerator` callback function

```
class TextGenerator(callbacks.Callback):
    def __init__(self, index_to_word, top_k=10):
        self.index_to_word = index_to_word
        self.word_to_index = {
            word: index for index, word in enumerate(index_to_word)
        } ❶

    def sample_from(self, probs, temperature): ❷
        probs = probs ** (1 / temperature)
        probs = probs / np.sum(probs)
        return np.random.choice(len(probs), p=probs), probs

    def generate(self, start_prompt, max_tokens, temperature):
        start_tokens = [
            self.word_to_index.get(x, 1) for x in start_prompt.split()
        ] ❸
        sample_token = None
        info = []
        while len(start_tokens) < max_tokens and sample_token != 0: ❹
            x = np.array([start_tokens])
            y = self.model.predict(x) ❺
```

```
            sample_token, probs = self.sample_from(y[0][-1], temperature) ❻
            info.append({'prompt': start_prompt , 'word_probs': probs})
            start_tokens.append(sample_token) ❼
            start_prompt = start_prompt + ' ' + self.index_to_word[sample_token]
        print(f"\ngenerated text:\n{start_prompt}\n")
        return info

    def on_epoch_end(self, epoch, logs=None):
        self.generate("recipe for", max_tokens = 100, temperature = 1.0)
```

❶ Create an inverse vocabulary mapping (from word to token).

❷ This function updates the probabilities with a `temperature` scaling factor.

❸ The start prompt is a string of words that you would like to give the model to start the generation process (for example, *recipe for*). The words are first converted to a list of tokens.

❹ The sequence is generated until it is `max_tokens` long or a stop token (0) is produced.

❺ The model outputs the probabilities of each word being next in the sequence.

❻ The probabilities are passed through the sampler to output the next word, parameterized by `temperature`.

❼ We append the new word to the prompt text, ready for the next iteration of the generative process.

Let's take a look at this in action, at two different temperature values (Figure 5-9).



Figure 5-9. Generated outputs at `temperature = 1.0` and `temperature = 0.2`

There are a few things to note about these two passages. First, both are stylistically similar to a recipe from the original training set. They both open with a recipe title and contain generally grammatically correct constructions. The difference is that the generated text with a temperature of 1.0 is more adventurous and therefore less accurate than the example with a temperature of 0.2. Generating multiple samples with a temperature of 1.0 will therefore lead to more variety, as the model is sampling from a probability distribution with greater variance.

To demonstrate this, Figure 5-10 shows the top five tokens with the highest probabilities for a range of prompts, for both temperature values.

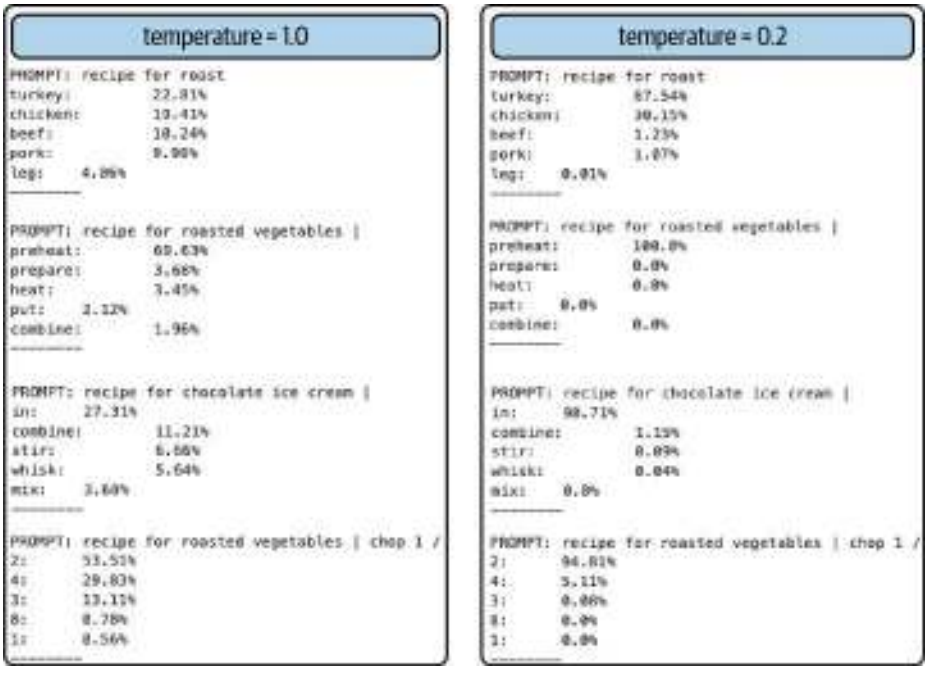


Figure 5-10. Distribution of word probabilities following various sequences, for temperature values of 1.0 and 0.2

The model is able to generate a suitable distribution for the next most likely word across a range of contexts. For example, even though the model was never told about parts of speech such as nouns, verbs, or numbers, it is generally able to separate words into these classes and use them in a way that is grammatically correct.

Moreover, the model is able to select an appropriate verb to begin the recipe instructions, depending on the preceding title. For roasted vegetables, it selects `preheat`, `prepare`, `heat`, `put`, or `combine` as the most likely possibilities, whereas for ice cream it selects `in`, `combine`, `stir`, `whisk`, and `mix`. This shows that the model has some contextual understanding of the differences between recipes depending on their ingredients.

Notice also how the probabilities for the `temperature = 0.2` examples are much more heavily weighted toward the first choice token. This is the reason why there is generally less variety in generations when the temperature is lower.

While our basic LSTM model is doing a great job at generating realistic text, it is clear that it still struggles to grasp some of the semantic meaning of the words that it is generating. It introduces ingredients that are not likely to work well together (for example, sour Japanese potatoes, pecan crumbs, and sorbet)! In some cases, this may be desirable—say, if we want our LSTM to generate interesting and unique patterns of words—but in other cases, we will need our model to have a deeper understanding of the ways in which words can be grouped together and a longer memory of ideas introduced earlier in the text.

In the next section, we'll explore some of the ways that we can improve our basic LSTM network. In Chapter 9, we'll take a look at a new kind of autoregressive model, the Transformer, which takes language modeling to the next level.

Recurrent Neural Network (RNN) Extensions

The model in the preceding section is a simple example of how an LSTM can be trained to learn how to generate text in a given style. In this section we will explore several extensions to this idea.

Stacked Recurrent Networks

The network we just looked at contained a single LSTM layer, but we can also train networks with stacked LSTM layers, so that deeper features can be learned from the text.

To achieve this, we simply introduce another LSTM layer after the first. The second LSTM layer can then use the hidden states from the first layer as its input data. This is shown in Figure 5-11, and the overall model architecture is shown in Table 5-2.

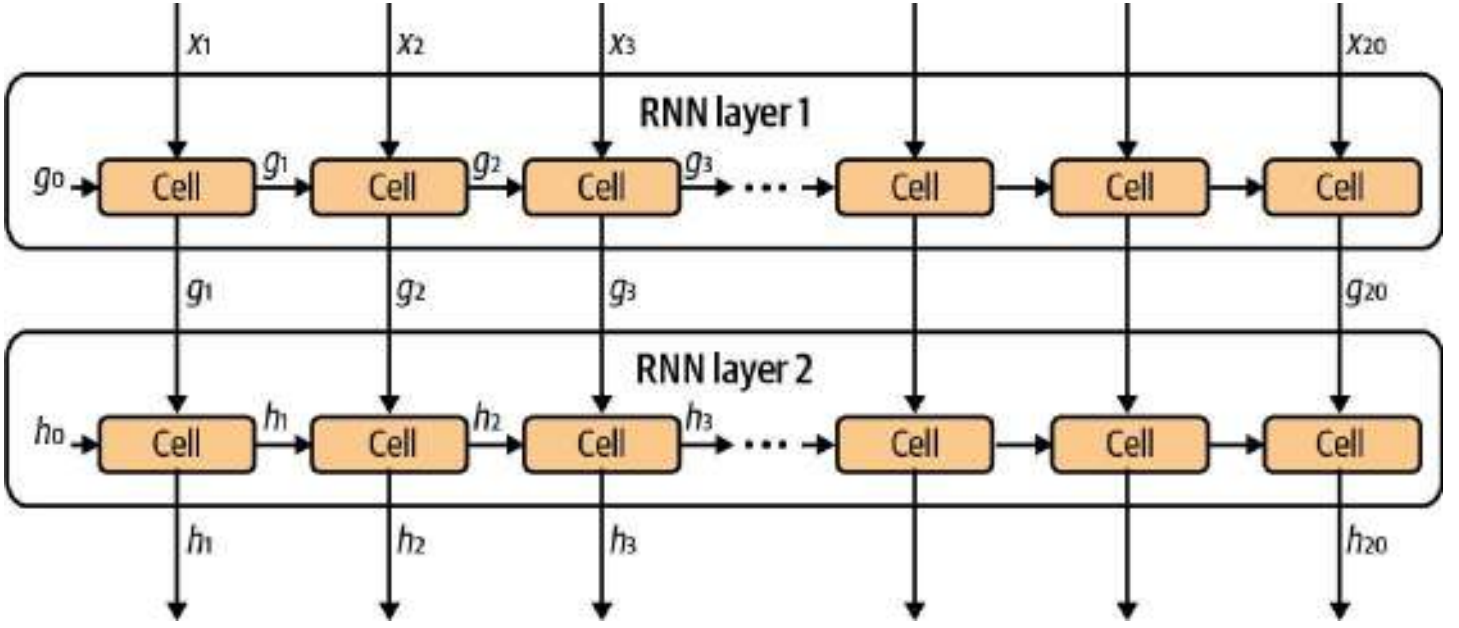


Figure 5-11. Diagram of a multilayer RNN: g_t denotes hidden states of the first layer and h_t denotes hidden states of the second layer

Table 5-2. Model summary of the stacked LSTM

Layer (type)	Output shape	Param #
InputLayer	(None, None)	0
Embedding	(None, None, 100)	1,000,000
LSTM	(None, None, 128)	117,248
LSTM	(None, None, 128)	131,584
Dense	(None, None, 10000)	1,290,000

Total params	2,538,832
Trainable params	2,538,832
Non-trainable params	0

The code to build the stacked LSTM is given in Example 5-10.

Example 5-10. Building a stacked LSTM

```
text_in = layers.Input(shape = (None,))
embedding = layers.Embedding(total_words, embedding_size)(text_in)
x = layers.LSTM(n_units, return_sequences = True)(x)
x = layers.LSTM(n_units, return_sequences = True)(x)
probabilites = layers.Dense(total_words, activation = 'softmax')(x)
model = models.Model(text_in, probabilites)
```

Gated Recurrent Units

Another type of commonly used RNN layer is the *gated recurrent unit* (GRU).[2] The key differences from the LSTM unit are as follows:

1. The *forget* and *input* gates are replaced by *reset* and *update* gates.
2. There is no *cell state* or *output* gate, only a *hidden state* that is output from the cell.

The hidden state is updated in four steps, as illustrated in Figure 5-12.

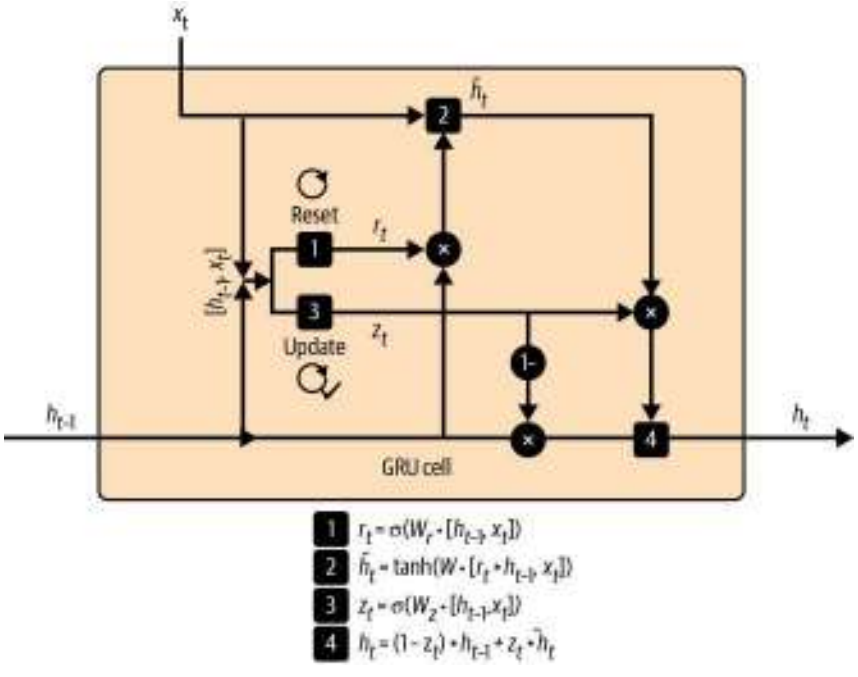


Figure 5-12. A single GRU cell

The process is as follows:

1. The hidden state of the previous timestep, h_{t-1}, and the current word embedding, x_t, are concatenated and used to create the *reset* gate. This gate is a dense layer, with weights matrix W_r and a sigmoid activation function. The resulting vector, r_t, has length equal to the number of units in the cell and stores values between 0 and 1 that determine how much of the previous hidden state, h_{t-1}, should be carried forward into the calculation for the new beliefs of the cell.
2. The reset gate is applied to the hidden state, h_{t-1}, and concatenated with the current word embedding, x_t. This vector is then fed to a dense layer with weights matrix W and a tanh activation function to generate a vector, $\tilde{h}_t$, that stores the new beliefs of the cell. It has length equal to the number of units in the cell and stores values between –1 and 1.
3. The concatenation of the hidden state of the previous timestep, h_{t-1}, and the current word embedding, x_t, are also used to create the *update* gate. This gate is a dense layer with weights matrix W_z and a sigmoid activation. The resulting vector, z_t, has length equal to the number of units in the cell and stores values

between 0 and 1, which are used to determine how much of the new beliefs, $\tilde{h}_t$, to blend into the current hidden state, h_{t-1}.

4. The new beliefs of the cell, $\tilde{h}_t$, and the current hidden state, h_{t-1}, are blended in a proportion determined by the update gate, z_t, to produce the updated hidden state, h_t, that is output from the cell.

Bidirectional Cells

For prediction problems where the entire text is available to the model at inference time, there is no reason to process the sequence only in the forward direction—it could just as well be processed backward. A `Bidirectional` layer takes advantage of this by storing two sets of hidden states: one that is produced as a result of the sequence being processed in the usual forward direction and another that is produced when the sequence is processed backward. This way, the layer can learn from information both preceding and succeeding the given timestep.

In Keras, this is implemented as a wrapper around a recurrent layer, as shown in Example 5-11.

Example 5-11. Building a bidirectional GRU layer

```
layer = layers.Bidirectional(layers.GRU(100))
```



Hidden State

The hidden states in the resulting layer are vectors of length equal to double the number of units in the wrapped cell (a concatenation of the forward and backward hidden states). Thus, in this example the hidden states of the layer are vectors of length 200.

So far, we have only applied autoregressive models (LSTMs) to text data. In the next section, we will see how autoregressive models can also be used to generate images.

PixelCNN

In 2016, van den Oord et al.[3] introduced a model that generates images pixel by pixel by predicting the likelihood of the next pixel based on the pixels before it. The model is called *PixelCNN*, and it can be trained to generate images autoregressively.

There are two new concepts that we need to introduce to understand the PixelCNN—*masked convolutional layers* and *residual blocks*.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/05_autoregressive/02_pixelcnn/pixelcnn.ipynb* in the book repository.

The code has been adapted from the excellent PixelCNN tutorial created by ADMoreau, available on the Keras website.

Masked Convolutional Layers

As we saw in Chapter 2, a convolutional layer can be used to extract features from an image by applying a series of filters. The output of the layer at a particular pixel is a weighted sum of the filter weights multiplied by the preceding layer values over a small square centered on the pixel. This method can detect edges and textures and, at deeper layers, shapes and higher-level features.

Whilst convolutional layers are extremely useful for feature detection, they cannot directly be used in an autoregressive sense, because there is no ordering placed on the pixels. They rely on the fact that all pixels are treated equally—no pixel is treated as the *start* or *end* of the image. This is in contrast to the text data that we have already seen in this chapter, where there is a clear ordering to the tokens so recurrent models such as LSTMs can be readily applied.

For us to be able to apply convolutional layers to image generation in an autoregressive sense, we must first place an ordering on the pixels and ensure that the filters are only able to see pixels that precede the pixel in question. We can then generate images one pixel at a time, by applying convolutional filters to the current image to predict the value of the next pixel from all preceding pixels.

We first need to choose an ordering for the pixels—a sensible suggestion is to order the pixels from top left to bottom right, moving first along the rows and then down the columns.

We then mask the convolutional filters so that the output of the layer at each pixel is only influenced by pixel values that precede the pixel in question. This is achieved by multiplying a mask of ones and zeros with the filter weights matrix, so that the values of any pixels that are after the target pixel are zeroed.

There are actually two different kinds of masks in a PixelCNN, as shown in Figure 5-13:

- Type A, where the value of the central pixel is masked
- Type B, where the value of the central pixel is *not* masked

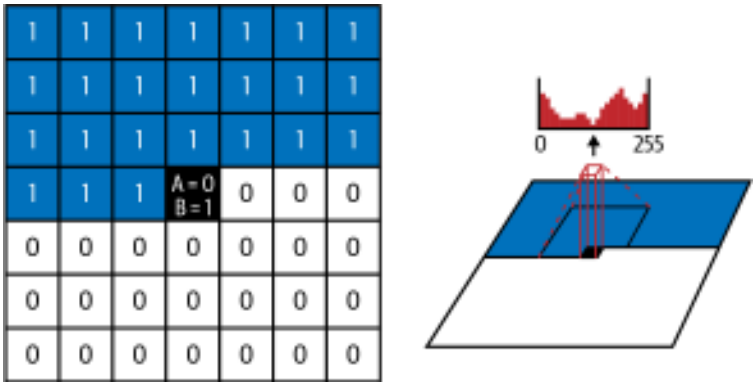


Figure 5-13. Left: a convolutional filter mask; right: a mask applied to a set of pixels to predict the distribution of the central pixel value (source: van den Oord et al., 2016)

The initial masked convolutional layer (i.e., the one that is applied directly to the input image) cannot use the central pixel, because this is precisely the pixel we want the network to guess! However, subsequent layers can use the central pixel because this will have been calculated only as a result of information from preceding pixels in the original input image.

We can see in Example 5-12 how a `MaskedConvLayer` can be built using Keras.

Example 5-12. A `MaskedConvLayer` in Keras

```
class MaskedConvLayer(layers.Layer):
    def __init__(self, mask_type, **kwargs):
        super(MaskedConvLayer, self).__init__()
        self.mask_type = mask_type
        self.conv = layers.Conv2D(**kwargs) ❶

    def build(self, input_shape):
        self.conv.build(input_shape)
        kernel_shape = self.conv.kernel.get_shape()
        self.mask = np.zeros(shape=kernel_shape) ❷
        self.mask[: kernel_shape[0] // 2, ...] = 1.0 ❸
        self.mask[kernel_shape[0] // 2, : kernel_shape[1] // 2, ...] = 1.0 ❹
        if self.mask_type == "B":
            self.mask[kernel_shape[0] // 2, kernel_shape[1] // 2, ...] = 1.0 ❺

    def call(self, inputs):
        self.conv.kernel.assign(self.conv.kernel * self.mask) ❻
        return self.conv(inputs)
```

❶ The `MaskedConvLayer` is based on the normal `Conv2D` layer.

❷ The mask is initialized with all zeros.

❸ The pixels in the preceding rows are unmasked with ones.

❹ The pixels in the preceding columns that are in the same row are unmasked with ones.

❺ If the mask type is B, the central pixel is unmasked with a one.

❻ The mask is multiplied with the filter weights.

Note that this simplified example assumes a grayscale image (i.e., with one channel). If we have color images, we'll have three color channels that we can also place an ordering on so that, for example, the red channel precedes the blue channel, which precedes the green channel.

Residual Blocks

Now that we have seen how to mask the convolutional layer, we can start to build our PixelCNN. The core building block that we will use is the residual block.

A *residual block* is a set of layers where the output is added to the input before being passed on to the rest of the network. In other words, the input has a *fast-track* route to the output, without having to go through the intermediate layers—this is called a *skip connection*. The rationale behind including a skip connection is that if the optimal transformation is just to keep the input the same, this can be achieved by simply zeroing the weights of the intermediate layers. Without the skip connection, the network would have to find an identity mapping through the intermediate layers, which is much harder.

A diagram of the residual block in our PixelCNN is shown in Figure 5-14.

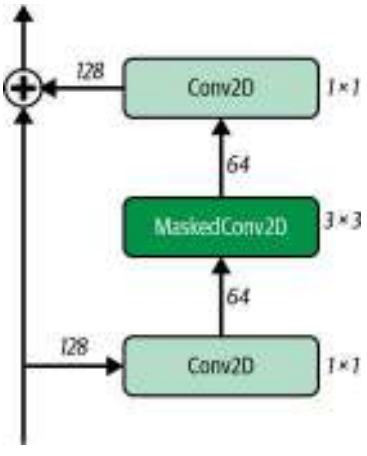


Figure 5-14. A PixelCNN residual block (the numbers of filters are next to the arrows and the filter sizes are next to the layers)

We can build a `ResidualBlock` using the code shown in Example 5-13.

Example 5-13. A `ResidualBlock`

```
class ResidualBlock(layers.Layer):
    def __init__(self, filters, **kwargs):
        super(ResidualBlock, self).__init__(**kwargs)
        self.conv1 = layers.Conv2D(
            filters=filters // 2, kernel_size=1, activation="relu"
        ) ❶
        self.pixel_conv = MaskedConv2D(
            mask_type="B",
            filters=filters // 2,
            kernel_size=3,
            activation="relu",
            padding="same",
        ) ❷
        self.conv2 = layers.Conv2D(
            filters=filters, kernel_size=1, activation="relu"
        ) ❸

    def call(self, inputs):
        x = self.conv1(inputs)
        x = self.pixel_conv(x)
        x = self.conv2(x)
        return layers.add([inputs, x]) ❹
```

❶ The initial `Conv2D` layer halves the number of channels.

❷ The Type B `MaskedConv2D` layer with kernel size of 3 only uses information from five pixels—three pixels in the row above the focus pixel, one to the left, and the focus pixel itself.

❸ The final `Conv2D` layer doubles the number of channels to again match the input shape.

❹ The output from the convolutional layers is added to the input—this is the skip connection.

Training the PixelCNN

In Example 5-14 we put together the whole PixelCNN network, approximately following the structure laid out in the original paper. In the original paper, the output layer is a 256-filter `Conv2D` layer, with softmax activation. In other words, the network tries to re-create its input by predicting the correct pixel values, a bit like an autoencoder. The difference is that the PixelCNN is constrained so that no information from earlier pixels can flow through to influence the prediction for each pixel, due to the way that network is designed, using `MaskedConv2D` layers.

A challenge with this approach is that the network has no way to understand that a pixel value of, say, 200 is very close to a pixel value of 201. It must learn every pixel output value independently, which means training can be very slow, even for the simplest datasets. Therefore, in our implementation, we instead simplify the input so that each pixel can take only one of four values. This way, we can use a 4-filter `Conv2D` output layer instead of 256.

Example 5-14. The PixelCNN architecture

```
inputs = layers.Input(shape=(16, 16, 1)) ❶
x = MaskedConv2D(mask_type="A"
                 , filters=128
                 , kernel_size=7
                 , activation="relu"
                 , padding="same")(inputs)❷

for _ in range(5):
    x = ResidualBlock(filters=128)(x) ❸

for _ in range(2):
    x = MaskedConv2D(
        mask_type="B",
        filters=128,
        kernel_size=1,
        strides=1,
        activation="relu",
```

```
        padding="valid",
    )(x) ❹

out = layers.Conv2D(
    filters=4, kernel_size=1, strides=1, activation="softmax", padding="valid"
)(x) ❺

pixel_cnn = models.Model(inputs, out) ❻

adam = optimizers.Adam(learning_rate=0.0005)
pixel_cnn.compile(optimizer=adam, loss="sparse_categorical_crossentropy")

pixel_cnn.fit(
    input_data
    , output_data
    , batch_size=128
    , epochs=150
) ❼
```

❶ The model `Input` is a grayscale image of size 16 × 16 × 1, with inputs scaled between 0 and 1.

❷ The first Type A `MaskedConv2D` layer with a kernel size of 7 uses information from 24 pixels—21 pixels in the three rows above the focus pixel and 3 to the left (the focus pixel itself is not used).

❸ Five `ResidualBlock` layer groups are stacked sequentially.

❹ Two Type B `MaskedConv2D` layers with a kernel size of 1 act as `Dense` layers across the number of channels for each pixel.

❺ The final `Conv2D` layer reduces the number of channels to four—the number of pixel levels for this example.

❻ The `Model` is built to accept an image and output an image of the same dimensions.

❼ Fit the model—`input_data` is scaled in the range [0, 1] (floats); `output_data` is scaled in the range [0, 3] (integers).

Analysis of the PixelCNN

We can train our PixelCNN on images from the Fashion-MNIST dataset that we encountered in Chapter 3. To generate new images, we need to ask the model to predict the next pixel given all preceding pixels, one pixel at a time. This is a very slow process compared to a model such as a variational autoencoder! For a 32 × 32

grayscale image, we need to make 1,024 predictions sequentially using the model, compared to the single prediction that we need to make for a VAE. This is one of the major downsides to autoregressive models such as a PixelCNN—they are slow to sample from, because of the sequential nature of the sampling process.

For this reason, we use an image size of 16 × 16, rather than 32 × 32, to speed up the generation of new images. The generation callback class is shown in Example 5-15.

Example 5-15. Generating new images using the PixelCNN

```
class ImageGenerator(callbacks.Callback):
    def __init__(self, num_img):
        self.num_img = num_img

    def sample_from(self, probs, temperature):
        probs = probs ** (1 / temperature)
        probs = probs / np.sum(probs)
        return np.random.choice(len(probs), p=probs)

    def generate(self, temperature):
        generated_images = np.zeros(
            shape=(self.num_img,) + (pixel_cnn.input_shape)[1:]
        ) ❶
        batch, rows, cols, channels = generated_images.shape

        for row in range(rows):
            for col in range(cols):
                for channel in range(channels):
                    probs = self.model.predict(generated_images)[
                        :, row, col, :
                    ] ❷
                    generated_images[:, row, col, channel] = [
                        self.sample_from(x, temperature) for x in probs
                    ] ❸
                    generated_images[:, row, col, channel] /= 4 ❹
        return generated_images

    def on_epoch_end(self, epoch, logs=None):
        generated_images = self.generate(temperature = 1.0)
        display(
            generated_images,
            save_to = "./output/generated_img_%03d.png" % (epoch)
        s)

img_generator_callback = ImageGenerator(num_img=10)
```

❶ Start with a batch of empty images (all zeros).

❷ Loop over the rows, columns, and channels of the current image, predicting the distribution of the next pixel value.

❸ Sample a pixel level from the predicted distribution (for our example, a level in the range [0, 3]).

❹ Convert the pixel level to the range [0, 1] and overwrite the pixel value in the current image, ready for the next iteration of the loop.

In Figure 5-15, we can see several images from the original training set, alongside images that have been generated by the PixelCNN.



Figure 5-15. Example images from the training set and generated images created by the PixelCNN model

The model does a great job of re-creating the overall shape and style of the original images! It is quite amazing that we can treat images as a series of tokens (pixel values) and apply autoregressive models such as a PixelCNN to produce realistic samples.

As mentioned previously, one of the downsides to autoregressive models is that they are slow to sample from, which is why a simple example of their application is presented in this book. However, as we shall see in Chapter 10, more complex forms of autoregressive model can be applied to images to produce state-of-the-art outputs. In

such cases, the slow generation speed is a necessary price to pay in return for exceptional-quality outputs.

Since the original paper was published, several improvements have been made to the architecture and training process of the PixelCNN. The following section introduces one of those changes—using mixture distributions—and demonstrates how to train a PixelCNN model with this improvement using a built-in TensorFlow function.

Mixture Distributions

For our previous example, we reduced the output of the PixelCNN to just 4 pixel levels to ensure the network didn't have to learn a distribution over 256 independent pixel values, which would slow the training process. However, this is far from ideal—for color images, we wouldn't want our canvas to be restricted to only a handful of possible colors.

To get around this problem, we can make the output of the network a *mixture distribution*, instead of a softmax over 256 discrete pixel values, following the ideas presented by Salimans et al.[4] A mixture distribution is quite simply a mixture of two or more other probability distributions. For example, we could have a mixture distribution of five logistic distributions, each with different parameters. The mixture distribution also requires a discrete categorical distribution that denotes the probability of choosing each of the distributions included in the mix. An example is shown in Figure 5-16.

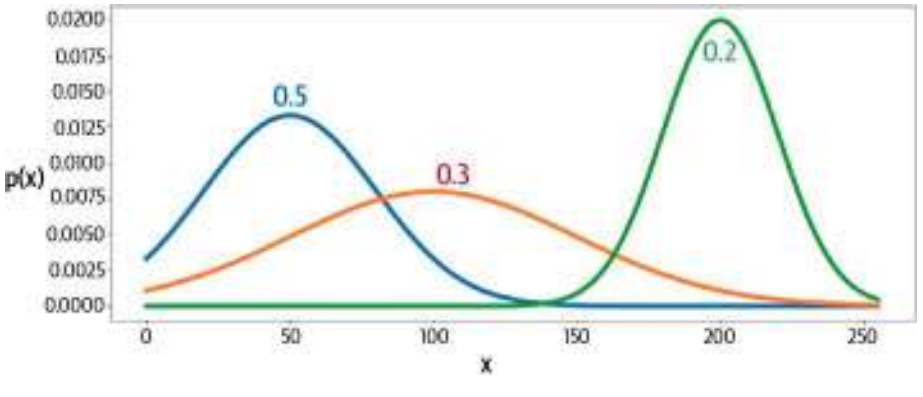


Figure 5-16. A mixture distribution of three normal distributions with different parameters—the categorical distribution over the three normal distributions is `[0.5, 0.3, 0.2]`

To sample from a mixture distribution, we first sample from the categorical distribution to choose a particular subdistribution and then sample from this in the usual way. This way, we can create complex distributions with relatively few parameters.

For example, the mixture distribution in Figure 5-16 only requires eight parameters—two for the categorical distribution and a mean and variance for each of the three normal distributions. This is compared to the 255 parameters that would define a categorical distribution over the entire pixel range.

Conveniently, the TensorFlow Probability library provides a function that allows us to create a PixelCNN with mixture distribution output in a single line. Example 5-16 illustrates how to build a PixelCNN using this function.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook in *notebooks/05_autoregressive/03_pixelcnn_md/pixelcnn_md.ipynb* in the book repository.

Example 5-16. Building a PixelCNN using the TensorFlow function

```
import tensorflow_probability as tfp

dist = tfp.distributions.PixelCNN(
    image_shape=(32, 32, 1),
    num_resnet=1,
    num_hierarchies=2,
    num_filters=32,
    num_logistic_mix=5,
    dropout_p=.3,
) ❶

image_input = layers.Input(shape=(32, 32, 1)) ❷

log_prob = dist.log_prob(image_input)

model = models.Model(inputs=image_input, outputs=log_prob) ❸
model.add_loss(-tf.reduce_mean(log_prob)) ❹
```

❶ Define the PixelCNN as a distribution—i.e., the output layer is a mixture distribution made up of five logistic distributions.

❷ The input is a grayscale image of size $32 \times 32 \times 1$.

❸ The `Model` takes a grayscale image as input and outputs the log-likelihood of the image under the mixture distribution calculated by the PixelCNN.

❹ The loss function is the mean negative log-likelihood over the batch of input images.

The model is trained in the same way as before, but this time accepting integer pixel values as input, in the range [0, 255]. Outputs can be generated from the distribution using the `sample` function, as shown in Example 5-17.

Example 5-17. Sampling from the PixelCNN mixture distribution

```
dist.sample(10).numpy()
```

Example generated images are shown in Figure 5-17. The difference from our previous examples is that now the full range of pixel values is being utilized.



Figure 5-17. Outputs from the PixelCNN using a mixture distribution output

Summary

In this chapter we have seen how autoregressive models such as recurrent neural networks can be applied to generate text sequences that mimic a particular style of writing, and also how a PixelCNN can generate images in a sequential fashion, one pixel at a time.

We explored two different types of recurrent layers—long short-term memory (LSTM) and gated recurrent unit (GRU)—and saw how these cells can be stacked or made bidirectional to form more complex network architectures. We built an LSTM to generate realistic recipes using Keras and saw how to manipulate the temperature of the sampling process to increase or decrease the randomness of the output.

We also saw how images can be generated in an autoregressive manner, using a PixelCNN. We built a PixelCNN from scratch using Keras, coding the masked convolutional layers and residual blocks to allow information to flow through the network so that only preceding pixels could be used to generate the current pixel. Finally, we discussed how the TensorFlow Probability library provides a standalone `PixelCNN` function that implements a mixture distribution as the output layer, allowing us to further improve the learning process.

In the next chapter we will explore another generative modeling family that explicitly models the data-generating distribution—normalizing flow models.

References

1. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation* 9 (1997): 1735–1780, *https://www.bioinf.jku.at/publications/older/2604.pdf*.

2. Kyunghyun Cho et al., "Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation," June 3, 2014, *https://arxiv.org/abs/1406.1078*.

3. Aaron van den Oord et al., "Pixel Recurrent Neural Networks," August 19, 2016, *https://arxiv.org/abs/1601.06759*.

4. Tim Salimans et al., "PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications," January 19, 2017, *http://arxiv.org/abs/1701.05517*.

CHAPTER 6
Normalizing Flow Models

Chapter Goals

In this chapter you will:

- Learn how normalizing flow models utilize the change of variables equation.
- See how the Jacobian determinant plays a vital role in our ability to compute an explicit density function.
- Understand how we can restrict the form of the Jacobian using coupling layers.
- See how the neural network is designed to be invertible.
- Build a RealNVP model—a particular example of a normalizing flow to generate points in 2D.
- Use the RealNVP model to generate new points that appear to have been drawn from the data distribution.
- Learn about two key extensions of the RealNVP model, GLOW and FFJORD.

So far, we have discussed three families of generative models: variational autoencoders, generative adversarial networks, and autoregressive models. Each presents a different way to address the challenge of modeling the distribution $p(x)$, either by introducing a latent variable that can be easily sampled (and transformed using the decoder in VAEs or generator in GANs), or by tractably modeling the distribution as a function of the values of preceding elements (autoregressive models).

In this chapter, we will cover a new family of generative models—normalizing flow models. As we shall see, normalizing flows share similarities with both autoregressive models and variational autoencoders. Like autoregressive models, normalizing flows are able to explicitly and tractably model the data-generating distribution $p(x)$. Like

VAEs, normalizing flows attempt to map the data into a simpler distribution, such as a Gaussian distribution. The key difference is that normalizing flows place a constraint on the form of the mapping function, so that it is invertible and can therefore be used to generate new data points.

We will dig into this definition in detail in the first section of this chapter before implementing a normalizing flow model called RealNVP using Keras. We will also see how normalizing flows can be extended to create more powerful models, such as GLOW and FFJORD.

Introduction

We will begin with a short story to illustrate the key concepts behind normalizing flows.

Jacob and the F.L.O.W. Machine

Upon visiting a small village, you notice a mysterious-looking shop with a sign above the door that says *JACOB'S*. Intrigued, you cautiously enter and ask the old man standing behind the counter what he sells (Figure 6-1).



Figure 6-1. Inside a steampunk shop, with a large metallic bell (created with Midjourney)

He replies that he offers a service for digitizing paintings, with a difference. After a brief moment rummaging around the back of the shop, he brings out a silver box, embossed with the letters F.L.O.W. He tells you that this stands for Finding Likenesses Of Watercolors, which approximately describes what the machine does. You decide to give the machine a try.

You come back the next day and hand the shopkeeper a set of your favorite paintings, and he passes them through the machine. The F.L.O.W. machine begins to hum and whistle and after a while outputs a set of numbers that appear randomly generated. The shopkeeper hands you the list and begins to walk to the till to calculate how much you owe him for the digitization process and the F.L.O.W. box. Distinctly unimpressed, you ask the shopkeeper what you should do with this long list of numbers, and how you can get your favorite paintings back.

The shopkeeper rolls his eyes, as if the answer should be obvious. He walks back to the machine and passes in the long list of numbers, this time from the opposite side. You hear the machine whir again and wait, puzzled, until finally your original paintings drop out from where they entered.

Relieved to finally have your paintings back, you decide that it might be best to just store them in the attic instead. However, before you have a chance to leave, the shopkeeper ushers you across to a different corner of the shop, where a giant bell hangs from the rafters. He hits the bell curve with a huge stick, sending vibrations around the store.

Instantly, the F.L.O.W. machine under your arm begins to hiss and whirr in reverse, as if a new set of numbers had just been passed in. After a few moments, more beautiful watercolor paintings begin to fall out of the F.L.O.W. machine, but they are not the same as the ones you originally digitized. They resemble the style and form of your original set of paintings, but each one is completely unique!

You ask the shopkeeper how this incredible device works. He explains that the magic lies in the fact that he has developed a special process that ensures the transformation is extremely fast and simple to calculate while still being sophisticated enough to convert the vibrations produced by the bell into the complex patterns and shapes present in the paintings.

Realizing the potential of this contraption, you hurriedly pay for the device and exit the store, happy that you now have a way to generate new paintings in your favorite style, simply by visiting the shop, chiming the bell, and waiting for your F.L.O.W. machine to work its magic!

The story of Jacob and the F.L.O.W. machine is a depiction of a normalizing flow model. Let's now explore the theory of normalizing flows in more detail, before we implement a practical example using Keras.

Normalizing Flows

The motivation of normalizing flow models is similar to that of variational autoencoders, which we explored in Chapter 3. To recap, in a variational autoencoder, we learn an *encoder* mapping function between a complex distribution and a much simpler distribution that we can sample from. We then also learn a *decoder* mapping

function from the simpler distribution to the complex distribution, so that we can generate a new data point by sampling a point z from the simpler distribution and applying the learned transformation. Probabilistically speaking, the decoder models $p(x|z)$ but the encoder is only an approximation $q(z|x)$ of the true $p(z|x)$—the encoder and decoder are two completely distinct neural networks.

In a normalizing flow model, the decoding function is designed to be the exact inverse of the encoding function and quick to calculate, giving normalizing flows the property of tractability. However, neural networks are not by default invertible functions. This raises the question of how we can create an invertible process that converts between a complex distribution (such as the data generation distribution of a set of watercolor paintings) and a much simpler distribution (such as a bell-shaped Gaussian distribution) while still making use of the flexibility and power of deep learning.

To answer this question, we first need to understand a technique known as *change of variables*. For this section, we will work with a simple example in just two dimensions, so that you can see exactly how normalizing flows work in fine detail. More complex examples are just extensions of the basic techniques presented here.

Change of Variables

Suppose we have a probability distribution $p_X(x)$ defined over a rectangle X in two dimensions ($x = (x_1, x_2)$), as shown in Figure 6-2.

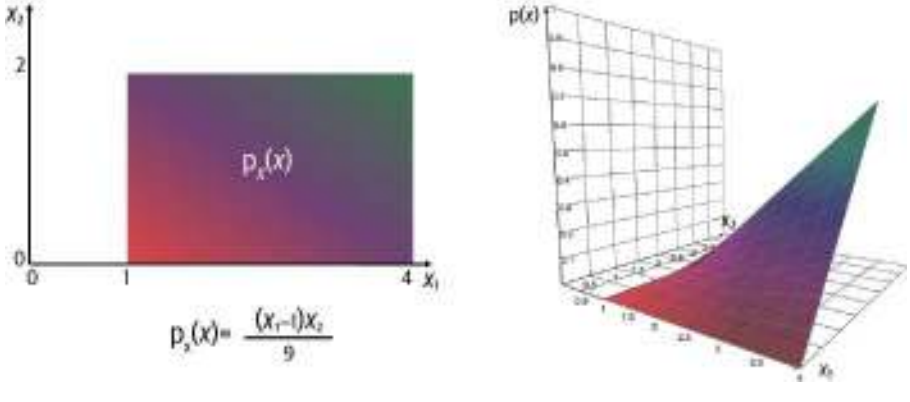


Figure 6-2. A probability distribution $p_X(x)$ defined over two dimensions, shown in 2D (left) and 3D (right)

This function integrates to 1 over the domain of the distribution (i.e., x_1 in the range [1, 4] and x_2 in the range [0, 2]), so it represents a well-defined probability distribution. We can write this as follows:

$$\int_0^2 \int_1^4 p_X(x) dx_1 dx_2 = 1$$

Let's say that we want to shift and scale this distribution so that it is instead defined over a unit square Z. We can achieve this by defining a new variable $z = (z_1, z_2)$ and a function f that maps each point in X to exactly one point in Z as follows:

$$\begin{aligned} z &= f(x) \\ z_1 &= \frac{x_1 - 1}{3} \\ z_2 &= \frac{x_2}{2} \end{aligned}$$

Note that this function is *invertible*. That is, there is a function g that maps every z back to its corresponding x. This is essential for a change of variables, as otherwise we cannot consistently map backward and forward between the two spaces. We can find g simply by rearranging the equations that define f, as shown in Figure 6-3.

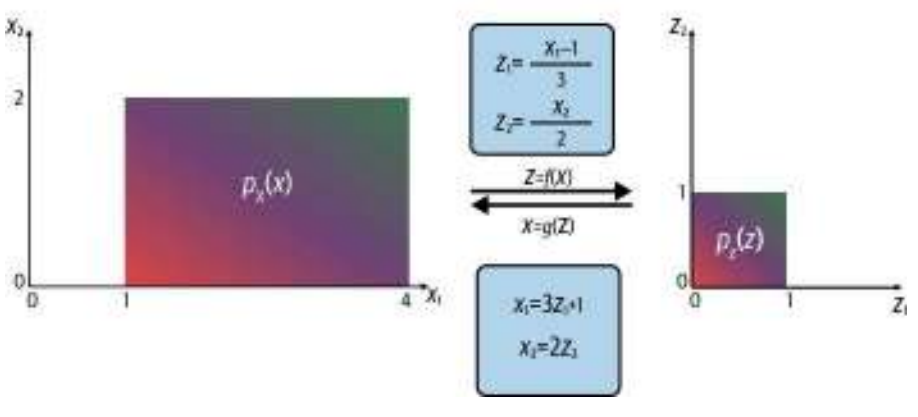


Figure 6-3. Changing variables between X and Z

We now need to see how the change of variables from X to Z affects the probability distribution $p_X(x)$. We can do this by plugging the equations that define g into $p_X(x)$ to transform it into a function $p_Z(z)$ that is defined in terms of z:

$$\begin{aligned} p_Z(z) &= \frac{\big((3z_1 + 1) - 1\big)(2z_2)}{9} \\ &= \frac{2z_1 z_2}{3} \end{aligned}$$

However, if we now integrate $p_Z(z)$ over the unit square, we can see that we have a problem!

$$\int_0^1 \int_0^1 \frac{2z_1 z_2}{3} dz_1 dz_2 = \frac{1}{6}$$

The transformed function $p_Z(z)$ is now no longer a valid probability distribution, because it only integrates to 1/6. If we want to transform our complex probability distribution over the data into a simpler distribution that we can sample from, we must ensure that it integrates to 1.

The missing factor of 6 is due to the fact that the domain of our transformed probability distribution is six times smaller than the original domain—the original rectangle X had area 6, and this has been compressed into a unit square Z that only has area 1. Therefore, we need to multiply the new probability distribution by a normalization factor that is equal to the relative change in area (or volume in higher dimensions).

Luckily, there is a way to calculate this volume change for a given transformation—it is the absolute value of the Jacobian determinant of the transformation. Let's unpack that!

The Jacobian Determinant

The *Jacobian* of a function $z = f(x)$ is the matrix of its first-order partial derivatives, as shown here:

$$\mathrm{J} = \frac{\partial z}{\partial x} = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_n} \\ \ddots & \vdots & \\ \frac{\partial z_m}{\partial x_1} & \cdots & \frac{\partial z_m}{\partial x_n} \end{bmatrix}$$

The best way to explain this is with our example. If we take the partial derivative of z_1 with respect to x_1, we obtain $\frac{1}{3}$. If we take the partial derivative of z_1 with respect to x_2, we obtain 0. Similarly, if we take the partial derivative of z_2 with respect to x_1, we obtain 0. Lastly, if we take the partial derivative of z_2 with respect to x_2, we obtain $\frac{1}{2}$.

Therefore, the Jacobian matrix for our function $f(x)$ is as follows:

$$J = \begin{pmatrix} \frac{1}{3} & 0 \\ 0 & \frac{1}{2} \end{pmatrix}$$

The *determinant* is only defined for square matrices and is equal to the signed volume of the parallelepiped created by applying the transformation represented by the matrix to the unit (hyper)cube. In two dimensions, this is therefore just the signed area of the parallelogram created by applying the transformation represented by the matrix to the unit square.

There is a general formula for calculating the determinant of a matrix with n dimensions, which runs in $\mathcal{O}\left(n^3\right)$ time. For our example, we only need the formula for two dimensions, which is simply as follows:

$$\det\begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

Therefore, for our example, the determinant of the Jacobian is $\frac{1}{3} \times \frac{1}{2} - 0 \times 0 = \frac{1}{6}$. This is the scaling factor of 1/6 that we need to ensure that the probability distribution after transformation still integrates to 1!



By definition, the determinant is signed—that is, it can be negative. Therefore we need to take the absolute value of the Jacobian determinant in order to obtain the relative change of volume.

The Change of Variables Equation

We can now write down a single equation that describes the process for changing variables between X and Z. This is known as the *change of variables equation* (Equation 6-1).

Equation 6-1. The change of variables equation

$$p_X(x) = p_Z(z)\left|\det\left(\frac{\partial z}{\partial x}\right)\right|$$

How does this help us build a generative model? The key is understanding that if $p_Z(z)$ is a simple distribution from which we can easily sample (e.g., a Gaussian), then in theory, all we need to do is find an appropriate invertible function $f(x)$ that can map from the data X into Z and the corresponding inverse function $g(z)$ that can be

used to map a sampled z back to a point x in the original domain. We can use the preceding equation involving the Jacobian determinant to find an exact, tractable formula for the data distribution $p(x)$.

However, there are two major issues when applying this in practice that we first need to address!

Firstly, calculating the determinant of a high-dimensional matrix is computationally extremely expensive—specifically, it is $\mathcal{O}(n^3)$. This is completely impractical to implement in practice, as even small 32×32–pixel grayscale images have 1,024 dimensions.

Secondly, it is not immediately obvious how we should go about calculating the invertible function $f(x)$. We could use a neural network to find some function $f(x)$ but we cannot necessarily invert this network—neural networks only work in one direction!

To solve these two problems, we need to use a special neural network architecture that ensures that the change of variables function f is invertible and has a determinant that is easy to calculate.

We shall see how to do this in the following section using a technique called *real-valued non-volume preserving (RealNVP) transformations*.

RealNVP

RealNVP was first introduced by Dinh et al. in 2017.[1] In this paper the authors show how to construct a neural network that can transform a complex data distribution into a simple Gaussian, while also possessing the desired properties of being invertible and having a Jacobian that can be easily calculated.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/06_normflow/01_realnvp/realnvp.ipynb* in the book repository.

The code has been adapted from the excellent RealNVP tutorial created by Mandolini Giorgio Maria et al. available on the Keras website.

The Two Moons Dataset

The dataset we will use for this example is created by the `make_moons` function from the Python library `sklearn`. This creates a noisy dataset of points in 2D that resemble two crescents, as shown in Figure 6-4.

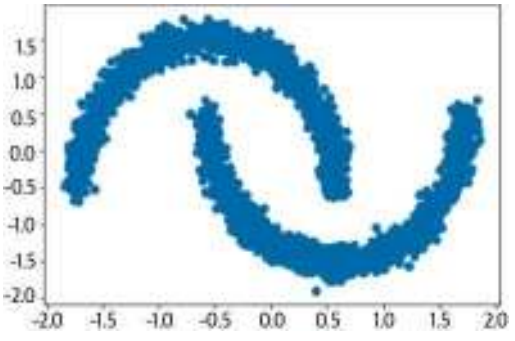


Figure 6-4. The two moons dataset in two dimensions

The code for creating this dataset is given in Example 6-1.

Example 6-1. Creating a moons dataset

```
data = datasets.make_moons(3000, noise=0.05)[0].astype("float32") ❶
norm = layers.Normalization()
norm.adapt(data)
normalized_data = norm(data) ❷
```

❶ Make a noisy, unnormalized moons dataset of 3,000 points.

❷ Normalize the dataset to have mean 0 and standard deviation 1.

We will build a RealNVP model that can generate points in 2D that follow a similar distribution to the two moons dataset. Whilst this is a very simple example, it will help us understand how a normalizing flow model works in practice, in fine detail.

First, however, we need to introduce a new type of layer, called a coupling layer.

Coupling Layers

A *coupling layer* produces a scale and translation factor for each element of its input. In other words, it produces two tensors that are exactly the same size as the input, one for the scale factor and one for the translation factor, as shown in Figure 6-5.



Figure 6-5. A coupling layer outputs two tensors that are the same shape as the input: a scaling factor (s) and a translation factor (t)

To build a custom `Coupling` layer for our simple example, we can stack `Dense` layers to create the scale output and a different set of `Dense` layers to create the translation output, as shown in Example 6-2.



For images, `Coupling` layer blocks use `Conv2D` layers instead of `Dense` layers.

Example 6-2. A `Coupling` layer in Keras

```
def Coupling():
    input_layer = layers.Input(shape=2) ❶

    s_layer_1 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(input_layer) ❷
    s_layer_2 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_1)
    s_layer_3 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_2)
    s_layer_4 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_3)
    s_layer_5 = layers.Dense(
        2, activation="tanh", kernel_regularizer=regularizers.l2(0.01)
    )(s_layer_4) ❸

    t_layer_1 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(input_layer) ❹
    t_layer_2 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(t_layer_1)
    t_layer_3 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(t_layer_2)
    t_layer_4 = layers.Dense(
        256, activation="relu", kernel_regularizer=regularizers.l2(0.01)
    )(t_layer_3)
    t_layer_5 = layers.Dense(
        2, activation="linear", kernel_regularizer=regularizers.l2(0.01)
    )(t_layer_4) ❺

    return models.Model(inputs=input_layer, outputs=[s_layer_5, t_layer_5]) ❻
```

❶ The input to the `Coupling` layer block in our example has two dimensions.

❷ The *scaling* stream is a stack of `Dense` layers of size 256.

❸ The final scaling layer is of size 2 and has `tanh` activation.

❹ The *translation* stream is a stack of `Dense` layers of size 256.

❺ The final translation layer is of size 2 and has `linear` activation.

❻ The `Coupling` layer is constructed as a Keras `Model` with two outputs (the scaling and translation factors).

Notice how the number of channels is temporarily increased to allow for a more complex representation to be learned, before being collapsed back down to the same number of channels as the input. In the original paper, the authors also use regularizers on each layer to penalize large weights.

Passing data through a coupling layer

The architecture of a coupling layer is not particularly interesting—what makes it unique is the way the input data is masked and transformed as it is fed through the layer, as shown in Figure 6-6.

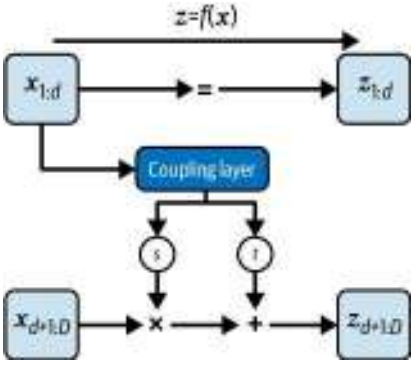


Figure 6-6. The process of transforming the input x through a coupling layer

Notice how only the first d dimensions of the data are fed through to the first coupling layer—the remaining $D - d$ dimensions are completely masked (i.e., set to zero). In our simple example with $D = 2$, choosing $d = 1$ means that instead of the coupling layer seeing two values, (x_1, x_2), the layer sees $(x_1, 0)$.

The outputs from the layer are the scale and translation factors. These are again masked, but this time with the *inverse* mask to previously, so that only the second halves are let through—i.e., in our example, we obtain $(0, s_2)$ and $(0, t_2)$. These are then applied element-wise to the second half of the input x_2 and the first half of the input x_1 is simply passed straight through, without being updated at all. In summary, for a vector with dimension D where $d < D$, the update equations are as follows:

$$\begin{aligned} z_{1:d} &= x_{1:d} \\ z_{d+1:D} &= x_{d+1:D} \odot \exp\left(s\left(x_{1:d}\right)\right) + t\left(x_{1:d}\right) \end{aligned}$$

You may be wondering why we go to the trouble of building a layer that masks so much information. The answer is clear if we investigate the structure of the Jacobian matrix of this function:

$$\frac{\partial z}{\partial x} = \begin{bmatrix} \mathbf{I} & 0 \\ \dfrac{\partial z_{d+1:D}}{\partial x_{1:d}} & \operatorname{diag}\left(\exp\left[s\left(x_{1:d}\right)\right]\right) \end{bmatrix}$$

The top-left $d \times d$ submatrix is simply the identity matrix, because $z_{1:d} = x_{1:d}$. These elements are passed straight through without being updated. The top-right submatrix is therefore 0, because $z_{1:d}$ is not dependent on $x_{d+1:D}$.

The bottom-left submatrix is complex, and we do not seek to simplify this. The bottom-right submatrix is simply a diagonal matrix, filled with the elements of $\exp\left(s\left(x_{1:d}\right)\right)$, because $z_{d+1:D}$ is linearly dependent on $x_{d+1:D}$ and the gradient is dependent only on the scaling factor (not on the translation factor). Figure 6-7 shows a diagram of this matrix form, where only the nonzero elements are filled in with color.

Notice how there are no nonzero elements above the diagonal—for this reason, this matrix form is called *lower triangular*. Now we see the benefit of structuring the matrix in this way—the determinant of a lower-triangular matrix is just equal to the product of the diagonal elements. In other words, the determinant is not dependent on any of the complex derivatives in the bottom-left submatrix!

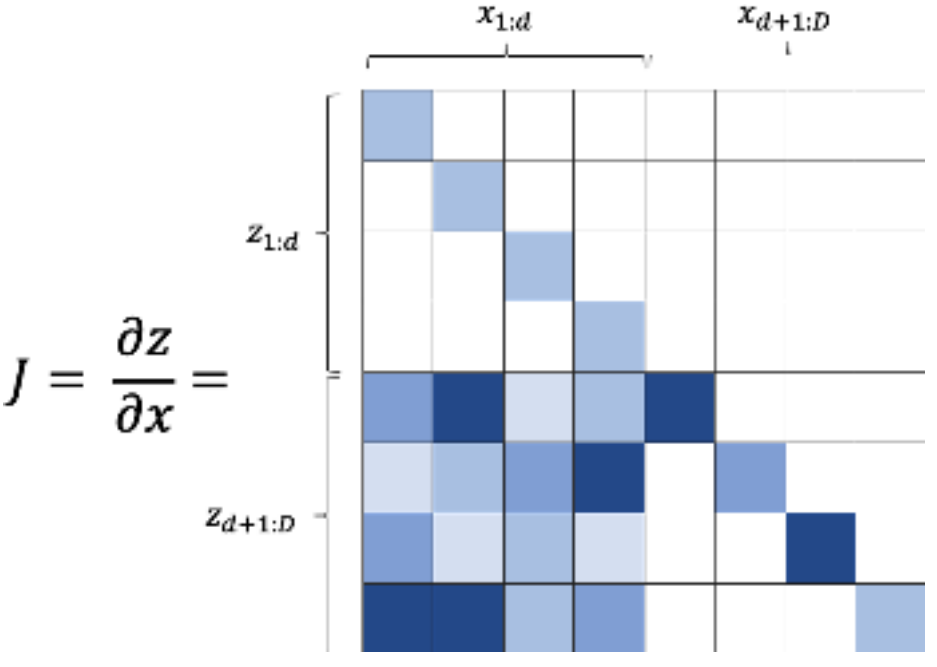


Figure 6-7. The Jacobian matrix of the transformation—a lower triangular matrix, with determinant equal to the product of the elements along the diagonal

Therefore, we can write the determinant of this matrix as follows:

$$\det(\mathrm{J}) = \exp\left[\sum_j s(x_{1:d})_j\right]$$

This is easily computable, which was one of the two original goals of building a normalizing flow model.

The other goal was that the function must be easily invertible. We can see that this is true as we can write down the invertible function just by rearranging the forward equations, as follows:

$$\begin{aligned} x_{1:d} &= z_{1:d} \\ x_{d+1:D} &= \left(z_{d+1:D} - t(x_{1:d})\right) \odot \exp\left(-s(x_{1:d})\right) \end{aligned}$$

The equivalent diagram is shown in Figure 6-8.

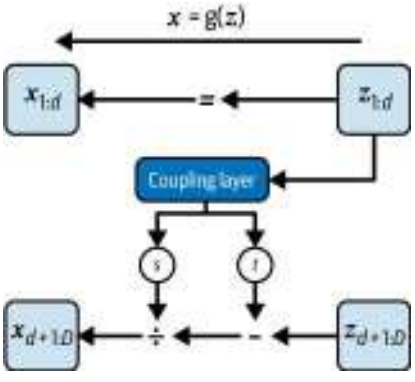


Figure 6-8. The inverse function x = g(z)

We now have almost everything we need to build our RealNVP model. However, there is one issue that still remains—how should we update the first *d* elements of the input? Currently they are left completely unchanged by the model!

Stacking coupling layers

To resolve this problem, we can use a really simple trick. If we stack coupling layers on top of each other but alternate the masking pattern, the layers that are left unchanged by one layer will be updated in the next. This architecture has the added benefit of being able to learn more complex representations of the data, as it is a deeper neural network.

The Jacobian of this composition of coupling layers will still be simple to compute, because linear algebra tells us that the determinant of a matrix product is the product of the determinants. Similarly, the inverse of the composition of two functions is just the composition of the inverses, as shown in the following equations:

$$\det(A \cdot B) = \det(A)\det(B)$$
$$(f_b \circ f_a)^{-1} = f_a^{-1} \circ f_b^{-1}$$

Therefore, if we stack coupling layers, flipping the masking each time, we can build a neural network that is able to transform the whole input tensor, while retaining the essential properties of having a simple Jacobian determinant and being invertible. Figure 6-9 shows the overall structure.

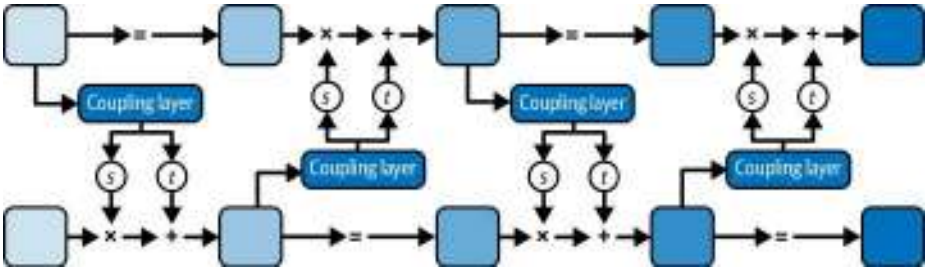


Figure 6-9. Stacking coupling layers, alternating the masking with each layer

Training the RealNVP Model

Now that we have built the RealNVP model, we can train it to learn the complex distribution of the two moons dataset. Remember, we want to minimize the negative log-likelihood of the data under the model $-\log p_X(x)$. Using Equation 6-1, we can write this as follows:

$$-\log p_X(x) = -\log p_Z(z) - \log \left|\det\left(\frac{\partial z}{\partial x}\right)\right|$$

We choose the target output distribution $p_Z(z)$ of the forward process f to be a standard Gaussian, because we can easily sample from this distribution. We can then transform a point sampled from the Gaussian back into the original image domain by applying the inverse process g, as shown in Figure 6-10.

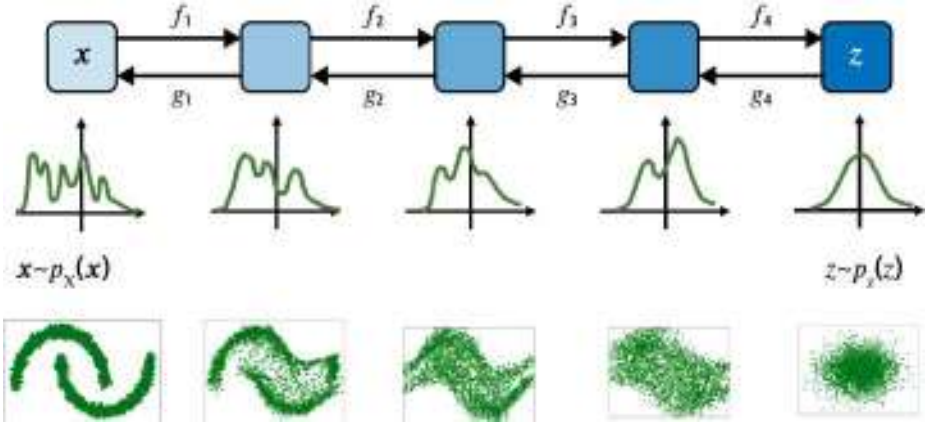


Figure 6-10. Transforming between the complex distribution $p_X(x)$ and a simple Gaussian $p_Z(z)$ in 1D (middle row) and 2D (bottom row)

Example 6-3 shows how to build a RealNVP network, as a custom Keras `Model`.

Example 6-3. Building the RealNVP model in Keras

```
class RealNVP(models.Model):
    def __init__(self, input_dim, coupling_layers, coupling_dim, regularization):
        super(RealNVP, self).__init__()
        self.coupling_layers = coupling_layers
        self.distribution = tfp.distributions.MultivariateNormalDiag(
            loc=[0.0, 0.0], scale_diag=[1.0, 1.0]
        ) ❶
        self.masks = np.array(
            [[0, 1], [1, 0]] * (coupling_layers // 2), dtype="float32"
        ) ❷
        self.loss_tracker = metrics.Mean(name="loss")
        self.layers_list = [
            Coupling(input_dim, coupling_dim, regularization)
            for i in range(coupling_layers)
        ] ❸

    @property
    def metrics(self):
        return [self.loss_tracker]

    def call(self, x, training=True):
        log_det_inv = 0
        direction = 1
        if training:
            direction = -1
        for i in range(self.coupling_layers)[::direction]: ❹
            x_masked = x * self.masks[i]
            reversed_mask = 1 - self.masks[i]
            s, t = self.layers_list[i](x_masked)
            s *= reversed_mask
            t *= reversed_mask
            gate = (direction - 1) / 2
            x = (
                reversed_mask
                * (x * tf.exp(direction * s) + direction * t * tf.exp(gate * s))
                + x_masked
            ) ❺
            log_det_inv += gate * tf.reduce_sum(s, axis = 1) ❻
        return x, log_det_inv

    def log_loss(self, x):
        y, logdet = self(x)
        log_likelihood = self.distribution.log_prob(y) + logdet ❼
        return -tf.reduce_mean(log_likelihood)

    def train_step(self, data):
        with tf.GradientTape() as tape:
```

```
            loss = self.log_loss(data)
        g = tape.gradient(loss, self.trainable_variables)
        self.optimizer.apply_gradients(zip(g, self.trainable_variables))
        self.loss_tracker.update_state(loss)
        return {"loss": self.loss_tracker.result()}

    def test_step(self, data):
        loss = self.log_loss(data)
        self.loss_tracker.update_state(loss)
        return {"loss": self.loss_tracker.result()}

model = RealNVP(
    input_dim = 2
    , coupling_layers= 6
    , coupling_dim = 256
    , regularization = 0.01
)

model.compile(optimizer=optimizers.Adam(learning_rate=0.0001))

model.fit(
    normalized_data
    , batch_size=256
    , epochs=300
)
```

❶ The target distribution is a standard 2D Gaussian.

❷ Here, we create the alternating mask pattern.

❸ A list of `Coupling` layers that define the RealNVP network.

❹ In the main `call` function of the network, we loop over the `Coupling` layers. If `training=True`, then we move forward through the layers (i.e., from data to latent space). If `training=False`, then we move backward through the layers (i.e., from latent space to data).

❺ This line describes both the forward and backward equations dependent on the `direction` (try plugging in `direction = -1` and `direction = 1` to prove this to yourself!).

❻ The log determinant of the Jacobian, which we need to calculate the loss function, is simply the sum of the scaling factors.

❼ The loss function is the negative sum of the log probability of the transformed data, under our target Gaussian distribution and the log determinant of the Jacobian.

Analysis of the RealNVP Model

Once the model is trained, we can use it to transform the training set into the latent space (using the forward direction, f) and, more importantly, to transform a sampled point in the latent space into a point that looks like it could have been sampled from the original data distribution (using the backward direction, g).

Figure 6-11 shows the output from the network before any learning has taken place—the forward and backward directions just pass information straight through with hardly any transformation.

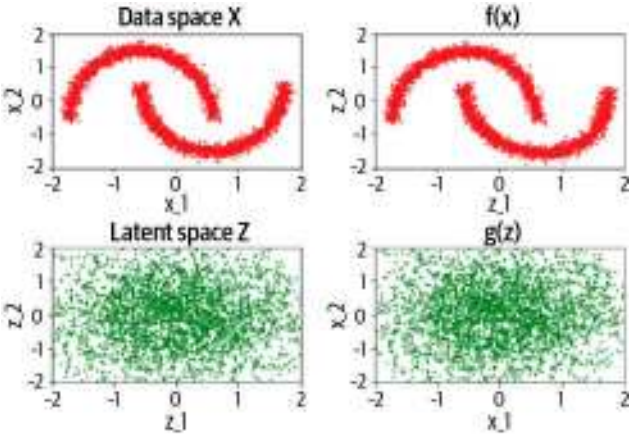


Figure 6-11. The RealNVP model inputs (left) and outputs (right) before training, for the forward process (top) and the reverse process (bottom)

After training (Figure 6-12), the forward process is able to convert the points from the training set into a distribution that resembles a Gaussian. Likewise, the backward process can take points sampled from a Gaussian distribution and map them back to a distribution that resembles the original data.

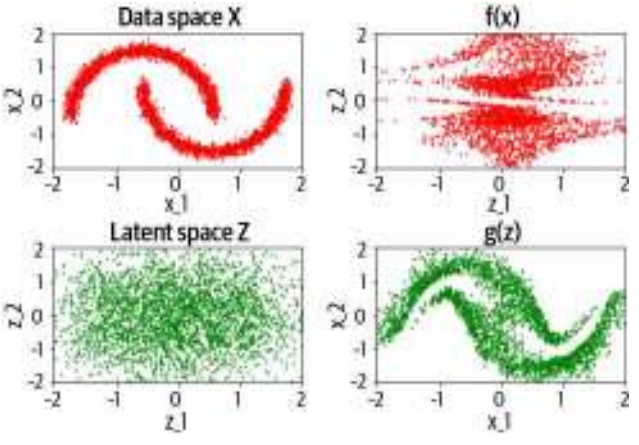


Figure 6-12. The RealNVP model inputs (left) and outputs (right) after training, for the forward process (top) and the reverse process (bottom)

The loss curve for the training process is shown in Figure 6-13.

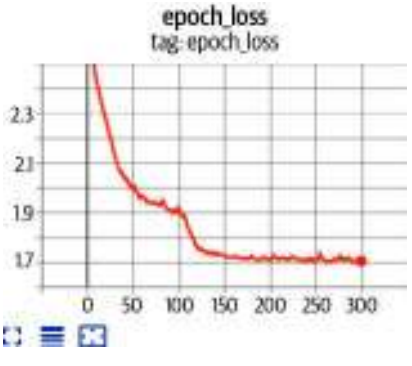


Figure 6-13. The loss curve for the RealNVP training process

This completes our discussion of RealNVP, a specific case of a normalizing flow generative model. In the next section, we'll cover some modern normalizing flow models that extend the ideas introduced in the RealNVP paper.

Other Normalizing Flow Models

Two other successful and important normalizing flow models are *GLOW* and *FFJORD*. The following sections describe the key advancements they made.

GLOW

Presented at NeurIPS 2018, GLOW was one of the first models to demonstrate the ability of normalizing flows to generate high-quality samples and produce a meaningful latent space that can be traversed to manipulate samples. The key step was to replace the reverse masking setup with invertible 1×1 convolutional layers. For example, with RealNVP applied to images, the ordering of the channels is flipped after each step, to ensure that the network gets the chance to transform all of the input. In GLOW a 1×1 convolution is applied instead, which effectively acts as a general method to produce any permutation of the channels that the model desires. The authors show that even with this addition, the distribution as a whole remains tractable, with determinants and inverses that are easy to compute at scale.



Figure 6-14. Random samples from the GLOW model (source: Kingma and Dhariwal, 2018)[2]

FFJORD

RealNVP and GLOW are discrete time normalizing flows—that is, they transform the input through a discrete set of coupling layers. FFJORD (Free-Form Continuous Dynamics for Scalable Reversible Generative Models), presented at ICLR 2019, shows how it is possible to model the transformation as a continuous time process (i.e., by taking the limit as the number of steps in the flow tends to infinity and the step size tends to zero). In this case, the dynamics are modeled using an ordinary differential equation (ODE) whose parameters are produced by a neural network (f_θ). A black-box solver is used to solve the ODE at time t_1—i.e., to find z_1 given some initial point z_0 sampled from a Gaussian at t_0, as described by the following equations:

$$
\begin{aligned}
z_0 &\sim p(z_0) \\
\frac{\partial z(t)}{\partial t} &= f_\theta(x(t), t) \\
x &= z_1
\end{aligned}
$$

A diagram of the transformation process is shown in Figure 6-15.

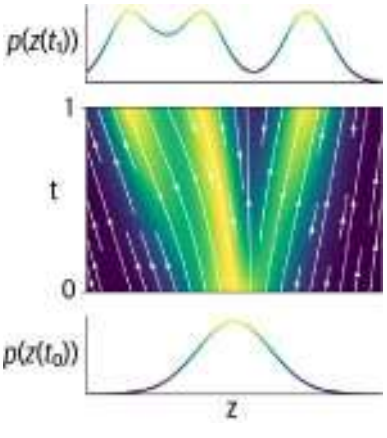


Figure 6-15. FFJORD models the transformation between the data distribution and a standard Gaussian via an ordinary differential equation, parameterized by a neural network (source: Will Grathwohl et al., 2018)[3]

Summary

In this chapter we explored normalizing flow models such as RealNVP, GLOW, and FFJORD.

A normalizing flow model is an invertible function defined by a neural network that allows us to directly model the data density via a change of variables. In the general case, the change of variables equation requires us to calculate a highly complex Jacobian determinant, which is impractical for all but the simplest of examples.

To sidestep this issue, the RealNVP model restricts the form of the neural network, such that it adheres to the two essential criteria: it is invertible and has a Jacobian determinant that is easy to compute.

It does this through stacking coupling layers, which produce scale and translation factors at each step. Importantly, the coupling layer masks the data as it flows through the network, in a way that ensures that the Jacobian is lower triangular and therefore has a simple-to-compute determinant. Full visibility of the input data is achieved through flipping the masks at each layer.

By design, the scale and translation operations can be easily inverted, so that once the model is trained it is possible to run data through the network in reverse. This means that we can target the forward transformation process toward a standard Gaussian, which we can easily sample from. We can then run the sampled points backward through the network to generate new observations.

The RealNVP paper also shows how it is possible to apply this technique to images, by using convolutions inside the coupling layers, rather than densely connected layers. The GLOW paper extended this idea to remove the necessity for any hardcoded permutation of the masks. The FFJORD model introduced the concept of continuous time normalizing flows, by modeling the transformation process as an ODE defined by a neural network.

Overall, we have seen how normalizing flows are a powerful generative modeling family that can produce high-quality samples, while maintaining the ability to tractably describe the data density function.

References

1. Laurent Dinh et al., "Density Estimation Using Real NVP," May 27, 2016, *https://arxiv.org/abs/1605.08803v3*.

2. Diedrick P. Kingma and Prafulla Dhariwal, "Glow: Generative Flow with Invertible 1x1 Convolutions," July 10, 2018, *https://arxiv.org/abs/1807.03039*.

3. Will Grathwohl et al., "FFJORD: Free-Form Continuous Dynamics for Scalable Reversible Generative Models," October 22, 2018, *https://arxiv.org/abs/1810.01367*.

CHAPTER 7

Energy-Based Models

Chapter Goals

In this chapter you will:

- Understand how to formulate a deep energy-based model (EBM).
- See how to sample from an EBM using Langevin dynamics.
- Train your own EBM using contrastive divergence.
- Analyze the EBM, including viewing snapshots of the Langevin dynamics sampling process.
- Learn about other types of EBM, such as restricted Boltzmann machines.

Energy-based models are a broad class of generative model that borrow a key idea from modeling physical systems—namely, that the probability of an event can be expressed using a Boltzmann distribution, a specific function that normalizes a real-valued energy function between 0 and 1. This distribution was originally formulated in 1868 by Ludwig Boltzmann, who used it to describe gases in thermal equilibrium.

In this chapter, we will see how we can use this idea to train a generative model that can be used to produce images of handwritten digits. We will explore several new concepts, including contrastive divergence for training the EBM and Langevin dynamics for sampling.

Introduction

We will begin with a short story to illustrate the key concepts behind energy-based models.

The Long-au-Vin Running Club

Diane Mixx was head coach of the long-distance running team in the fictional French town of Long-au-Vin. She was well known for her exceptional abilities as a trainer and had acquired a reputation for being able to turn even the most mediocre of athletes into world-class runners (Figure 7-1).



Figure 7-1. A running coach training some elite athletes (created with Midjourney)

Her methods were based around assessing the energy levels of each athlete. Over years of working with athletes of all abilities, she had developed an incredibly accurate sense of just how much energy a particular athlete had left after a race, just by looking at them. The lower an athlete's energy level, the better—elite athletes always gave everything they had during the race!

To keep her skills sharp, she regularly trained herself by measuring the contrast between her energy sensing abilities on known elite athletes and the best athletes from her club. She ensured that the divergence between her predictions for these two groups was as large as possible, so that people would take her seriously if she said that she had found a true elite athlete within her club.

The real magic was her ability to convert a mediocre runner into a top-class runner. The process was simple—she measured the current energy level of the athlete and worked out the optimal set of adjustments the athlete needed to make to improve their performance next time. Then, after making these adjustments, she measured the athlete's energy level again, looking for it to be slightly lower than before, explaining the improved performance on the track. This process of assessing the optimal adjustments and taking a small step in the right direction would continue until eventually the athlete was indistinguishable from a world-class runner.

After many years Diane retired from coaching and published a book on her methods for generating elite athletes—a system she branded the "Long-au-Vin, Diane Mixx" technique.

The story of Diane Mixx and the Long-au-Vin running club captures the key ideas behind energy-based modeling. Let's now explore the theory in more detail, before we implement a practical example using Keras.

Energy-Based Models

Energy-based models attempt to model the true data-generating distribution using a *Boltzmann distribution* (Equation 7-1) where $E(x)$ is know as the *energy function* (or *score*) of an observation x.

Equation 7-1. Boltzmann distribution

$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{\int_{\hat{\mathbf{x}} \in \mathbf{X}} e^{-E(\hat{\mathbf{x}})}}$$

In practice, this amounts to training a neural network $E(x)$ to output low scores for likely observations (so $p\mathbf{x}$ is close to 1) and high scores for unlikely observations (so $p\mathbf{x}$ is close to 0).

There are two challenges with modeling the data in this way. Firstly, it is not clear how we should use our model for sampling new observations—we can use it to generate a score given an observation, but how do we generate an observation that has a low score (i.e., a plausible observation)?

Secondly, the normalizing denominator of Equation 7-1 contains an integral that is intractable for all but the simplest of problems. If we cannot calculate this integral, then we cannot use maximum likelihood estimation to train the model, as this requires that $p\mathbf{x}$ is a valid probability distribution.

The key idea behind an energy-based model is that we can use approximation techniques to ensure we never need to calculate the intractable denominator. This is in contrast to, say, a normalizing flow, where we go to great lengths to ensure that the transformations that we apply to our standard Gaussian distribution do not change the fact that the output is still a valid probability distribution.

We sidestep the tricky intractable denominator problem by using a technique called contrastive divergence (for training) and a technique called Langevin dynamics (for sampling), following the ideas from Du and Mordatch's 2019 paper "Implicit

Generation and Modeling with Energy-Based Models."[1] We shall explore these techniques in detail while building our own EBM later in the chapter.

First, let's get set up with a dataset and design a simple neural network that will represent our real-valued energy function $E(x)$.



Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/07_ebm/01_ebm/ebm.ipynb* in the book repository.

The code is adapted from the excellent tutorial on deep energy-based generative models by Phillip Lippe.

The MNIST Dataset

We'll be using the standard MNIST dataset, consisting of grayscale images of handwritten digits. Some example images from the dataset are shown in Figure 7-2.

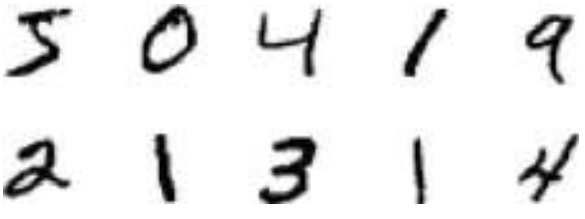


Figure 7-2. Examples of images from the MNIST dataset

The dataset comes prepackaged with TensorFlow, so it can be downloaded as shown in Example 7-1.

Example 7-1. Loading the MNIST dataset

```
from tensorflow.keras import datasets
(x_train, _), (x_test, _) = datasets.mnist.load_data()
```

As usual, we'll scale the pixel values to the range [-1, 1] and add some padding to make the images 32 × 32 pixels in size. We also convert it to a TensorFlow Dataset, as shown in Example 7-2.

Example 7-2. Preprocessing the MNIST dataset

```
def preprocess(imgs):
    imgs = (imgs.astype("float32") - 127.5) / 127.5
    imgs = np.pad(imgs , ((0,0), (2,2), (2,2)), constant_values= -1.0)
```

```
    imgs = np.expand_dims(imgs, -1)
    return imgs

x_train = preprocess(x_train)
x_test = preprocess(x_test)
x_train = tf.data.Dataset.from_tensor_slices(x_train).batch(128)
x_test = tf.data.Dataset.from_tensor_slices(x_test).batch(128)
```

Now that we have our dataset, we can build the neural network that will represent our energy function $E(x)$.

The Energy Function

The energy function $E_\theta(x)$ is a neural network with parameters θ that can transform an input image x into a scalar value. Throughout this network, we make use of an activation function called *swish*, as described in the following sidebar.

Swish Activation

Swish is an alternative to ReLU that was introduced by Google in 2017[2] and is defined as follows:

$$\text{swish}(x) = x \cdot \text{sigmoid}(x) = \frac{x}{e^{-x} + 1}$$

Swish is visually similar to ReLU, with the key difference being that it is smooth, which helps to alleviate the vanishing gradient problem. This is particularly important for energy-based models. A plot of the swish function is shown in Figure 7-3.

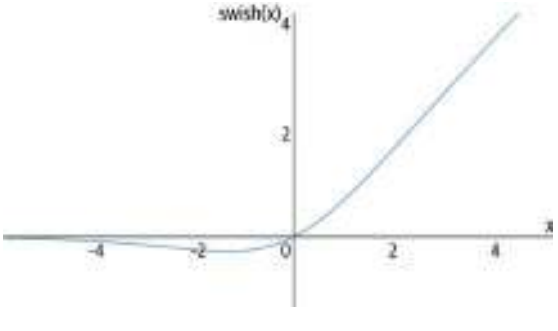


Figure 7-3. The swish activation function

The network is a set of stacked `Conv2D` layers that gradually reduce the size of the image while increasing the number of channels. The final layer is a single fully connected unit with linear activation, so the network can output values in the range $(-\infty, \infty)$. The code to build it is given in Example 7-3.

Example 7-3. Building the energy function $E(x)$ neural network

```
ebm_input = layers.Input(shape=(32, 32, 1))
x = layers.Conv2D(
    16, kernel_size=5, strides=2, padding="same", activation = activations.swish
)(ebm_input) ❶
x = layers.Conv2D(
    32, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Conv2D(
    64, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Conv2D(
    64, kernel_size=3, strides=2, padding="same", activation = activations.swish
)(x)
x = layers.Flatten()(x)
x = layers.Dense(64, activation = activations.swish)(x)
ebm_output = layers.Dense(1)(x) ❷
model = models.Model(ebm_input, ebm_output) ❸
```

❶ The energy function is a set of stacked `Conv2D` layers, with swish activation.

❷ The final layer is a single fully connected unit, with a linear activation function.

❸ A Keras `Model` that converts the input image into a scalar energy value.

Sampling Using Langevin Dynamics

The energy function only outputs a score for a given input—how can we use this function to generate new samples that have a low energy score?

We will use a technique called *Langevin dynamics*, which makes use of the fact that we can compute the gradient of the energy function with respect to its input. If we start from a random point in the sample space and take small steps in the opposite direction of the calculated gradient, we will gradually reduce the energy function. If our neural network is trained correctly, then the random noise should transform into an image that resembles an observation from the training set before our eyes!



Stochastic Gradient Langevin Dynamics

Importantly, we must also add a small amount of random noise to the input as we travel across the sample space; otherwise, there is a risk of falling into local minima. The technique is therefore known as stochastic gradient Langevin dynamics.[3]

We can visualize this gradient descent as shown in Figure 7-4, for a two-dimensional space with the energy function value on the third dimension. The path is a noisy descent downhill, following the negative gradient of the energy function $E(x)$ with respect to the input x. In the MNIST image dataset, we have 1,024 pixels so are navigating a 1,024-dimensional space, but the same principles apply!

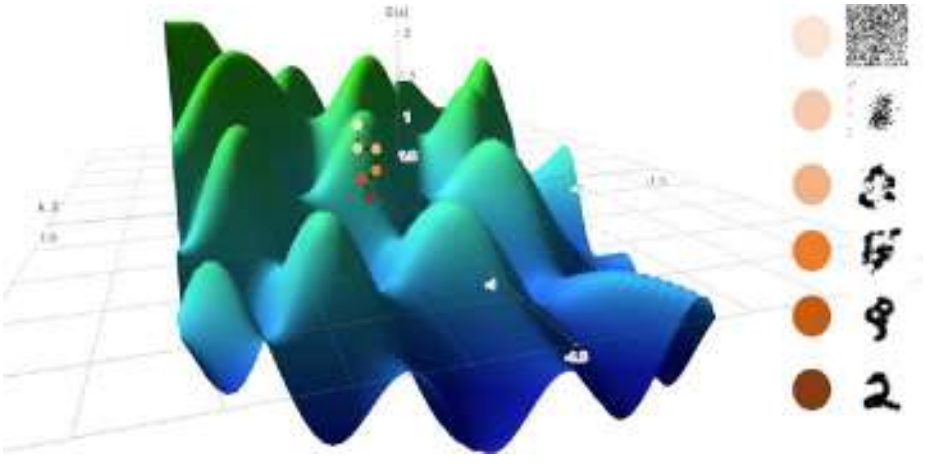


Figure 7-4. Gradient descent using Langevin dynamics

It is worth noting the difference between this kind of gradient descent and the kind of gradient descent we normally use to train a neural network.

When training a neural network, we calculate the gradient of the *loss function* with respect to the *parameters* of the network (i.e., the weights) using backpropagation. Then we update the parameters a small amount in the direction of the negative gradient, so that over many iterations, we gradually minimize the loss.

With Langevin dynamics, we keep the neural network weights *fixed* and calculate the gradient of the *output* with respect to the *input*. Then we update the input a small amount in the direction of the negative gradient, so that over many iterations, we gradually minimize the output (the energy score).

Both processes utilize the same idea (gradient descent), but are applied to different functions and with respect to different entities.

Formally, Langevin dynamics can be described by the following equation:

$$x^k = x^{k-1} - \eta \nabla_x E_\theta\left(x^{k-1}\right) + \omega$$

where $\omega \sim \mathcal{N}(0, \sigma)$ and $x^0 \sim \mathcal{U}(-1,1)$. η is the step size hyperparameter that must be tuned—too large and the steps jump over minima, too small and the algorithm will be too slow to converge.



$x^0 \sim \mathcal{U}(-1,1)$ is the uniform distribution on the range [–1, 1].

We can code up our Langevin sampling function as illustrated in Example 7-4.

Example 7-4. The Langevin sampling function

```
def generate_samples(model, inp_imgs, steps, step_size, noise):
    imgs_per_step = []
    for _ in range(steps): ❶
        inp_imgs += tf.random.normal(inp_imgs.shape, mean = 0, stddev = noise) ❷
        inp_imgs = tf.clip_by_value(inp_imgs, -1.0, 1.0)
        with tf.GradientTape() as tape:
            tape.watch(inp_imgs)
            out_score = -model(inp_imgs) ❸
        grads = tape.gradient(out_score, inp_imgs) ❹
        grads = tf.clip_by_value(grads, -0.03, 0.03)
        inp_imgs += -step_size * grads ❺
        inp_imgs = tf.clip_by_value(inp_imgs, -1.0, 1.0)
        return inp_imgs
```

❶ Loop over given number of steps.

❷ Add a small amount of noise to the image.

❸ Pass the image through the model to obtain the energy score.

❹ Calculate the gradient of the output with respect to the input.

❺ Add a small amount of the gradient to the input image.

Training with Contrastive Divergence

Now that we know how to sample a novel low-energy point from the sample space, let's turn our attention to training the model.

We cannot apply maximum likelihood estimation, because the energy function does not output a probability; it outputs a score that does not integrate to 1 across the sample space. Instead, we will apply a technique first proposed in 2002 by Geoffrey Hinton, called *contrastive divergence*, for training unnormalized scoring models.[4]

The value that we want to minimize (as always) is the negative log-likelihood of the data:

$$\mathscr{L} = -\mathbb{E}_{x \sim \text{data}}\left[\log p_\theta(\mathbf{x})\right]$$

When $p_\theta(\mathbf{x})$ has the form of a Boltzmann distribution, with energy function $E_\theta(\mathbf{x})$, it can be shown that the gradient of this value can be written as follows (Oliver Woodford's "Notes on Contrastive Divergence" for the full derivation):[5]

$$\nabla_\theta \mathscr{L} = \mathbb{E}_{x \sim \text{data}}\left[\nabla_\theta E_\theta(\mathbf{x})\right] - \mathbb{E}_{x \sim \text{model}}\left[\nabla_\theta E_\theta(\mathbf{x})\right]$$

This intuitively makes a lot of sense—we want to train the model to output large negative energy scores for real observations and large positive energy scores for generated fake observations so that the contrast between these two extremes is as large as possible.

In other words, we can calculate the difference between the energy scores of real and fake samples and use this as our loss function.

To calculate the energy scores of fake samples, we would need to be able to sample exactly from the distribution $p_\theta(\mathbf{x})$, which isn't possible due to the intractable denominator. Instead, we can use our Langevin sampling procedure to generate a set of observations with low energy scores. The process would need to run for infinitely many steps to produce a perfect sample (which is obviously impractical), so instead we run for some small number of steps, on the assumption that this is good enough to produce a meaningful loss function.

We also maintain a buffer of samples from previous iterations, so that we can use this as the starting point for the next batch, rather than pure random noise. The code to produce the sampling buffer is shown in Example 7-5.

Example 7-5. The `Buffer`

```
class Buffer:
    def __init__(self, model):
        super().__init__()
        self.model = model
        self.examples = [
            tf.random.uniform(shape = (1, 32, 32, 1)) * 2 - 1
            for _ in range(128)
        ] ❶

    def sample_new_exmps(self, steps, step_size, noise):
        n_new = np.random.binomial(128, 0.05) ❷
        rand_imgs = (
            tf.random.uniform((n_new, 32, 32, 1)) * 2 - 1
        )
        old_imgs = tf.concat(
            random.choices(self.examples, k=128-n_new), axis=0
        ) ❸
        inp_imgs = tf.concat([rand_imgs, old_imgs], axis=0)
        inp_imgs = generate_samples(
            self.model, inp_imgs, steps=steps, step_size=step_size, noise = noise
        ) ❹
        self.examples = tf.split(inp_imgs, 128, axis = 0) + self.examples ❺
        self.examples = self.examples[:8192]
        return inp_imgs
```

❶ The sampling buffer is initialized with a batch of random noise.

❷ On average, 5% of observations are generated from scratch (i.e., random noise) each time.

❸ The rest are pulled at random from the existing buffer.

❹ The observations are concatenated and run through the Langevin sampler.

❺ The resulting sample is added to the buffer, which is trimmed to a max length of 8,192 observations.

Figure 7-5 shows one training step of contrastive divergence. The scores of real observations are pushed down by the algorithm and the scores of fake observations are pulled up, without caring about normalizing these scores after each step.

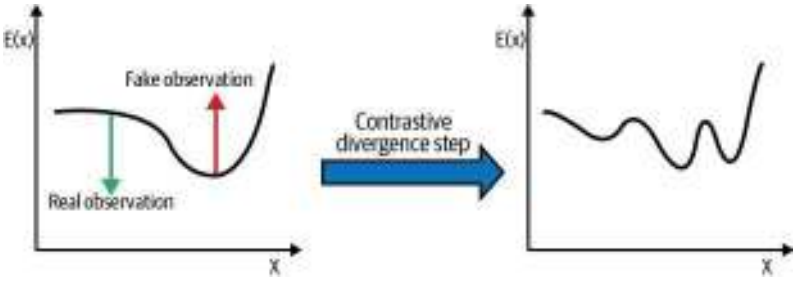


Figure 7-5. One step of contrastive divergence

We can code up the training step of the contrastive divergence algorithm within a custom Keras model as shown in Example 7-6.

Example 7-6. EBM trained using contrastive divergence

```
class EBM(models.Model):
    def __init__(self):
        super(EBM, self).__init__()
        self.model = model
        self.buffer = Buffer(self.model)
        self.alpha = 0.1
        self.loss_metric = metrics.Mean(name="loss")
        self.reg_loss_metric = metrics.Mean(name="reg")
        self.cdiv_loss_metric = metrics.Mean(name="cdiv")
        self.real_out_metric = metrics.Mean(name="real")
        self.fake_out_metric = metrics.Mean(name="fake")

    @property
    def metrics(self):
        return [
            self.loss_metric,
            self.reg_loss_metric,
            self.cdiv_loss_metric,
            self.real_out_metric,
            self.fake_out_metric
        ]

    def train_step(self, real_imgs):
        real_imgs += tf.random.normal(
            shape=tf.shape(real_imgs), mean = 0, stddev = 0.005
        ) ❶
        real_imgs = tf.clip_by_value(real_imgs, -1.0, 1.0)
        fake_imgs = self.buffer.sample_new_exmps(
            steps=60, step_size=10, noise = 0.005
        ) ❷
        inp_imgs = tf.concat([real_imgs, fake_imgs], axis=0)
        with tf.GradientTape() as training_tape:
```

```
            real_out, fake_out = tf.split(self.model(inp_imgs), 2, axis=0) ❸
            cdiv_loss = tf.reduce_mean(fake_out, axis = 0) - tf.reduce_mean(
                real_out, axis = 0
            ) ❹
            reg_loss = self.alpha * tf.reduce_mean(
                real_out ** 2 + fake_out ** 2, axis = 0
            ) ❺
            loss = reg_loss + cdiv_loss
        grads = training_tape.gradient(loss, self.model.trainable_variables) ❻
        self.optimizer.apply_gradients(
            zip(grads, self.model.trainable_variables)
        )
        self.loss_metric.update_state(loss)
        self.reg_loss_metric.update_state(reg_loss)
        self.cdiv_loss_metric.update_state(cdiv_loss)
        self.real_out_metric.update_state(tf.reduce_mean(real_out, axis = 0))
        self.fake_out_metric.update_state(tf.reduce_mean(fake_out, axis = 0))
        return {m.name: m.result() for m in self.metrics}

    def test_step(self, real_imgs): ❼
        batch_size = real_imgs.shape[0]
        fake_imgs = tf.random.uniform((batch_size, 32, 32, 1)) * 2 - 1
        inp_imgs = tf.concat([real_imgs, fake_imgs], axis=0)
        real_out, fake_out = tf.split(self.model(inp_imgs), 2, axis=0)
        cdiv = tf.reduce_mean(fake_out, axis = 0) - tf.reduce_mean(
            real_out, axis = 0
        )
        self.cdiv_loss_metric.update_state(cdiv)
        self.real_out_metric.update_state(tf.reduce_mean(real_out, axis = 0))
        self.fake_out_metric.update_state(tf.reduce_mean(fake_out, axis = 0))
        return {m.name: m.result() for m in self.metrics[2:]}

ebm = EBM()
ebm.compile(optimizer=optimizers.Adam(learning_rate=0.0001), run_eagerly=True)
ebm.fit(x_train, epochs=60, validation_data = x_test,)
```

❶ A small amount of random noise is added to the real images, to avoid the model overfitting to the training set.

❷ A set of fake images are sampled from the buffer.

❸ The real and fake images are run through the model to produce real and fake scores.

❹ The contrastive divergence loss is simply the difference between the scores of real and fake observations.

❺ A regularization loss is added to avoid the scores becoming too large.